Graph Representation Learning

MDS

Motivation: Creating a Classifier

The goal is to generate a **label** $y \in Y$ given some **input data** X using a **classification algorithm**:

x_1	x_2	x_3	x_4
Age	**Salary**	**# Depend.**	**Credit Limit**
25	25.000	0	1.000
35	32.000	1	1.800
31	38.000	1	1.500
48	52.000	2	2.200

Y = [Accept, Decline]

Logistic Regression

Neural Network

Decision Tree

Motivation: Creating a Classifier

Depending on the input data X, the set of **features** has to be **generated** which may not be trivial.

x_1	x_2	x_3	x_4
Age	**Salary**	**# Depend.**	**Credit Limit**
25	25.000	0	1.000
35	32.000	1	1.800
31	38.000	1	1.500
48	52.000	2	2.200

Y = [Accept, Decline]

Logistic Regression

Neural Network

Decision Tree

Motivation: Creating a Classifier

Let’s imagine we want to create a classifier to detect **spam emails**:

“Win a brand new iPhone! Click here now!!!”

“As discussed in the previous meeting […]. Best”

“Limited offer, claim your REWARD today”

“Dear Alice, the meeting is postponed to Friday”

Motivation: Creating a Classifier

Can you think of **features** that could be useful to classify these emails?

"Win a brand new iPhone! Click here now!!!"

"As discussed in the previous meeting […]. Best"

"Limited offer, claim your REWARD today"

"Dear Alice, the meeting is postponed to Friday"

- *Presence of **word** like: 'free', 'win', 'reward'…*
- *Presence of **links***
- ***Length** of the email*
- *Ratio of **capital letters***
- *Sender **domain***
- *Number of **exclamation marks***
- *…*

Motivation: Creating a Classifier

Let's imagine we want to create a classifier to detect **spam emails**:

"Win a brand new iPhone! Click here now!!!"

"As discussed in the previous meeting […]. Best"

"Limited offer, claim your REWARD today"

"Dear Alice, the meeting is postponed to Friday"

P. Words	Links	Length	…	T. Domain
1	1	8	…	0
0	0	50	…	0
1	1	6	…	1
0	0	8	…	0

Logistic Regression

Neural Network

Decision Tree

Motivation: Creating a Classifier

This approach has some **limitations**:

1. Did we select an **appropriate or complete set of features**?

   - *Presence of **word** like: 'free', 'win', 'reward'…* → Is the list complete? Does it contain False Positives?

   - ***Length** of the email* → *Is length really a distinctive feature of spam emails?*

Motivation: Creating a Classifier

This approach has some **limitations**:

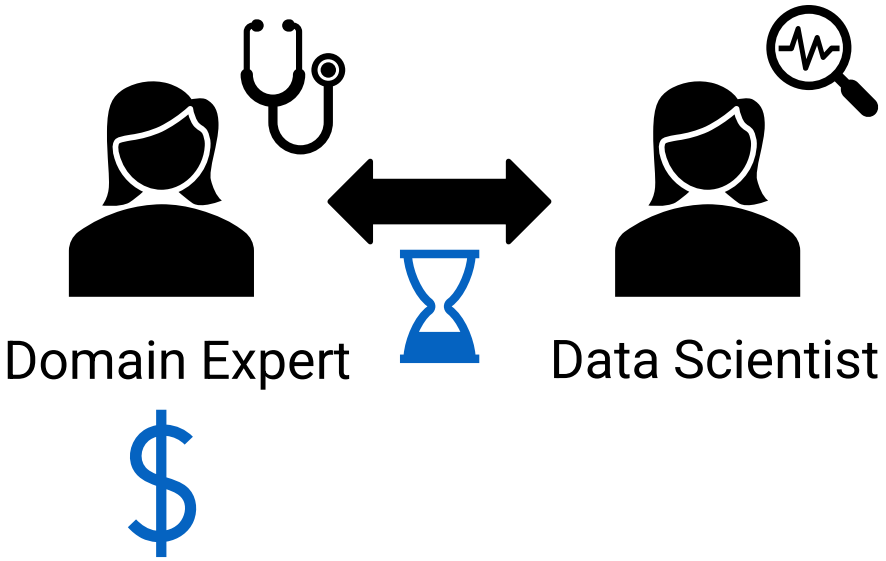
2. Does it require extensive data preprocessing and data engineering task, even requiring **human involvement**?

- ***Length*** *of the tail*
- ***Shape*** *of the ears*
- ***Size***
- *Presence of* ***whiskers***
- ***Fur*** *texture*

Motivation: Creating a Classifier

This approach has some **limitations**:

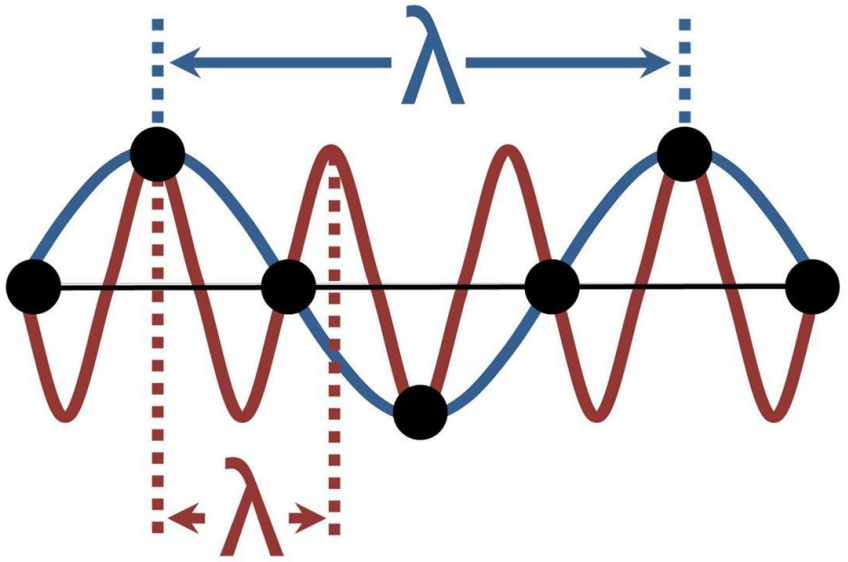
3. Do we (data scientist/data engineers) even have the **knowledge** to design or preprocess the data?



Motivation: Creating a Classifier

This approach has some **limitations**:

3. Do we (data scientist/data engineers) even have the **knowledge** to design or preprocess the data?



Motivation: Creating a Classifier

This approach has some **limitations**:

4. Are the designed features **biased**?

   - Data bias
   - Time bias
   - Cultural bias
   - Socio-economic bias

Motivation: Creating a Classifier

This approach has some **limitations**:

5. Are we able to **generalize** with the selected features?

We need to translate them to other languages

"Win a brand new iPhone! Click here now!!!"

"As discussed in the previous meeting […]. Best"

"Limited offer, claim your REWARD today"

"Dear Alice, the meeting is postponed to Friday"

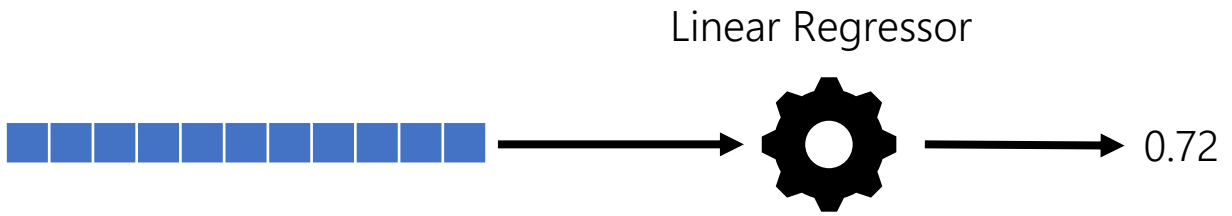
- *Presence of **word** like: 'free', 'win', 'reward'…*
- *Presence of **links***
- ***Length** of the email*
- *Ratio of **capital letters***
- *Sender **domain***
- *Number of **exclamation marks***
- …

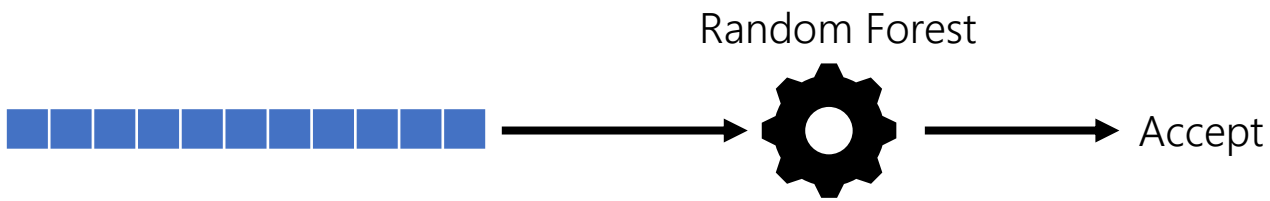
Arabic? Chinese?

Motivation:

Therefore, given that:

- Typically, **Machine Learning** models rely on vectors as their **input data**.



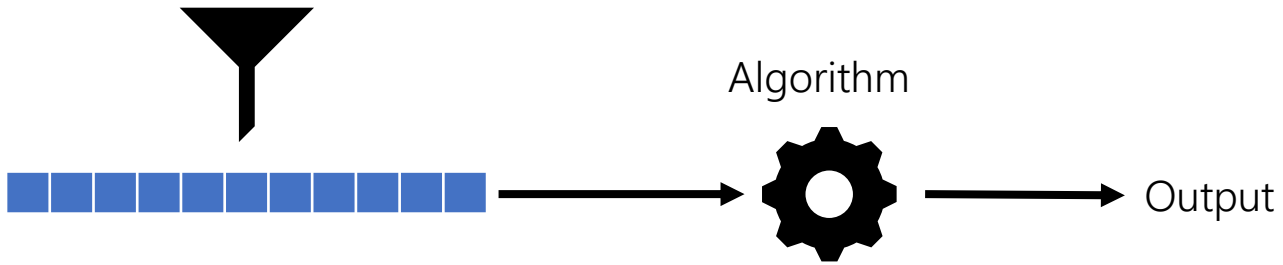


Motivation:

Therefore, given that:

- Typically, **Machine Learning** models rely on vectors as their **input data**. These is straightforward for **tabular** data, but can be problematic for **other data representations**.

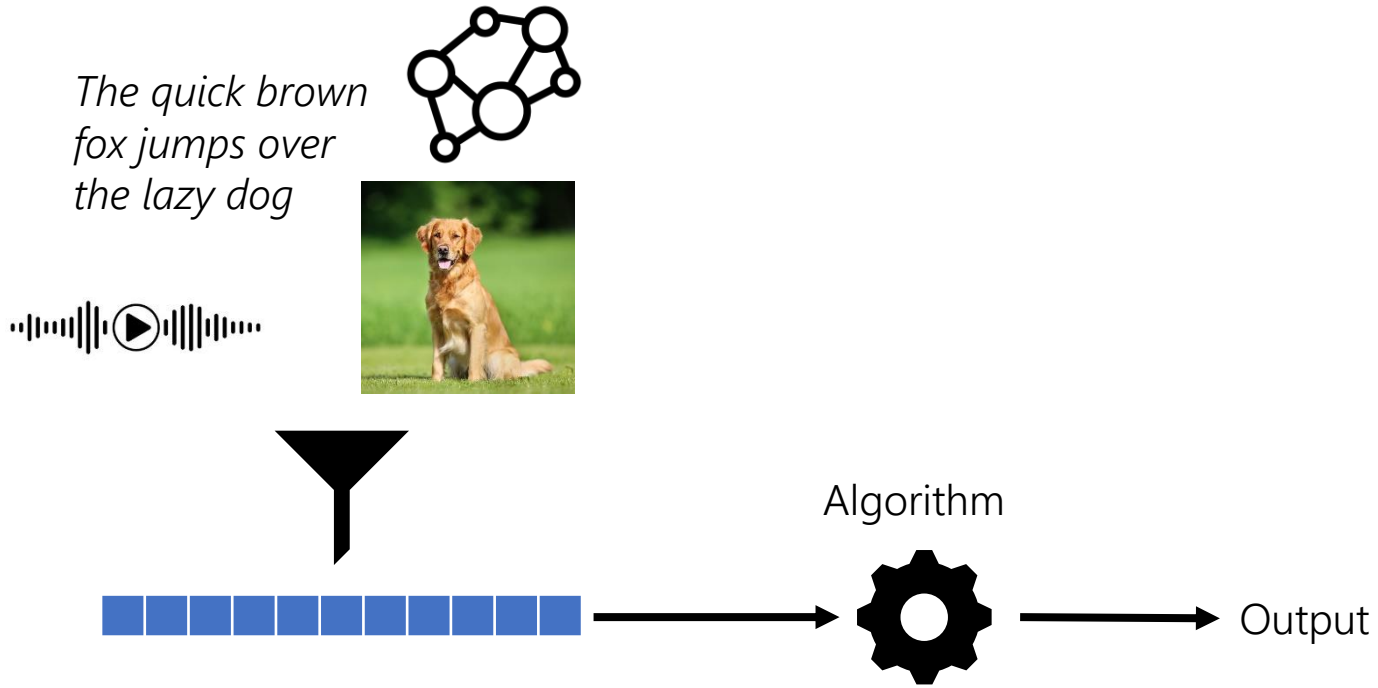
Column 1	Column 2	Column 3	Column 4	Column 5	Column 6
98	0,4	High	1	56,4	1,32
76	0,7	Low	5	43,7	1,11
45	0,8	Low	7	94,2	0,78
…	…	…	…	…	…



Motivation:

Therefore, given that:

- Typically, **Machine Learning** models rely on vectors as their **input data**. These is straightforward for **tabular** data, but can be problematic for **other data representations**.



Motivation:

Therefore, given that:

- Typically, **Machine Learning** models rely on vectors as their **input data**. These is straightforward for **tabular** data, but can be problematic for **other data representations**.
- Generating these vectors (i.e., extracting features) can be **labor intensive**, **expensive**, **incomplete**, **biased** and/or **poorly generalizable**.
- The **performance** of ML models is **heavily dependent** on the choice of the input data representation, often more than the choice of the **learning algorithm** itself.

Motivation:

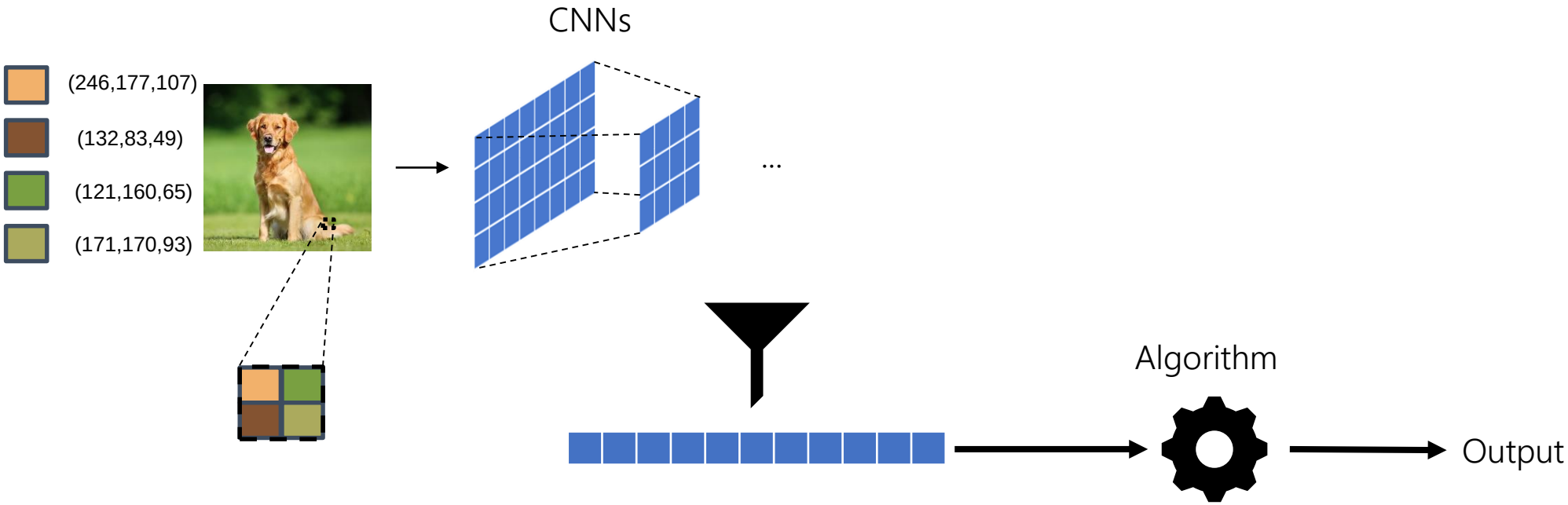
Therefore, given that:

- Typically, **Machine Learning** models rely on vectors as their **input data**. These is straightforward for **tabular** data, but can be problematic for **other data representations**.
- Generating these vectors (i.e., extracting features) can be **labor intensive**, **expensive**, **incomplete**, **biased** and/or **poorly generalizable**.
- The **performance** of ML models is **heavily dependent** on the choice of the input data representation, often more than the choice of the **learning algorithm** itself.

Efforts have made towards generating optimal **machine learned** representations of different data objects (known as **embeddings**), allowing to excel in the **downstream task** (e.g., classification).

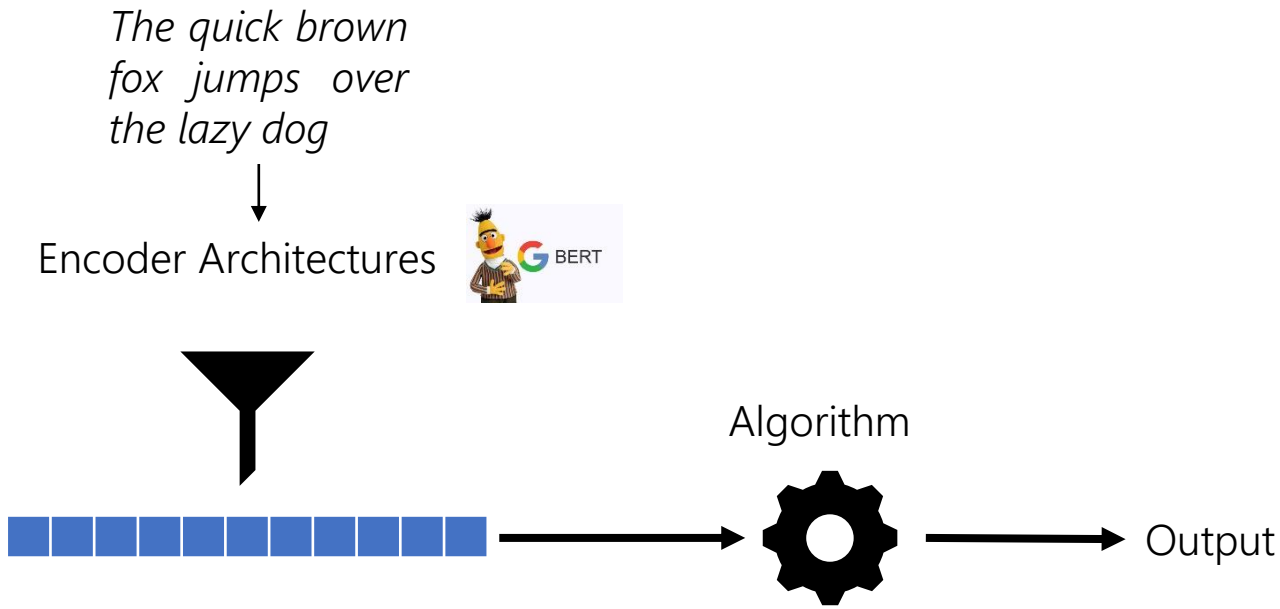
Motivation

These transformations from data to vectors are straightforward when the input is **tabular**, but can be problematic for **other data representations**.



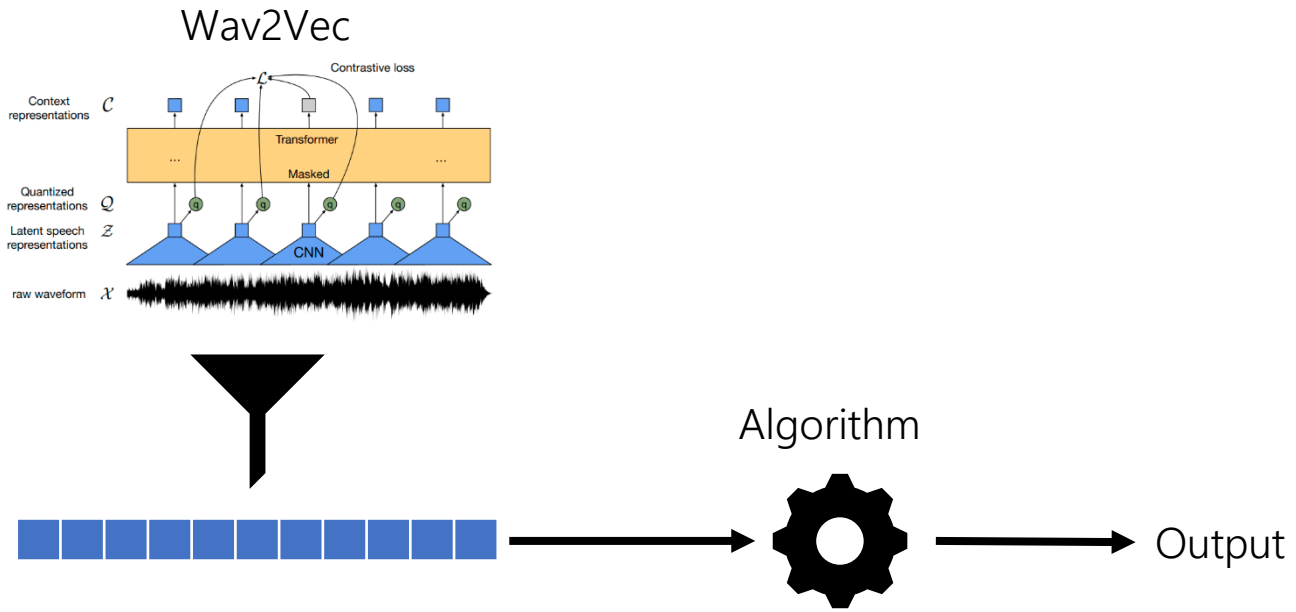
Motivation

These transformations from data to vectors are straightforward when the input is **tabular**, but can be problematic for **other data representations**.



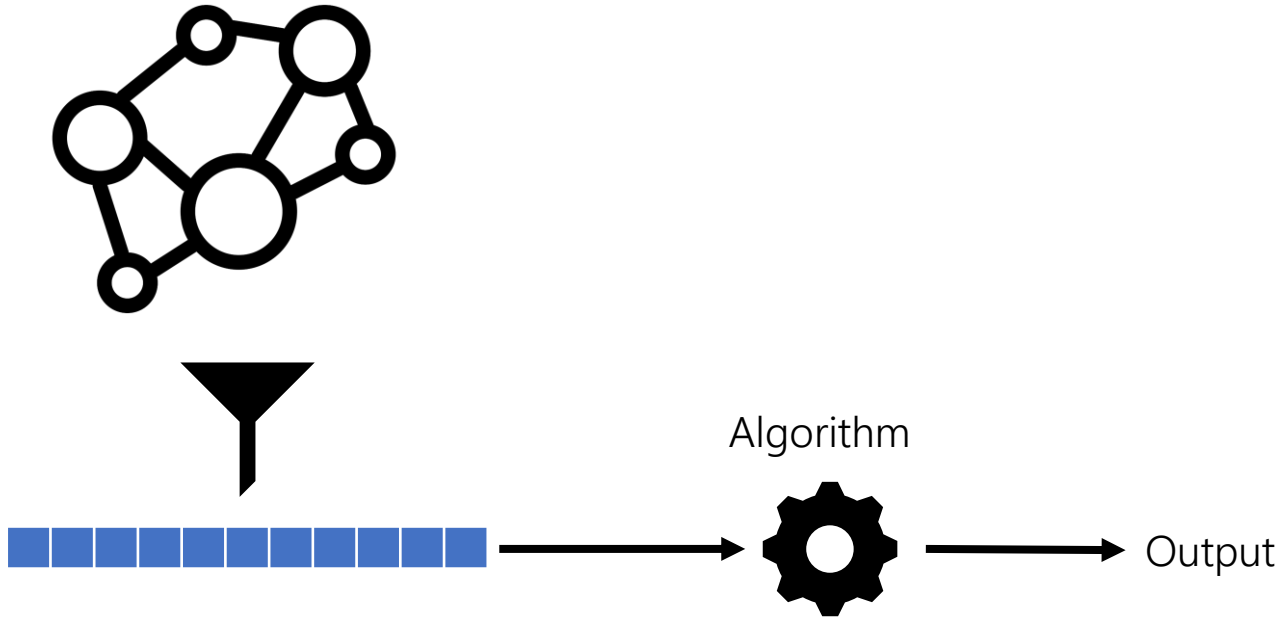
Motivation

These transformations from data to vectors are straightforward when the input is **tabular**, but can be problematic for **other data representations**.



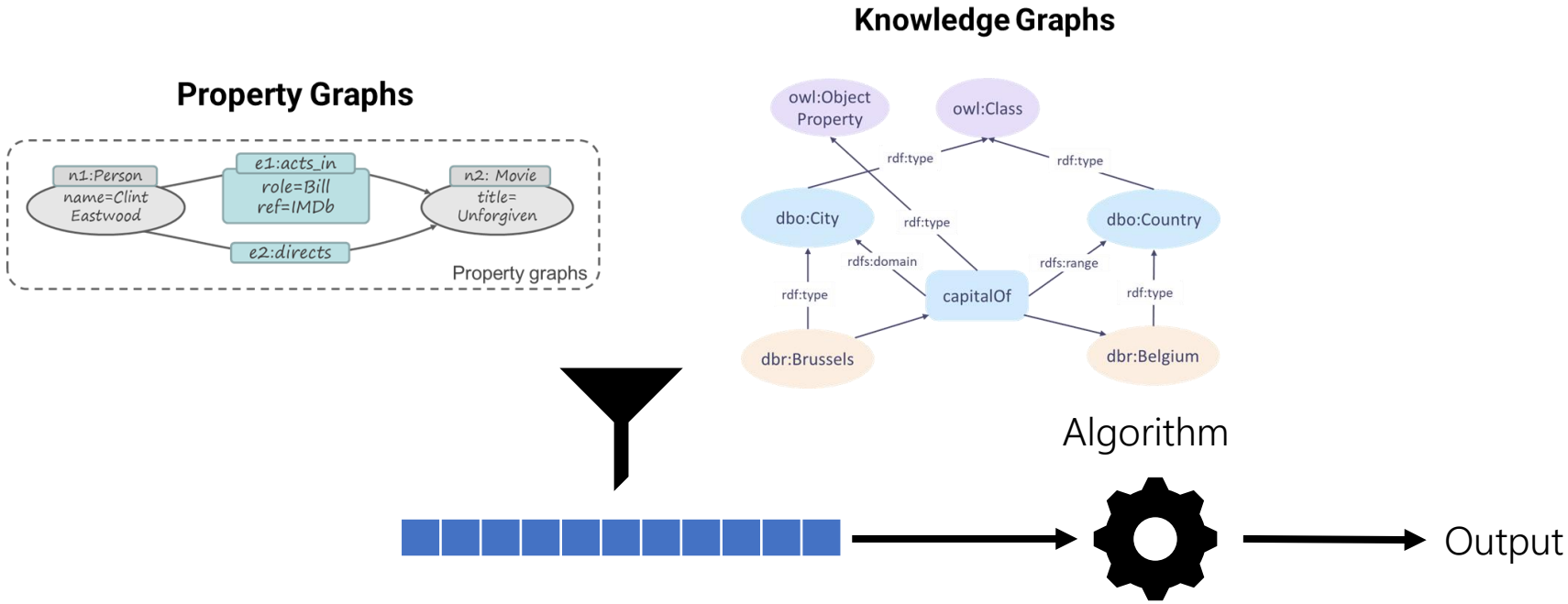
Motivation

These transformations from data to vectors are straightforward when the input is **tabular**, but can be problematic for **other data representations**.



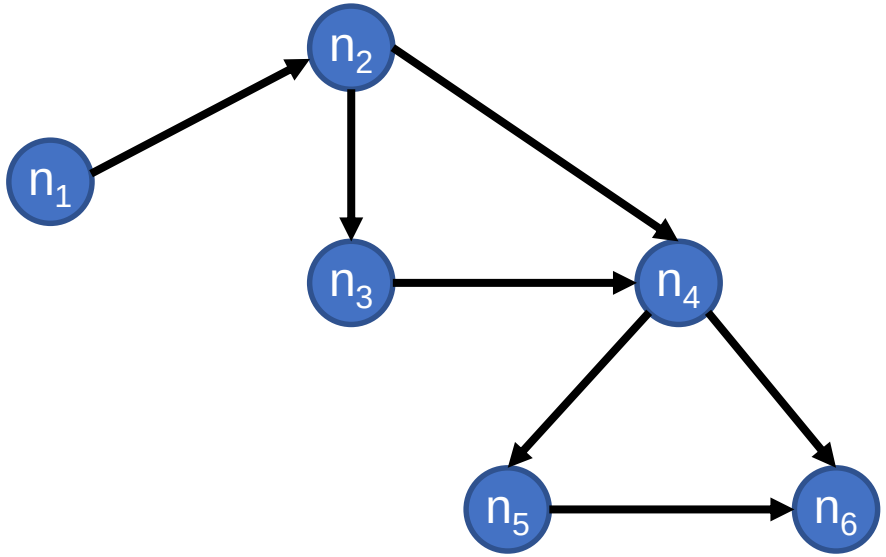
Motivation

These transformations from data to vectors are straightforward when the input is **tabular**, but can be problematic for **other data representations**.



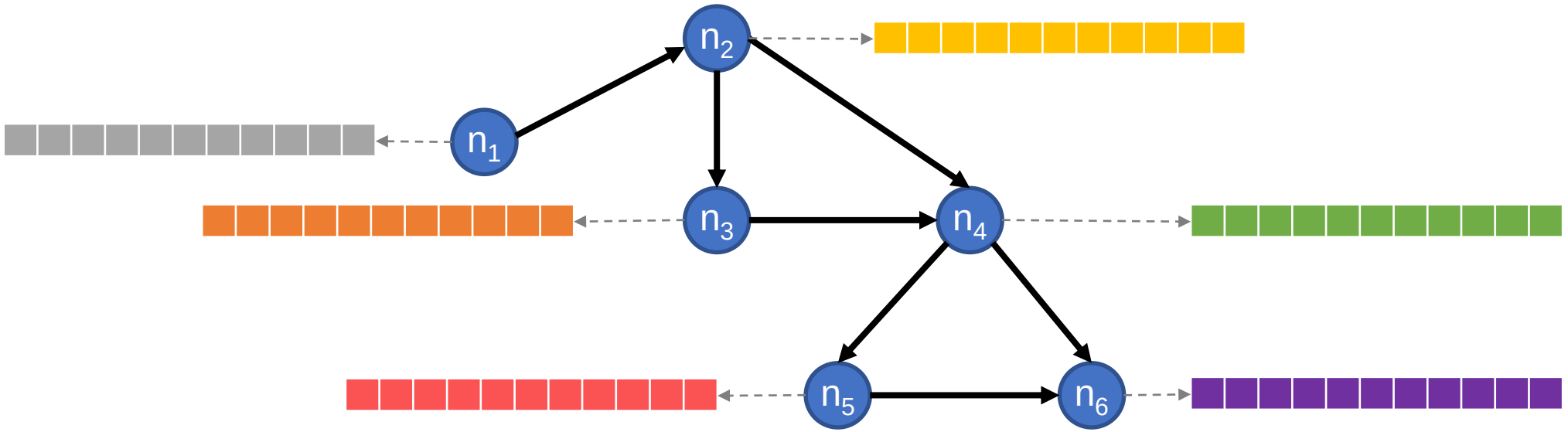
Goal:

Given a **graph**, we want to obtain **numerical representations** for its elements (e.g., nodes, edges), allowing us to use them for **downstream tasks** (e.g., clustering, classification, retrieval…), which are more expressive/versatile/scalable than **existing graph algorithms**.



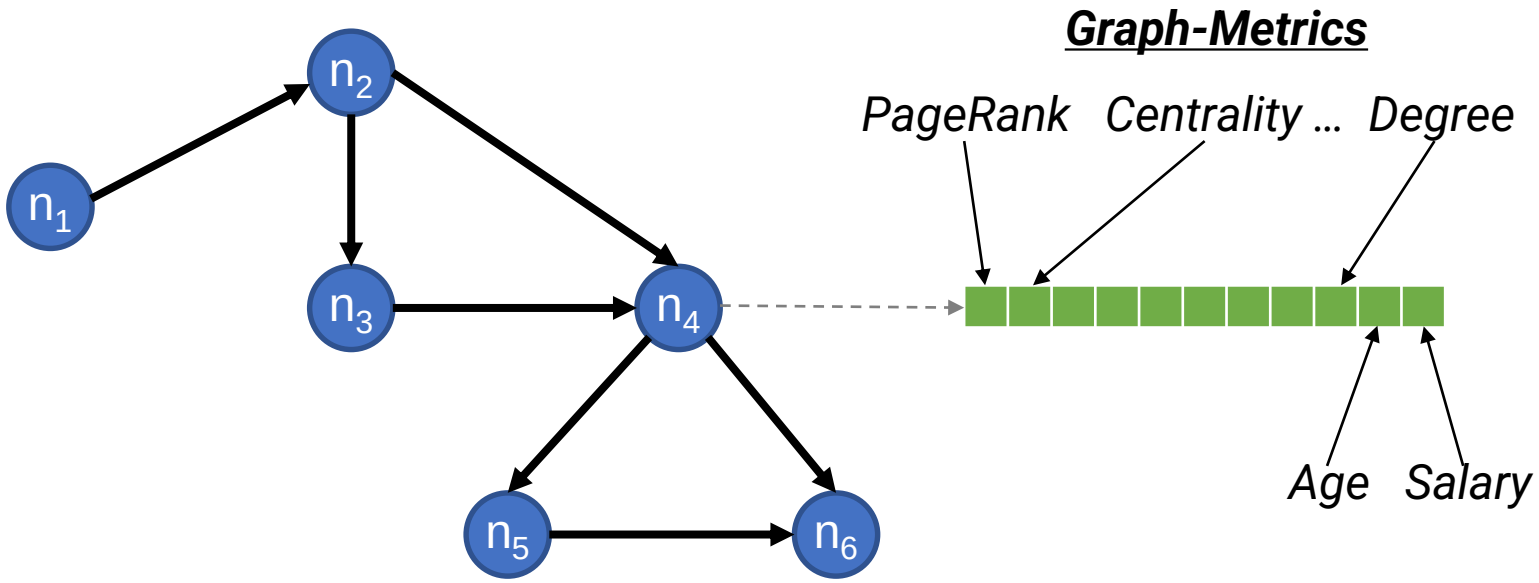
Goal:

Given a **graph**, we want to obtain **numerical representations** for its elements (e.g., nodes, edges), allowing us to use them for **downstream tasks** (e.g., clustering, classification, retrieval…), which are more expressive/versatile/scalable than **existing graph algorithms**.



Feature-based:

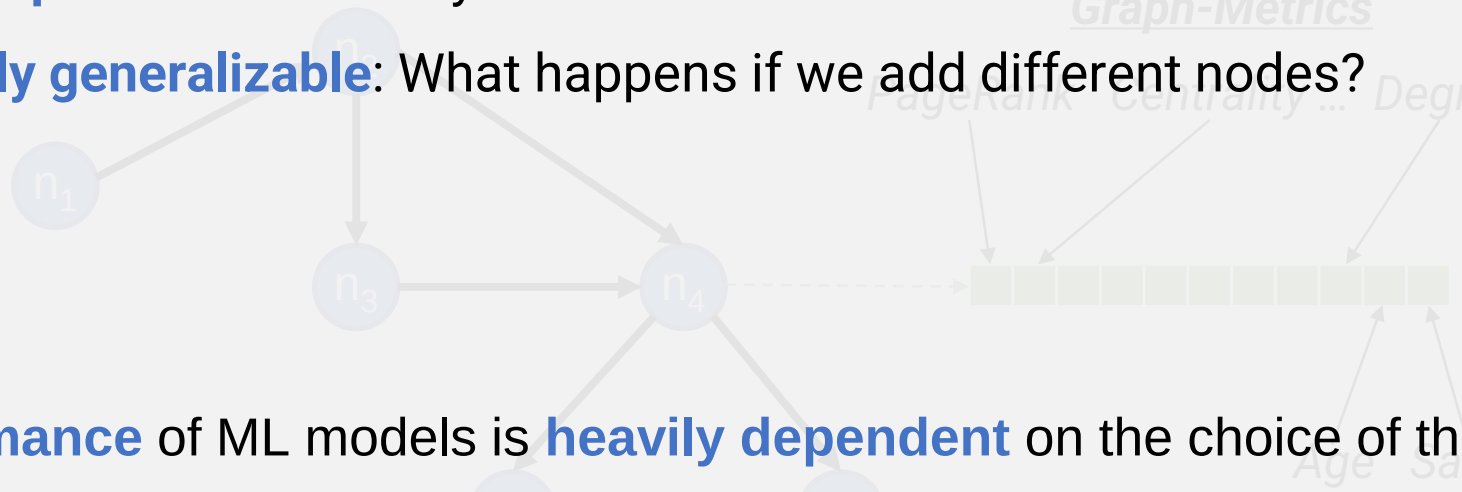
The most intuitive approach is based on **generating features** for the nodes (e.g., graph based metrics) or **leveraging existing ones** (e.g., properties).



LIMITATIONS

Generating these vectors can be:

- **Biased**: Depending on the “expert” who decided which features to include
- **Incomplete**: Did we really use the best features?
- **Poorly generalizable**: What happens if we add different nodes?
- **…**

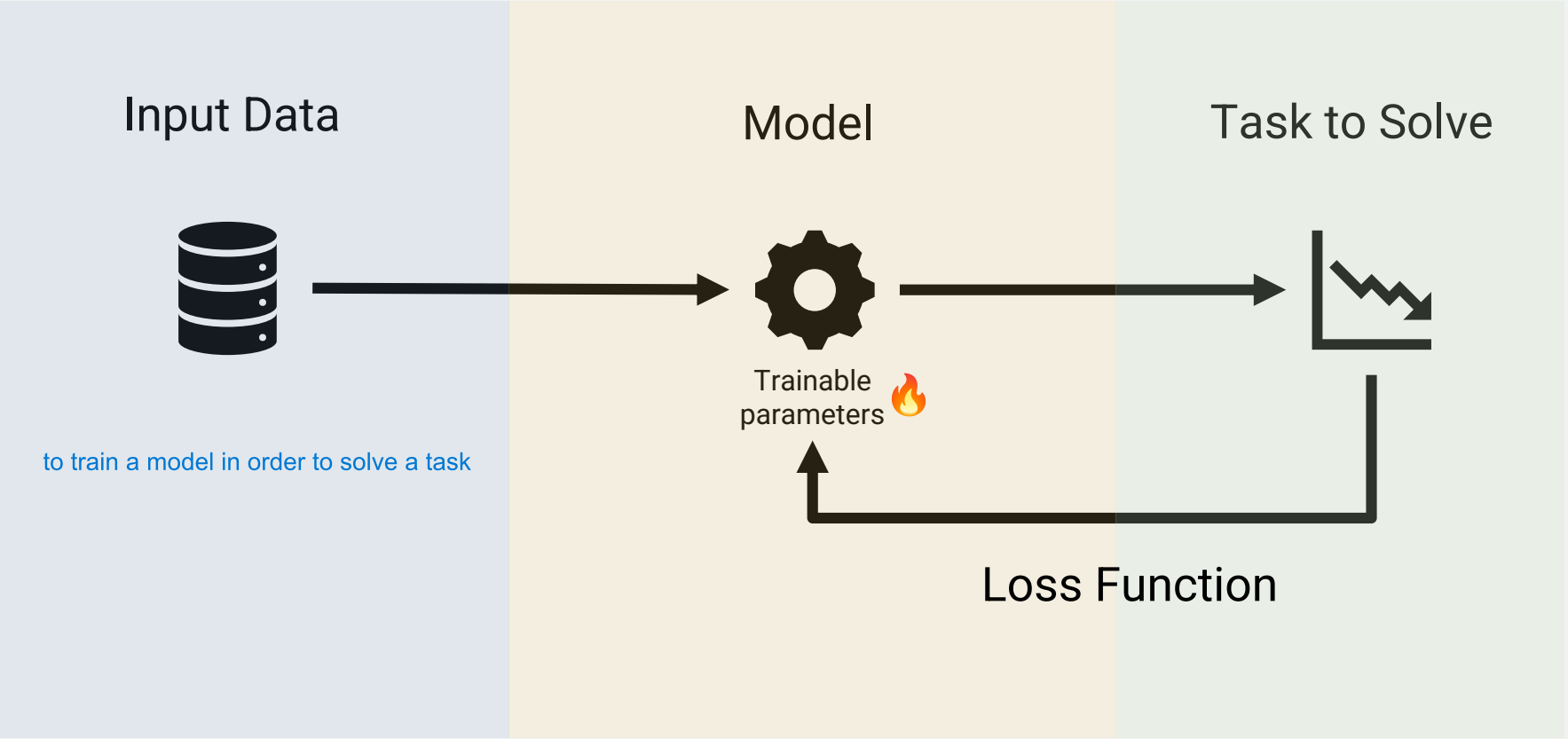


The **performance** of ML models is **heavily dependent** on the choice of the input data representation, often more than the choice of the **learning algorithm** itself.

Efforts have made towards generating optimal **machine learned** representations (**embeddings**), aiming to excel in the **downstream task** (e.g., classification).

Neural Networks/Deep Learning: 101

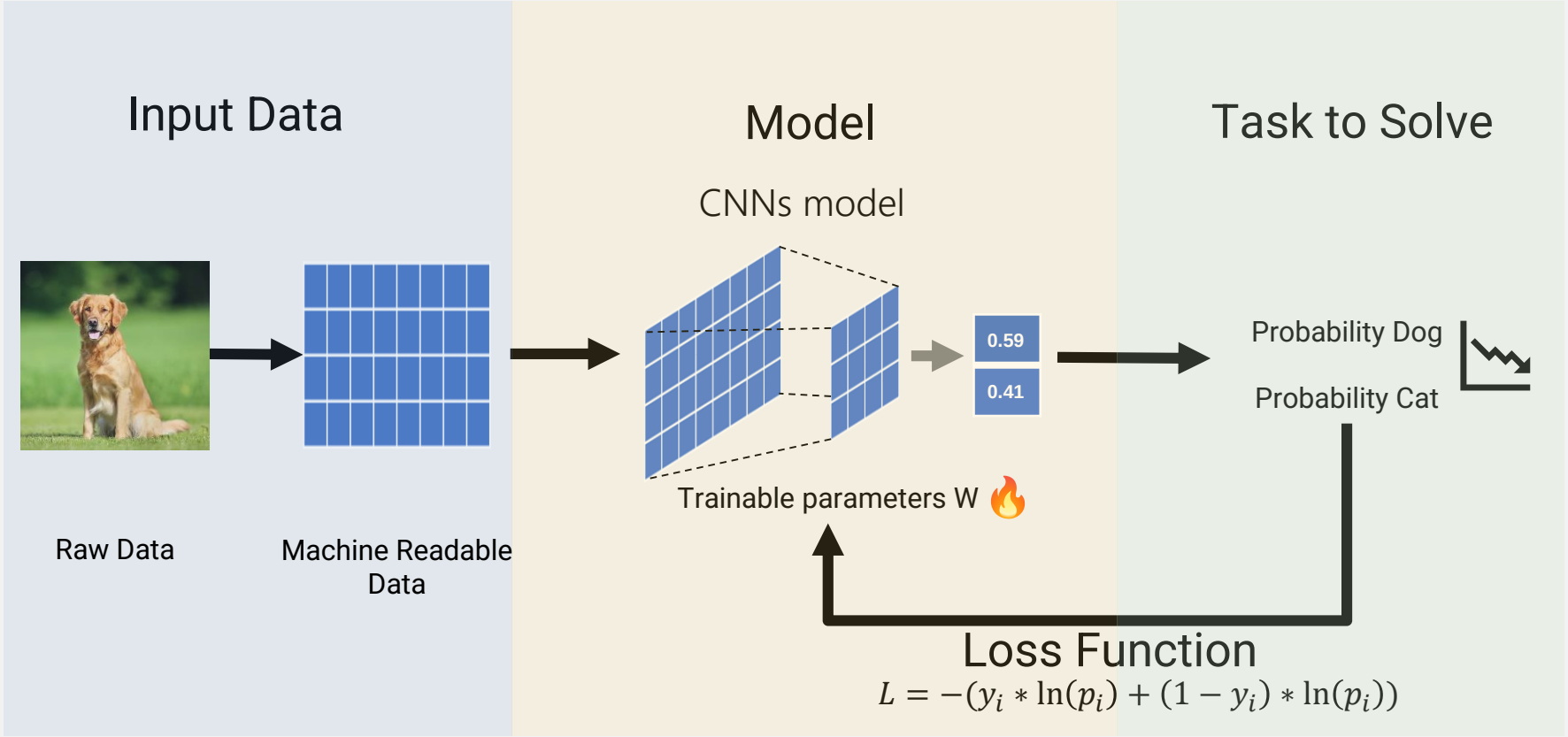
Typically, to train a **Deep Learning model** we need:



if we work with log reg y= beta*x1 + gamma*x2 + alfa las x1 y x2 sn input y beta, gamma y alfa a entrenar (ajustarse)

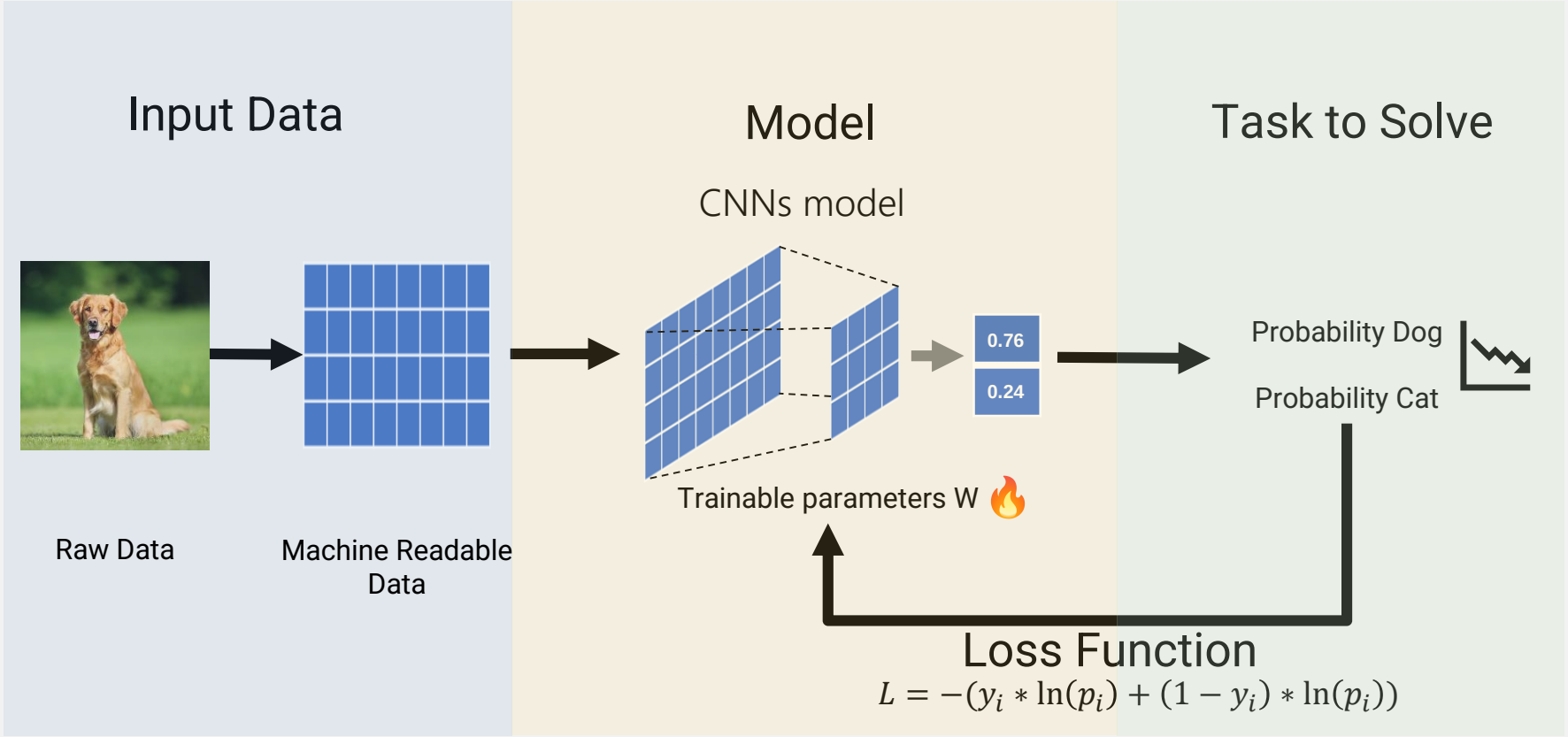
Neural Networks/Deep Learning: 101

Let's imagine a cat and dog **classifier**:



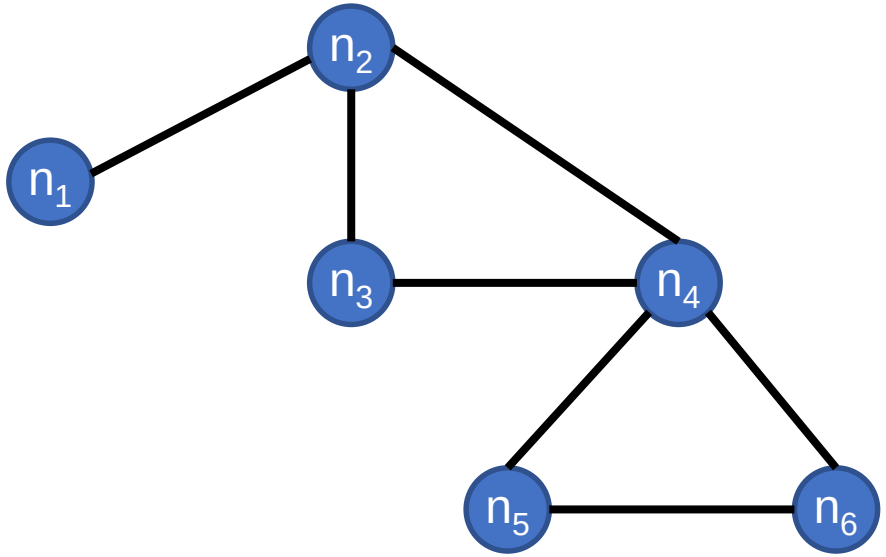
Neural Networks/Deep Learning: 101

Let's imagine a cat and dog **classifier**:



Graph Embeddings: Origins

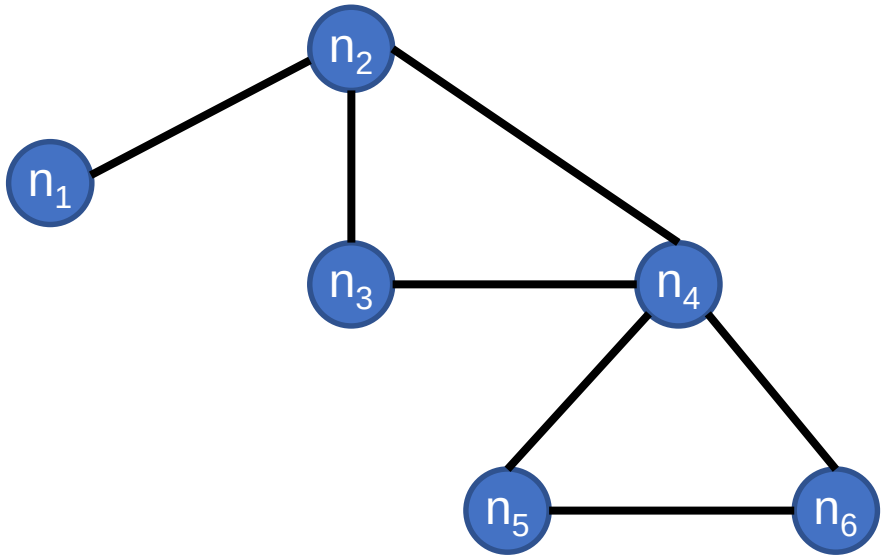
Some of the original **graph embedding** methods (DeepWalk and node2vec) are based on **random walks** and **on text embedding** principles.

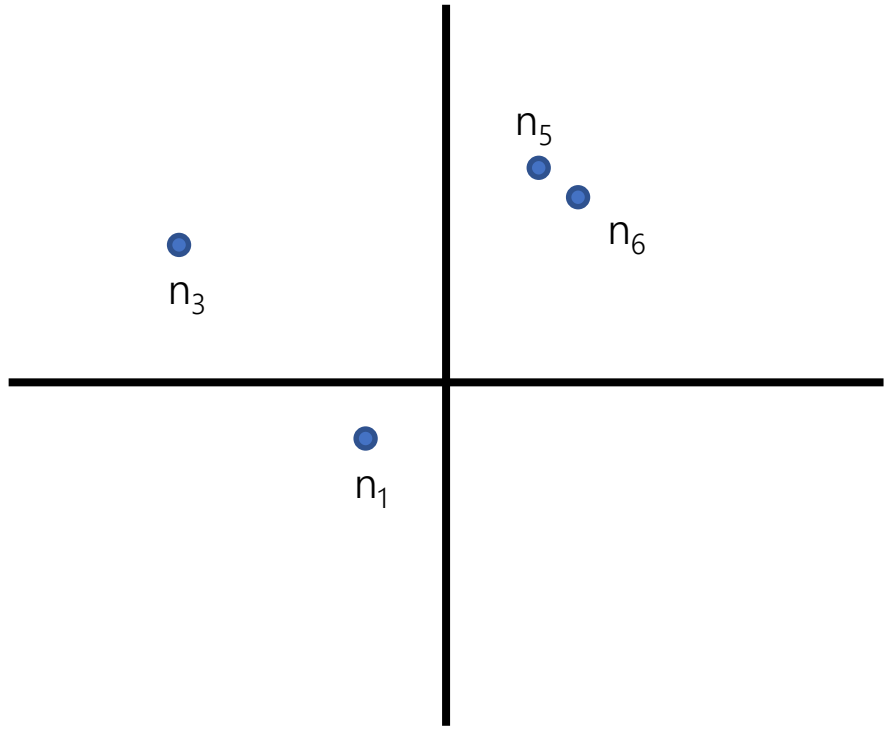


nodes closer should have similar rperesentation

DeepWalk and node2vec: General Ideas

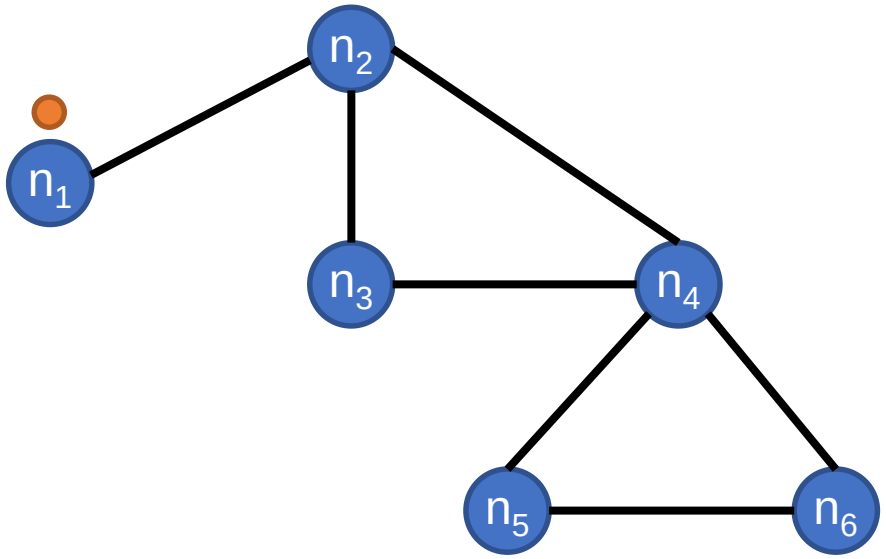
These methods are based on the idea **Network Homophily**: *Nodes that are close together in the graph should be close together in the embedding space*.





DeepWalk and node2vec: General Ideas

The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



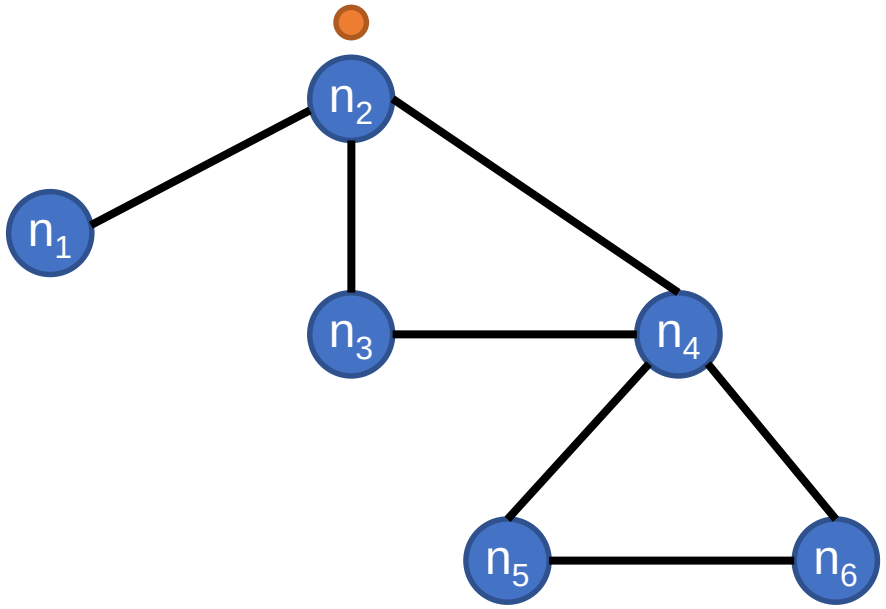
n_1

will model diferent the transitions

the first step is to generate these seqs that are random

DeepWalk and node2vec: General Ideas

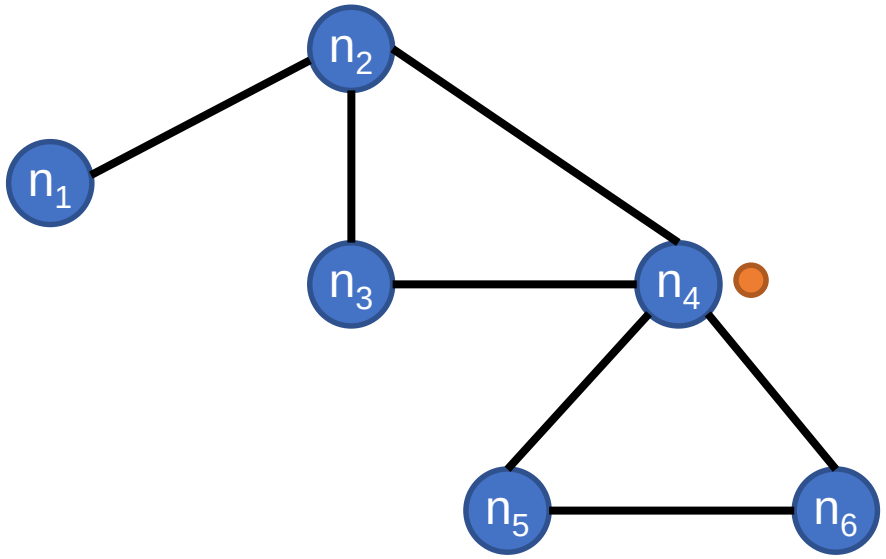
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



n_1, n_2

DeepWalk and node2vec: General Ideas

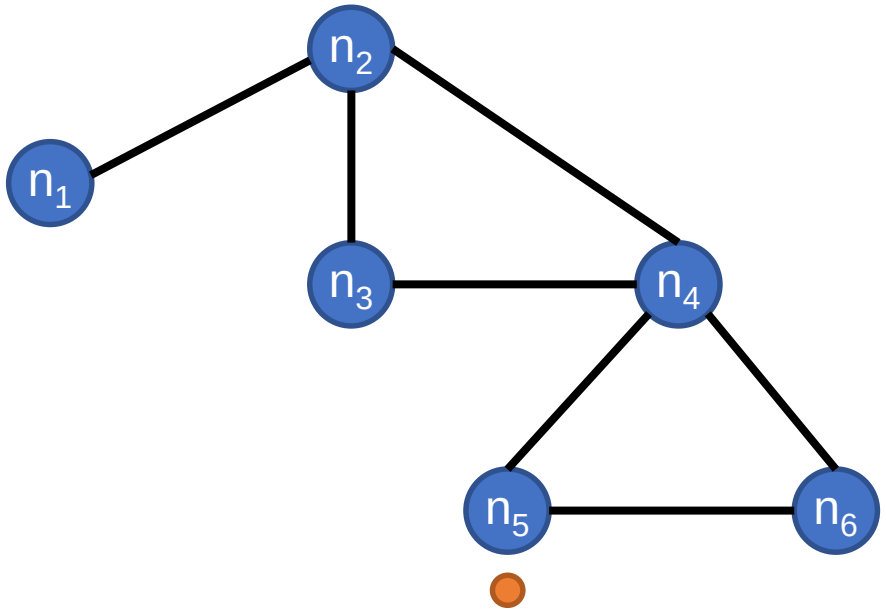
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



n_1, n_2, n_4

DeepWalk and node2vec: General Ideas

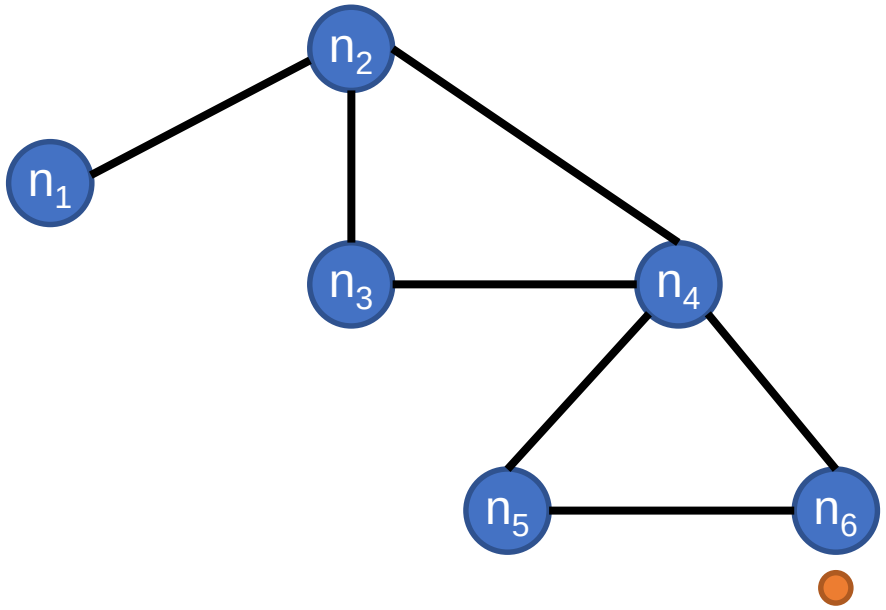
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



n_1, n_2, n_4, n_5

DeepWalk and node2vec: General Ideas

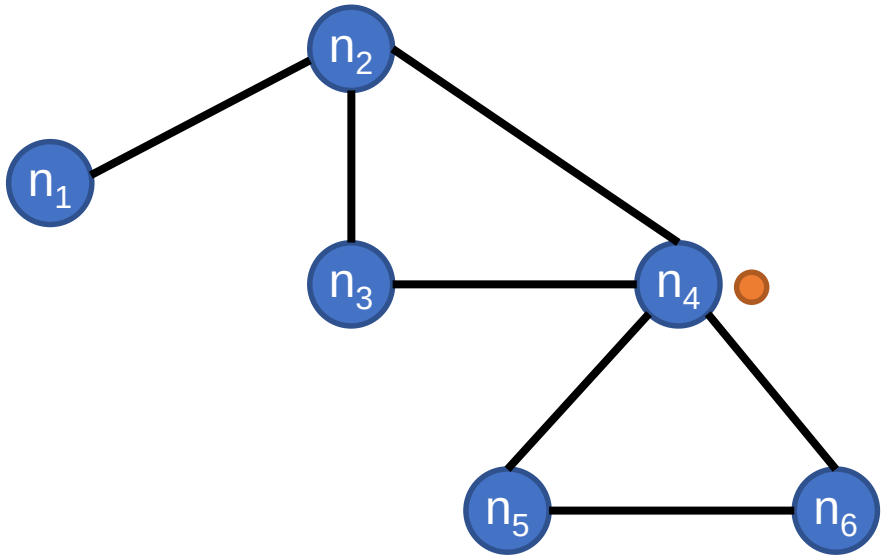
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



n_1, n_2, n_4, n_5, n_6

DeepWalk and node2vec: General Ideas

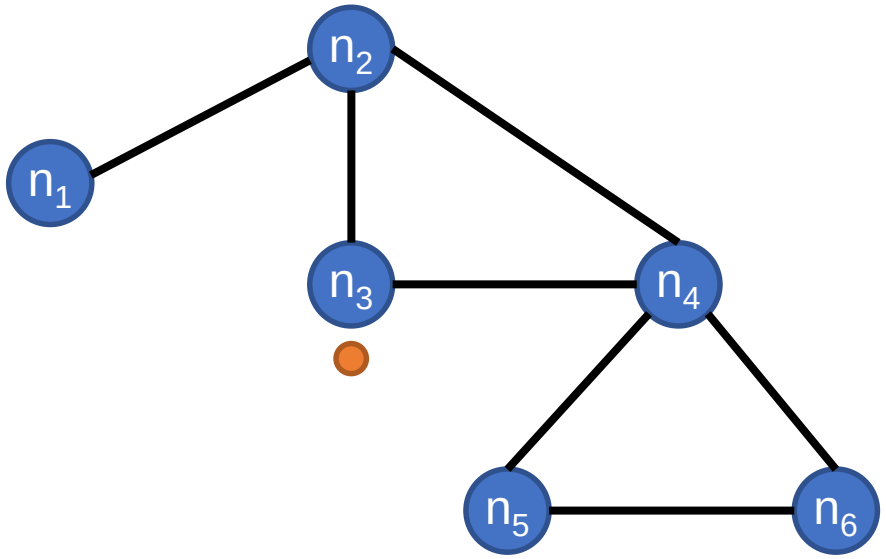
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



$n_1, n_2, n_4, n_5, n_6, n_4$

DeepWalk and node2vec: General Ideas

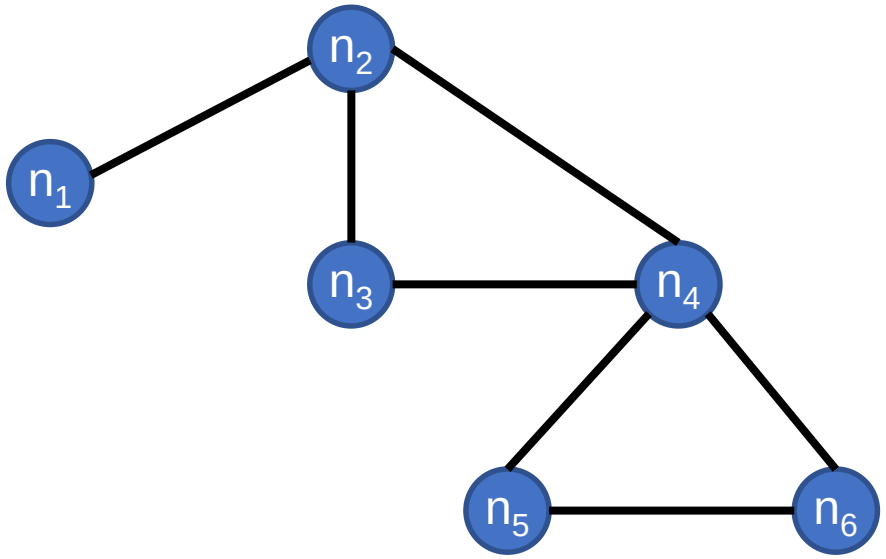
The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



$n_1, n_2, n_4, n_5, n_6, n_4, n_3$

DeepWalk and node2vec: General Ideas

The first step is the generation of **random walks** starting all the nodes of the graph. The random walk mechanism is different depending on the algorithm.



$n_1, n_2, n_4, n_5, n_6, n_4, n_3$
$n_1, n_2, n_3, n_4, n_2, n_3, n_4$
$n_2, n_3, n_4, n_5, n_6, n_4, n_2$
...
$n_6, n_5, n_4, n_3, n_2, n_1, n_2$

DeepWalk and node2vec: General Ideas

To obtain the embeddings, these random walks are viewed as **sentences**, and standard **text embedding** techniques are used.

$n_1, n_2, n_4, n_5, n_6, n_4, n_3$ $\approx$ The dog is barking at the stranger.

DeepWalk and node2vec: General Ideas

To obtain the embeddings, these random walks are viewed as **sentences**, and standard **text embedding** techniques are used.

CBOW: Predict the central word (i.e., predict a word based on its context)

The dog is barking at the stranger

DeepWalk and node2vec: General Ideas

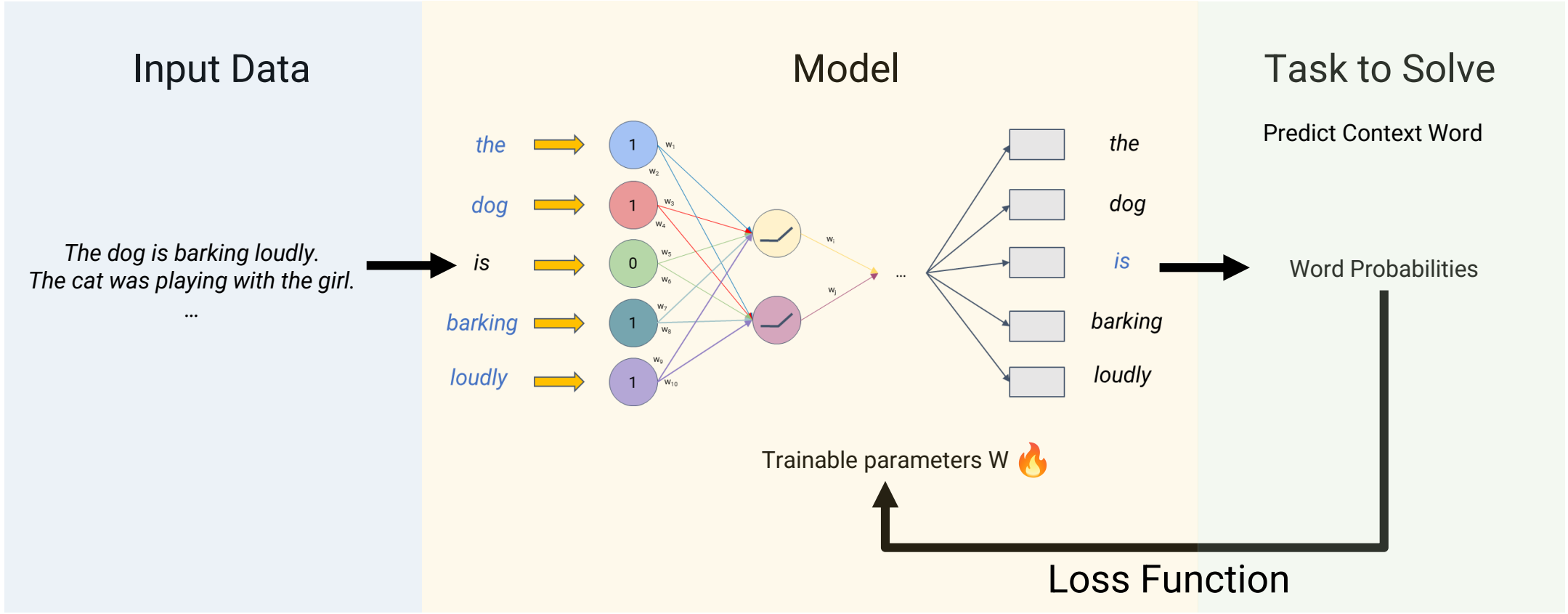
To obtain the embeddings, these random walks are viewed as **sentences**, and standard **text embedding** techniques are used.

Skip-Gram: Predict surrounding words (i.e., from a word predict its context)

The dog is barking at the stranger

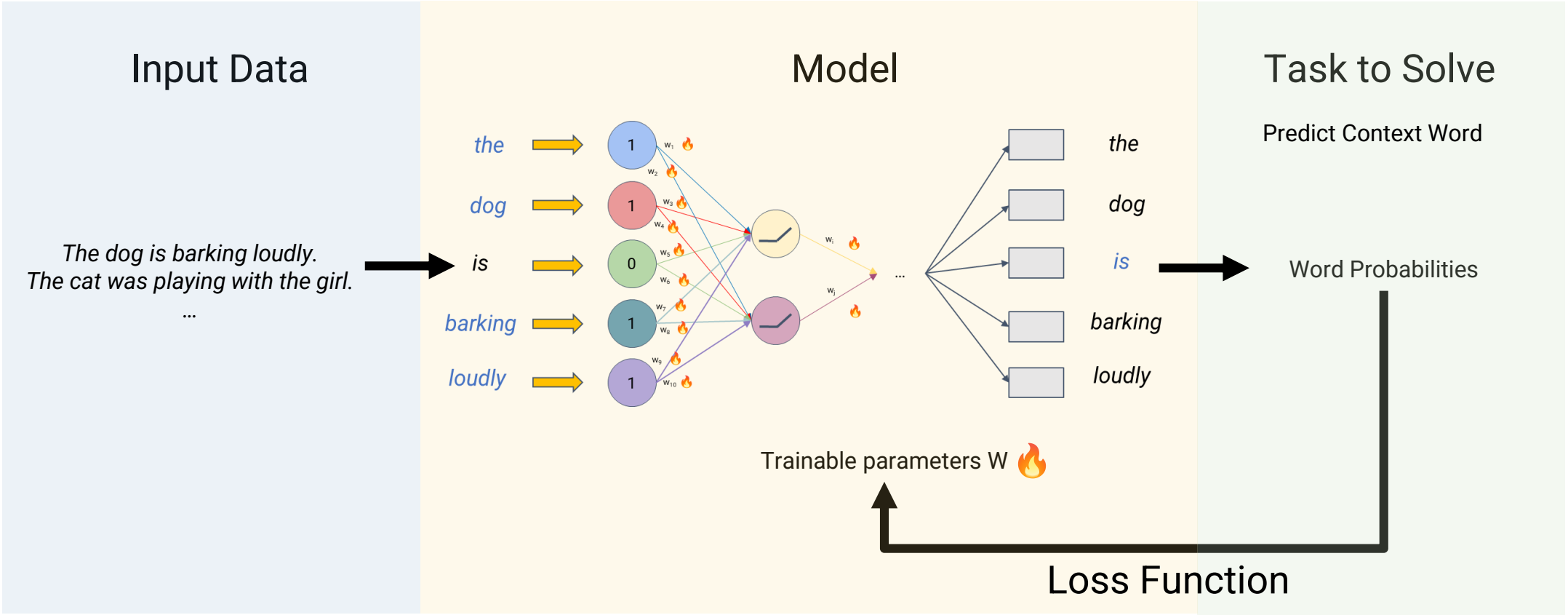
DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:



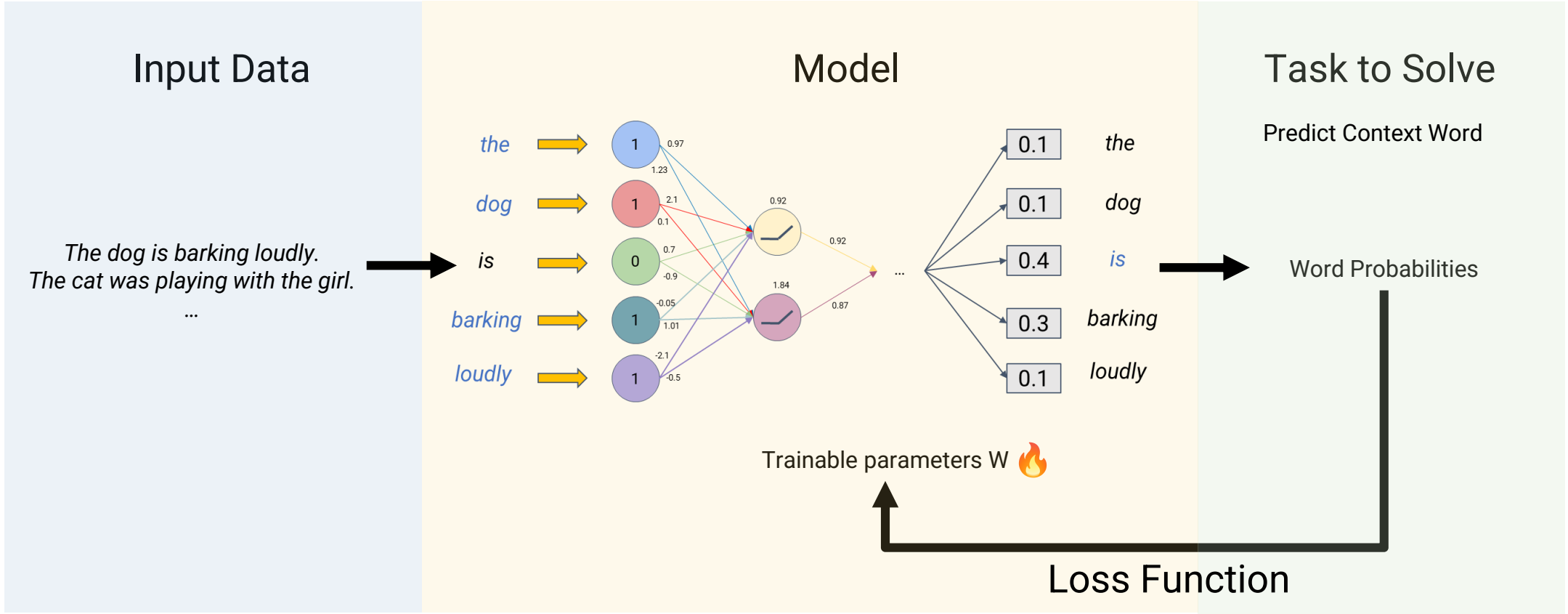
DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:



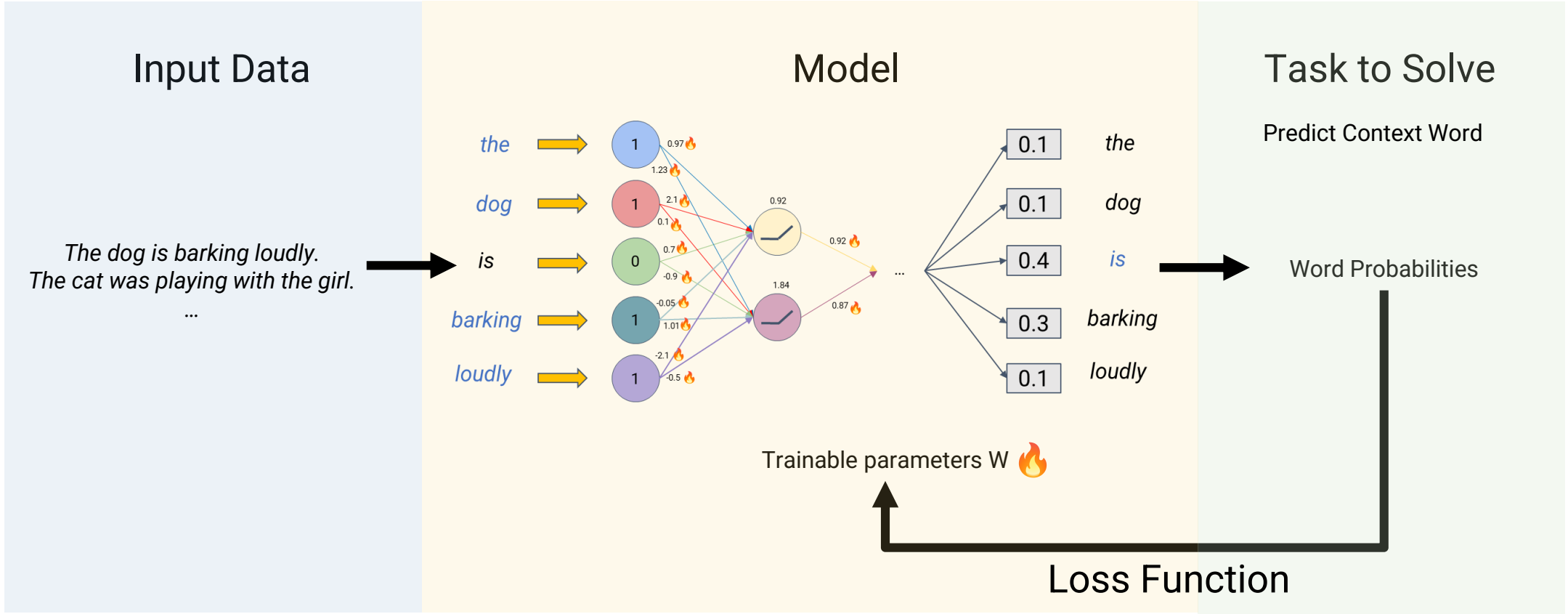
DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:



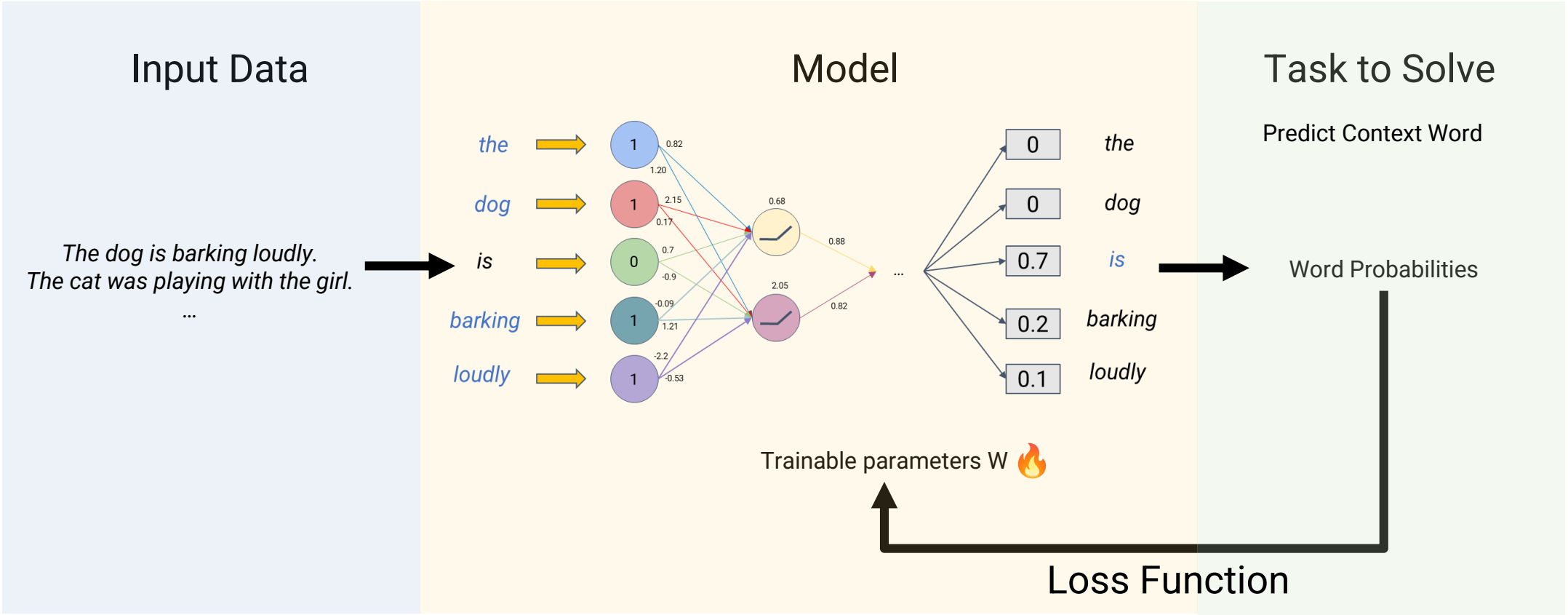
DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:



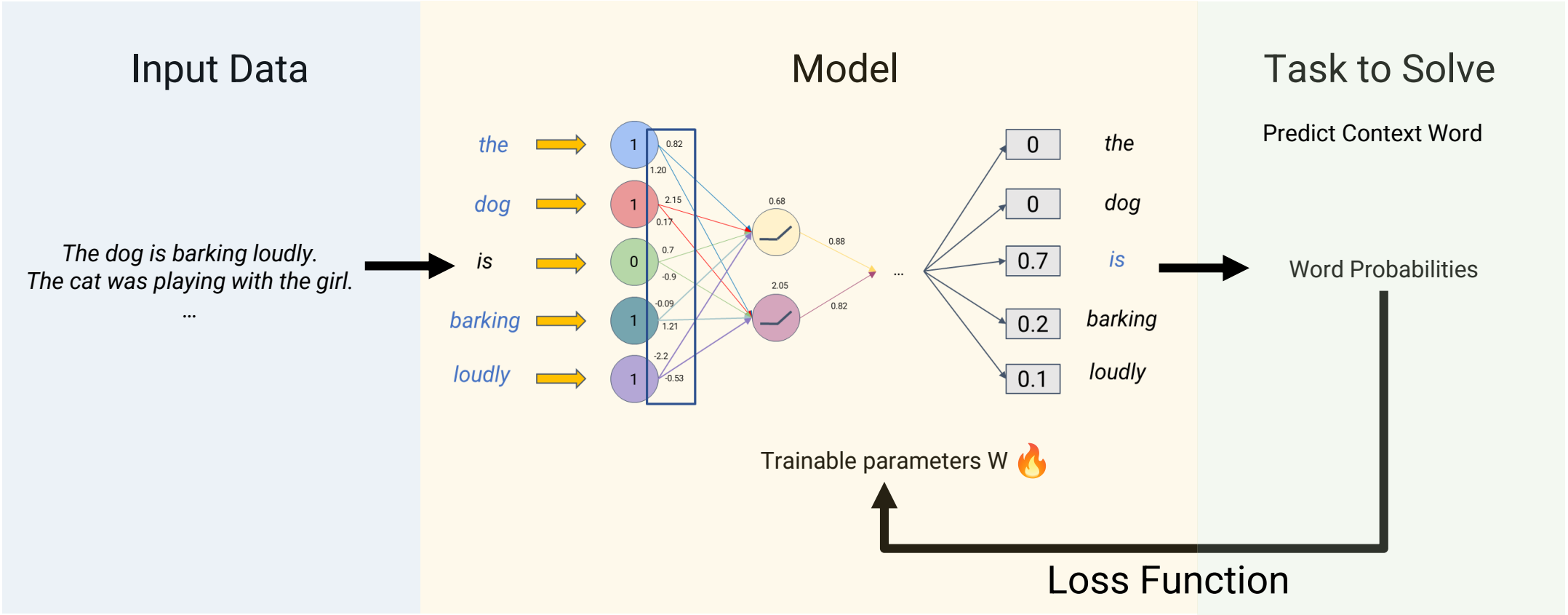
DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:



DeepWalk and node2vec: General Ideas

Let's consider **CBOW**:

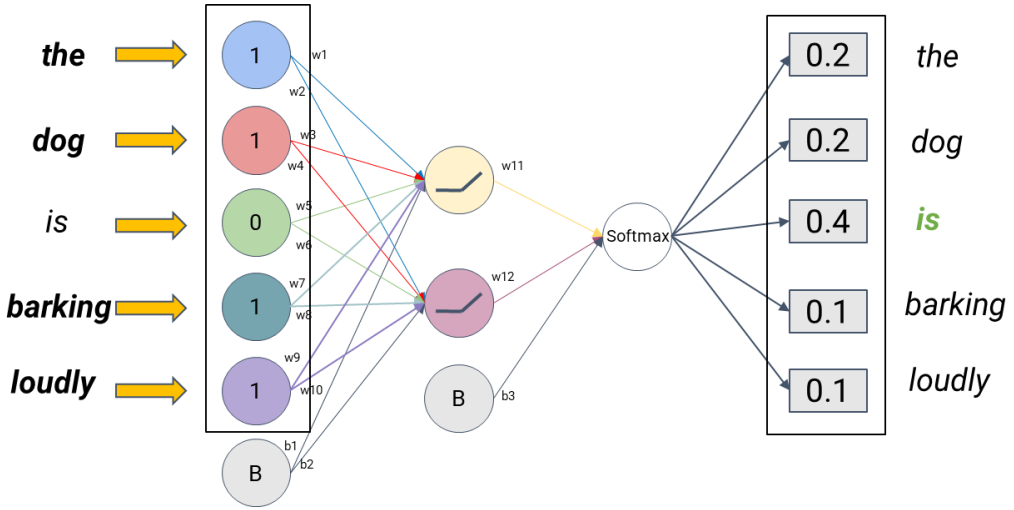


DeepWalk and node2vec: General Ideas

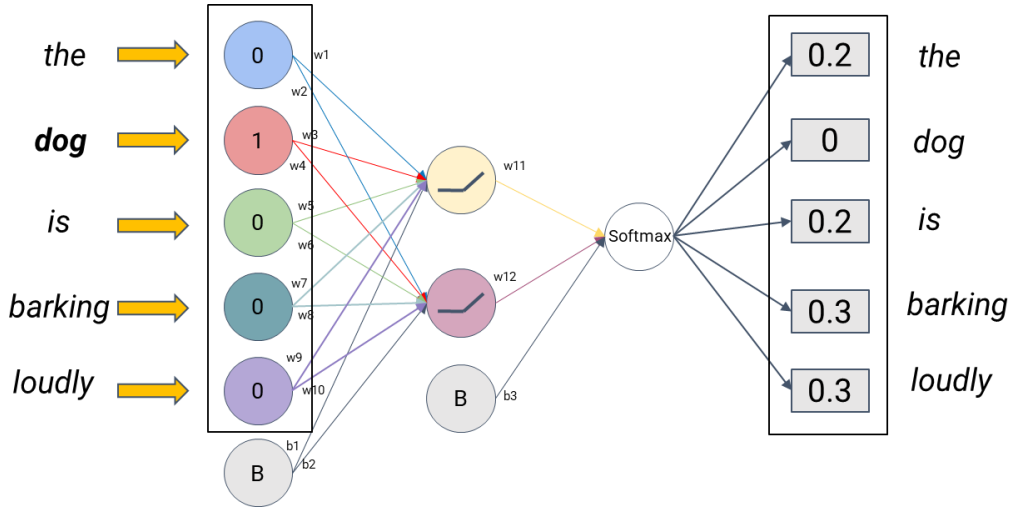
To obtain the embeddings, these random walks are viewed as **sentences**, and standard **text embedding** techniques are used.

en node2vec en vez de words como input (the) tendremos nodos donde si tiene un 1 el nodo significa q esta presente y 0 lo contrario

CBOW

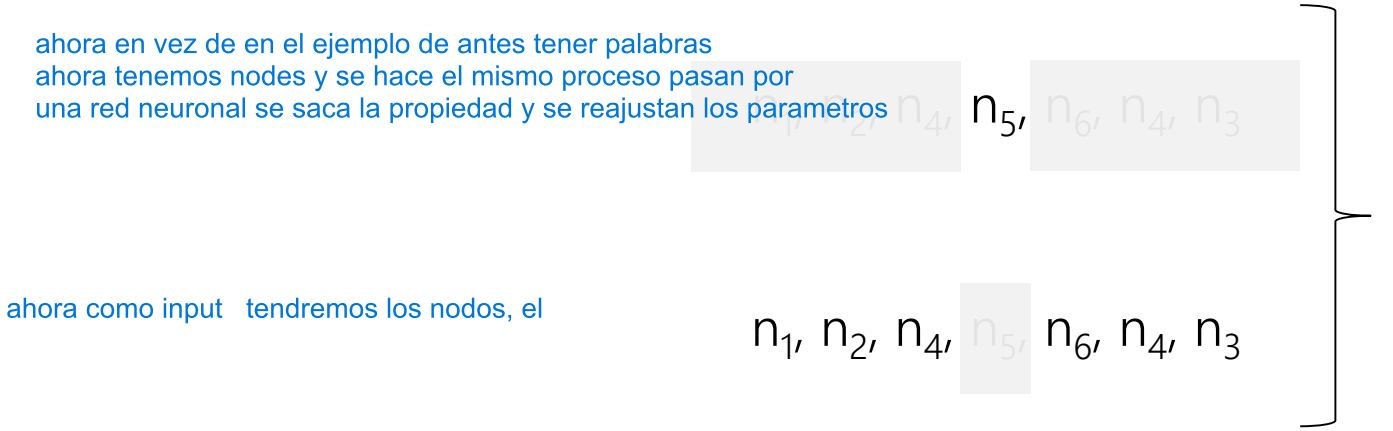


Skip-Gram



DeepWalk and node2vec: General Ideas

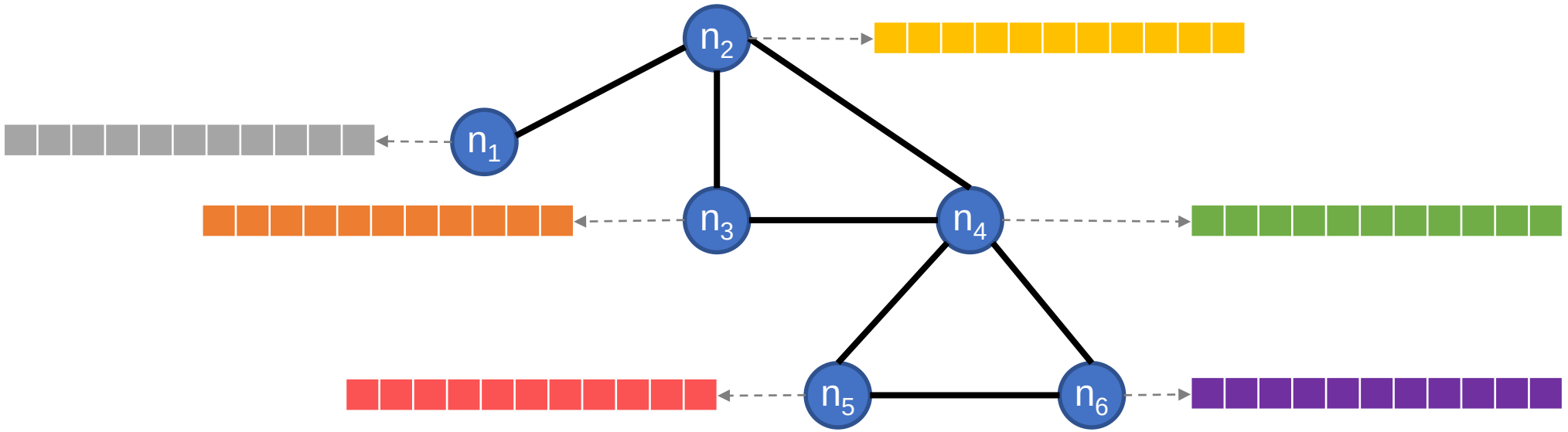
To obtain the embeddings, these random walks are viewed as **sentences**, and standard **text embedding** techniques are used.



We learn node representations based on their **context**.

DeepWalk and node2vec: General Ideas

After training, the **optimal embeddings** are obtained and can be used in **downstream tasks**.



DeepWalk and node2vec: Limitations

When applying this methods to Knowledge or Property Graphs, some limitations arise:

1. The **Network Homophily** principal is not well suited for them.

we would like to have these similar emb space where places arwe in one part, people in other, etc.

This principle makes sense when working with simple graphs (e.g., representing friends in social networks) but now when multiple concepts are included.

(Antoni Gaudí, born_in, Reus)

DeepWalk and node2vec: Limitations

When applying this methods to Knowledge or Property Graphs, some limitations arise:

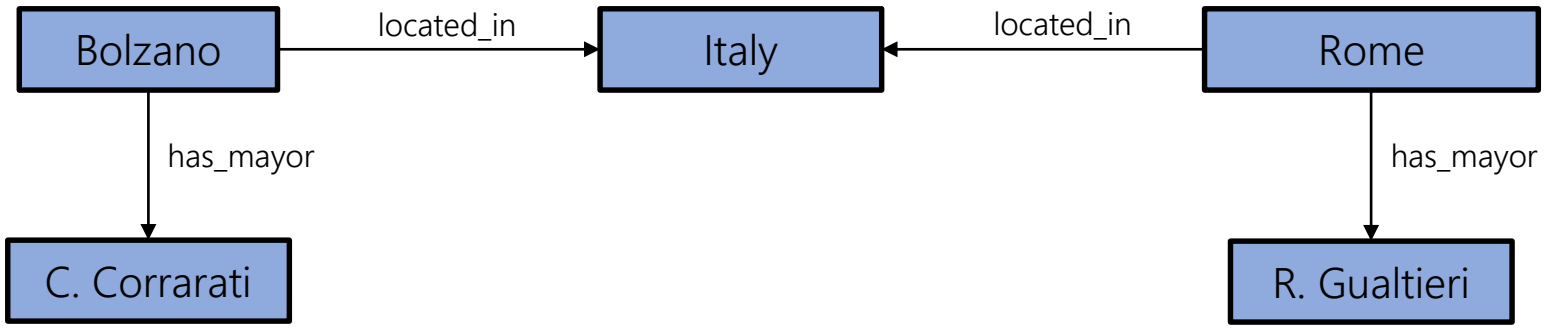
1. The **Network Homophily** principal is not well suited for them.
2. **Different types of relations** are not taken into account.
3. The **directed knowledge** is not enforced (i.e., hierarchies in Knowledge Graphs).
4. The node/edge properties are lost (Property Graphs).

Knowledge Graph Embeddings **Graph Neural Networks**

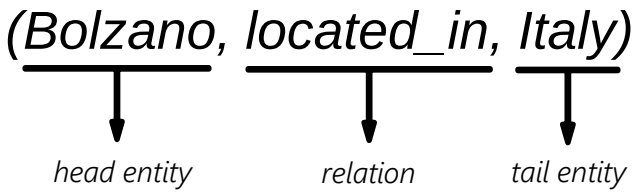
Knowledge Graph Embeddings

Knowledge Graph Embeddings:

Knowledge Graphs (KGs) are a network of **entities** interconnected by **relations.** These links have semantics, making a KG both *machine-readable* and *human-interpretable*.

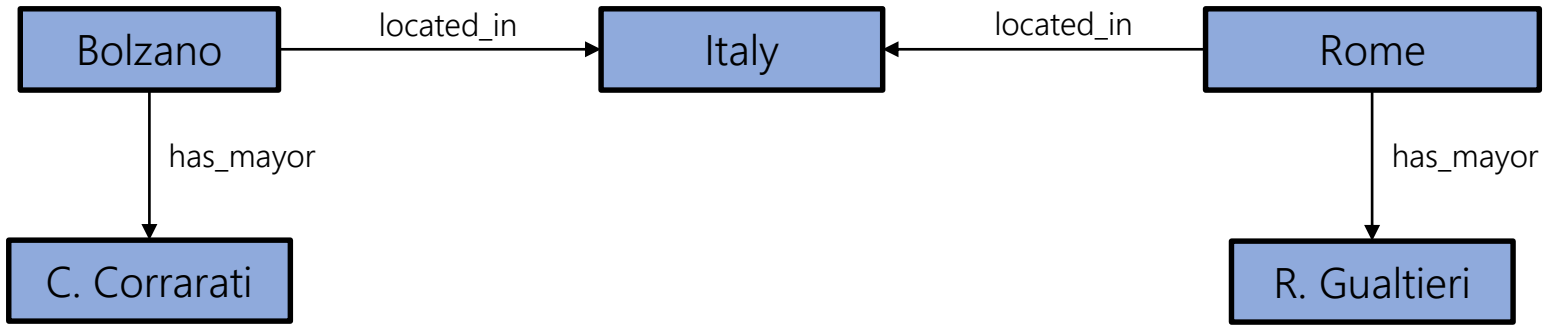


They can be represented using **triples:**



Knowledge Graph Embeddings:

While knowledge graphs support the use of **graph-specific algorithms**, such approaches are typically **limited** and with high **computational costs**.



Path-finding

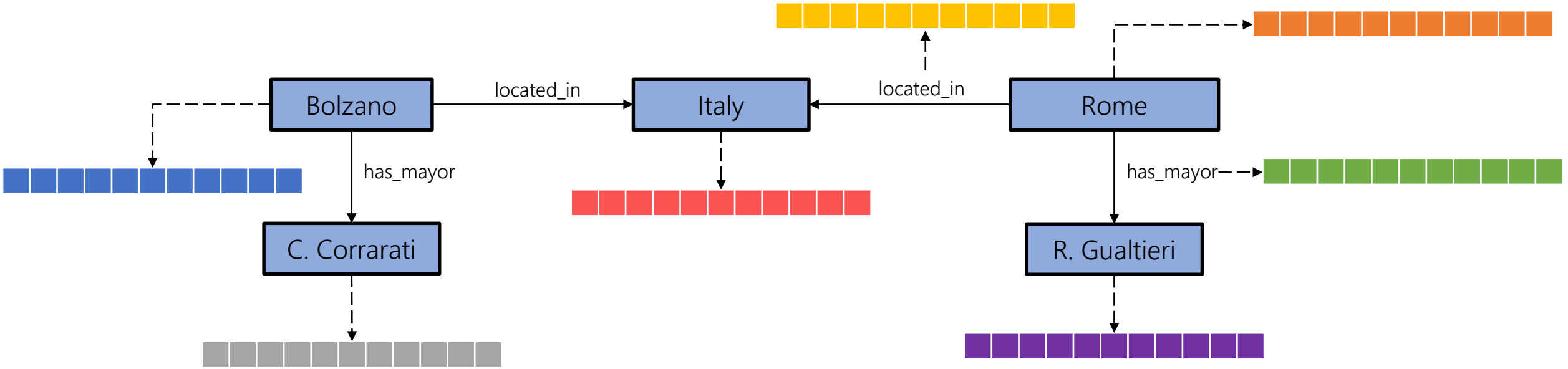
Connectivity

Community Detection

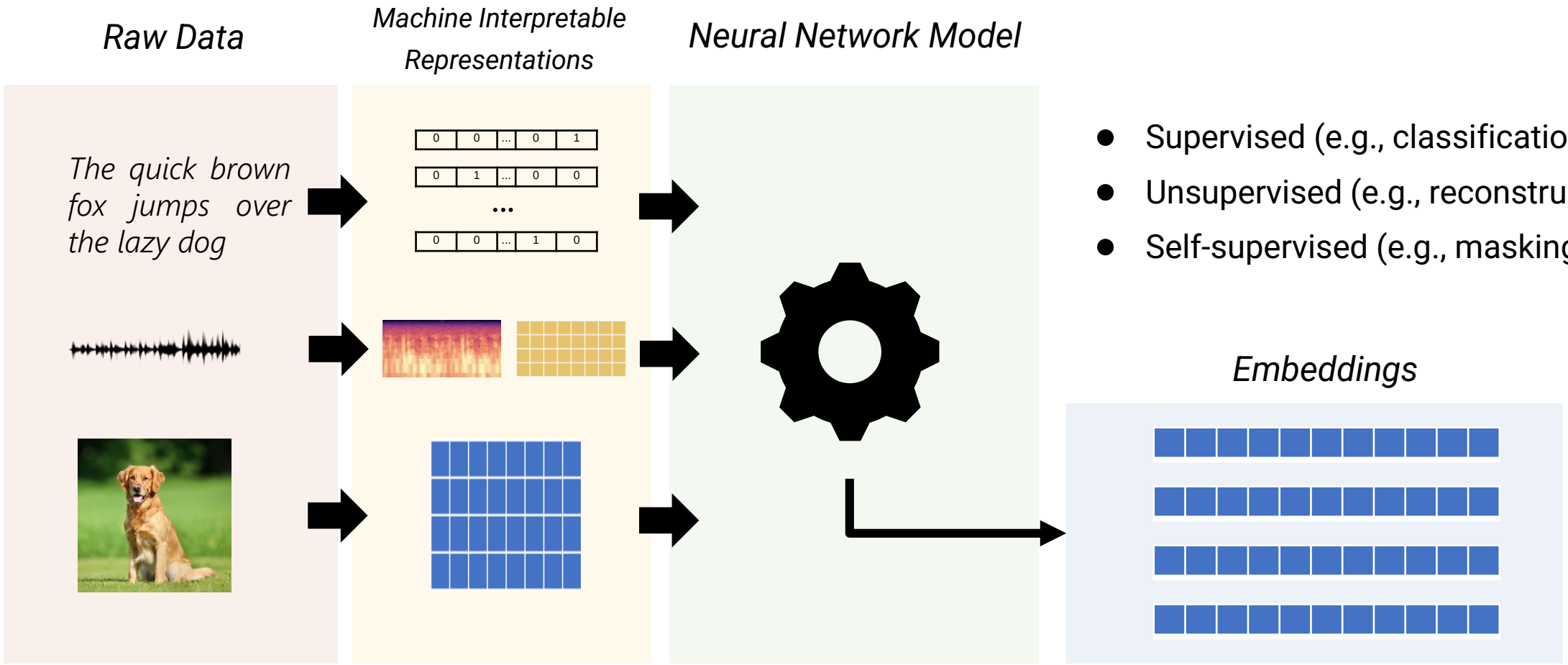
…

Knowledge Graph Embeddings:

We want to learn representations for both entities and relations, **retaining** as much contextual/structural/sematic information as possible and enabling **inference** and **generalization**.



Embeddings Summary:



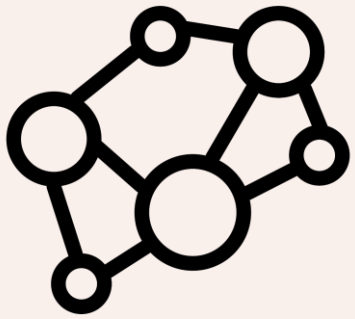
- Supervised (e.g., classification)
- Unsupervised (e.g., reconstruction)
- Self-supervised (e.g., masking)

Embeddings Summary:

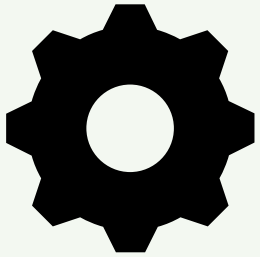
Raw Data

Machine Interpretable Representations

KGE Model

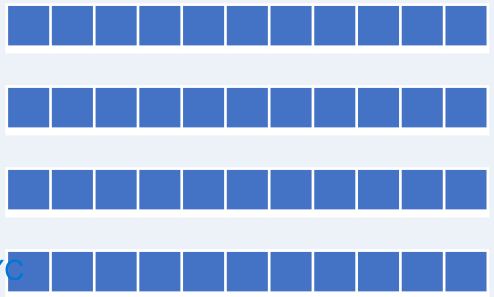


(h,r,t)



- Link Prediction

Embeddings



por ejemplo (A.Gaudí,bornin, ?) queremos que prediga Reus entre las opciones que hay Reus, BCN y NYC

KGE Models:

To achieve these goals, different Knowledge Graph Embedding (KGE) **Models** have been proposed, falling into two main categories:

Translational

Semantic

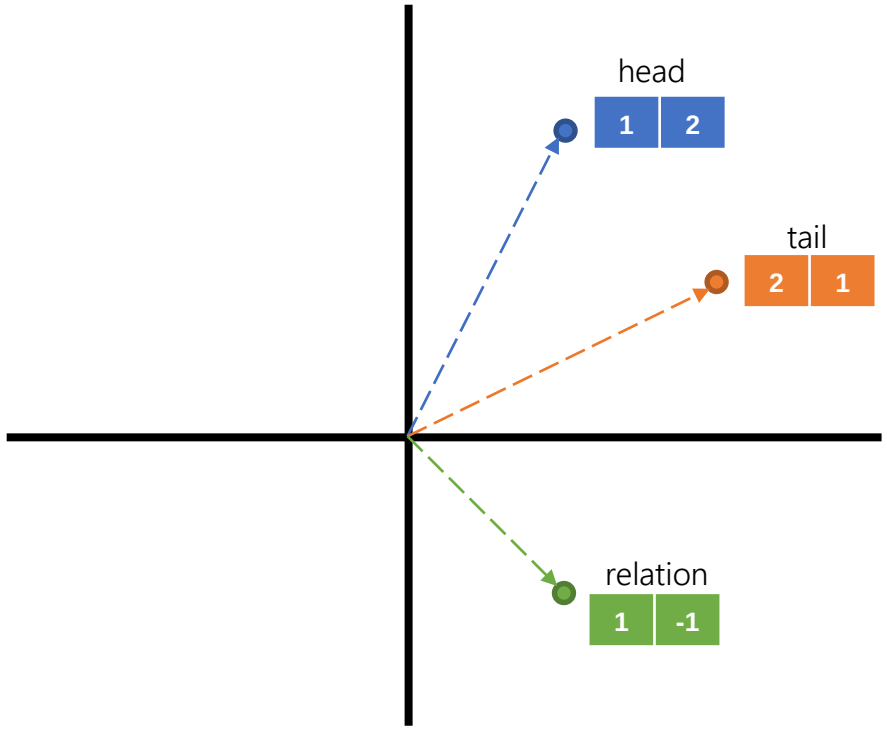
These families of models differ in how they design the **loss function** they optimize to produce the final embeddings.

$$L(h,r,t)$$

KGE Models: Translational

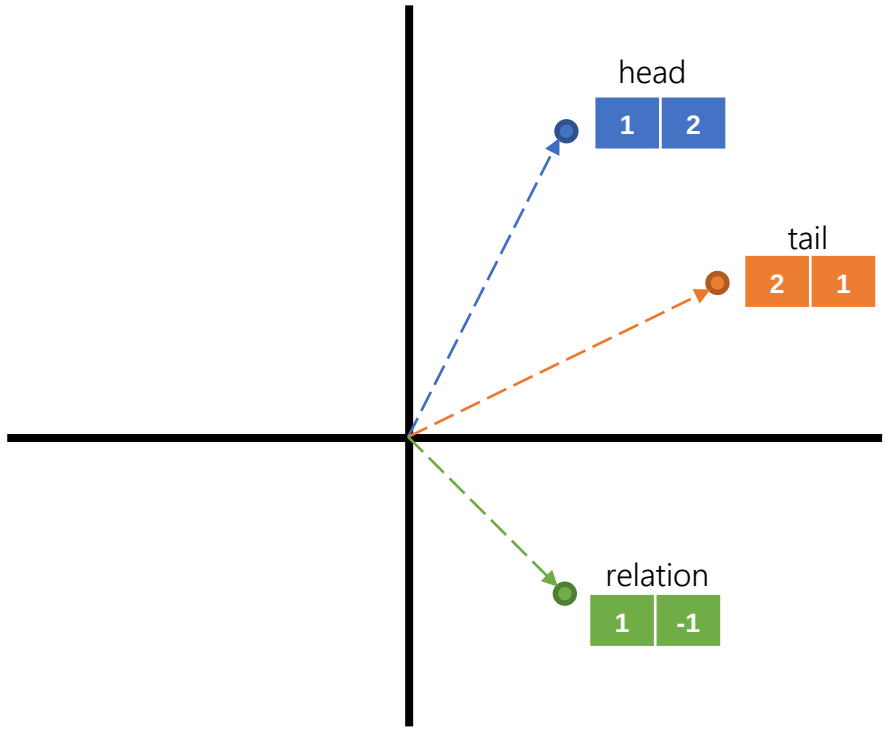
Translational KGE models are based on **geometric transformations**, usually considering the embedding of the relation as a translation from the head entity to the tail entity.

(head, relation, tail)



KGE Models: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.



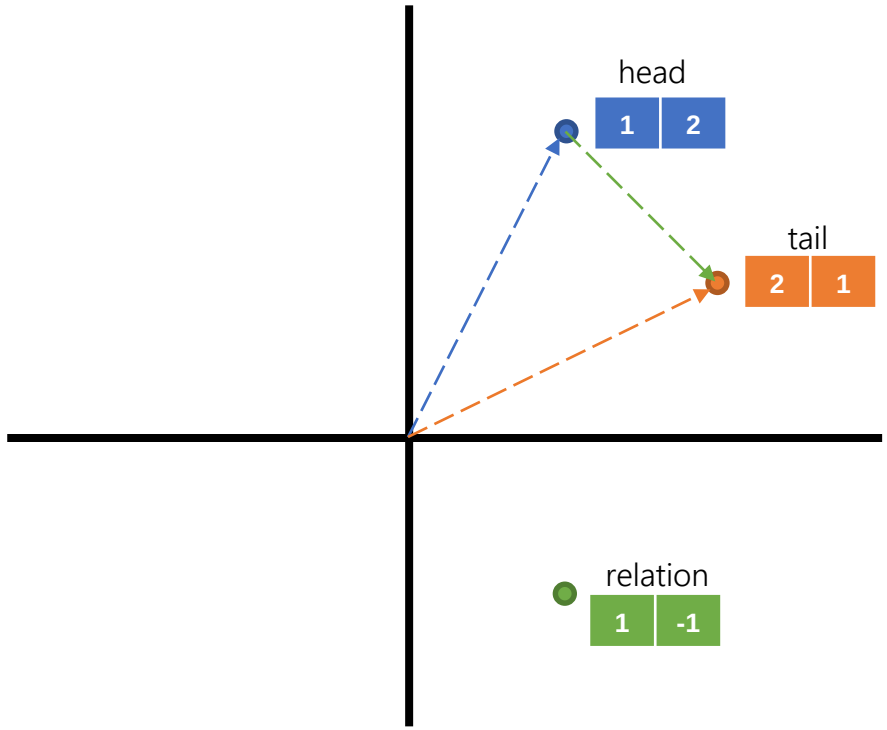
TransE: Loss= |head + relation - tail|

KGE Models: Translational

are based on trasnlation that go from the head entity to tail entity

The most simple translational model is **TransE**, which views the translations as vector additions.

the embedding of the relation should go from the head to the tail. Head + realtion go to the tail, one embedding of head + relation=tail



TransE: Loss= |head + relation - tail|

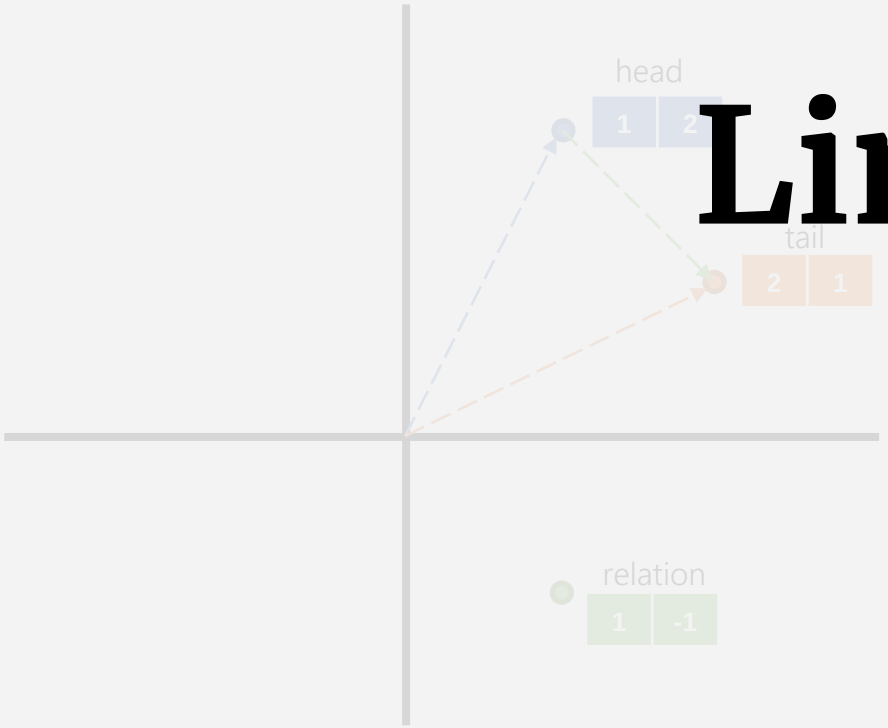
(head, relation, tail)

Loss= |(1,2)+(1,-1) – (2,1)| = 0

KGE Models: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.

libreta



TransE: Loss= |head + relation - tail|

(head, relation, tail)

Loss= |(1,2)+(1,-1) − (2,1)| = 0

Limitations?

LIMITATIONS: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.

Symmetry

TransE: Loss= |head + relation - tail|

(entity1, relation, entity2)

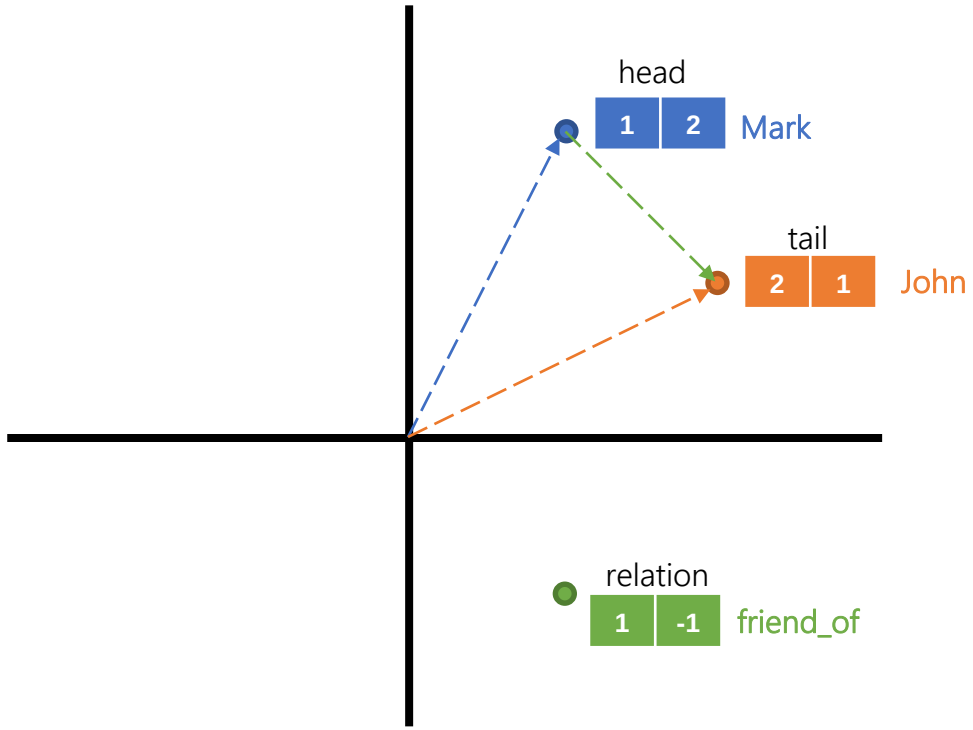
(entity2, relation, entity1)

(head, relation, tail)

Loss= |(1,2)+(1,-1) – (2,1)| = 0

KGE Models: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.

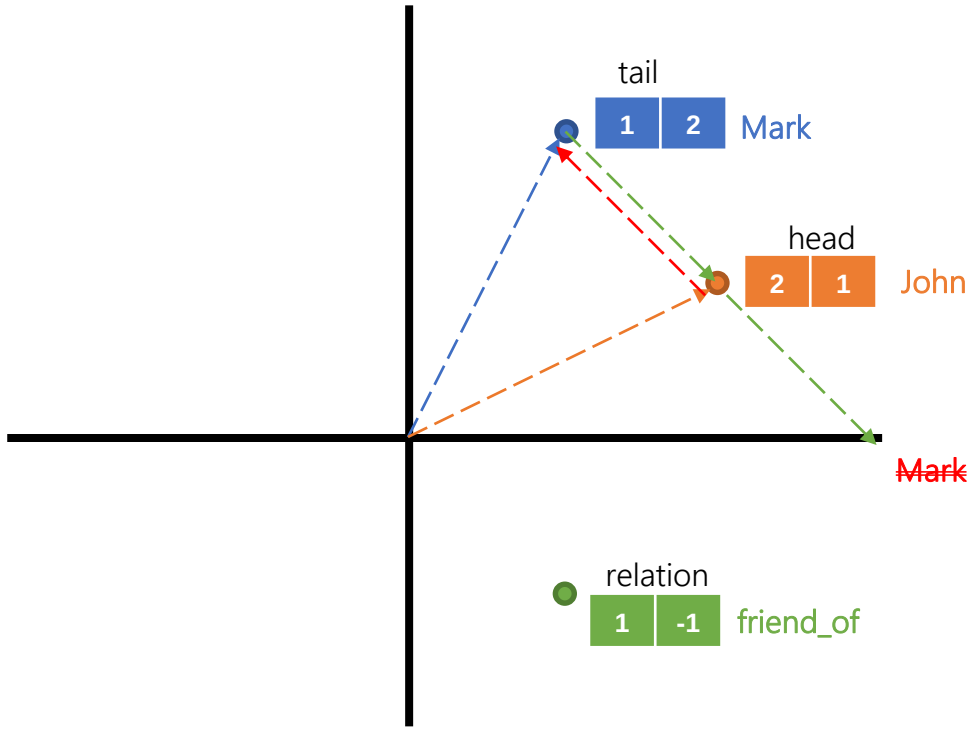


TransE: Loss= |head + relation - tail|

(Mark, friend_of, John)

KGE Models: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.



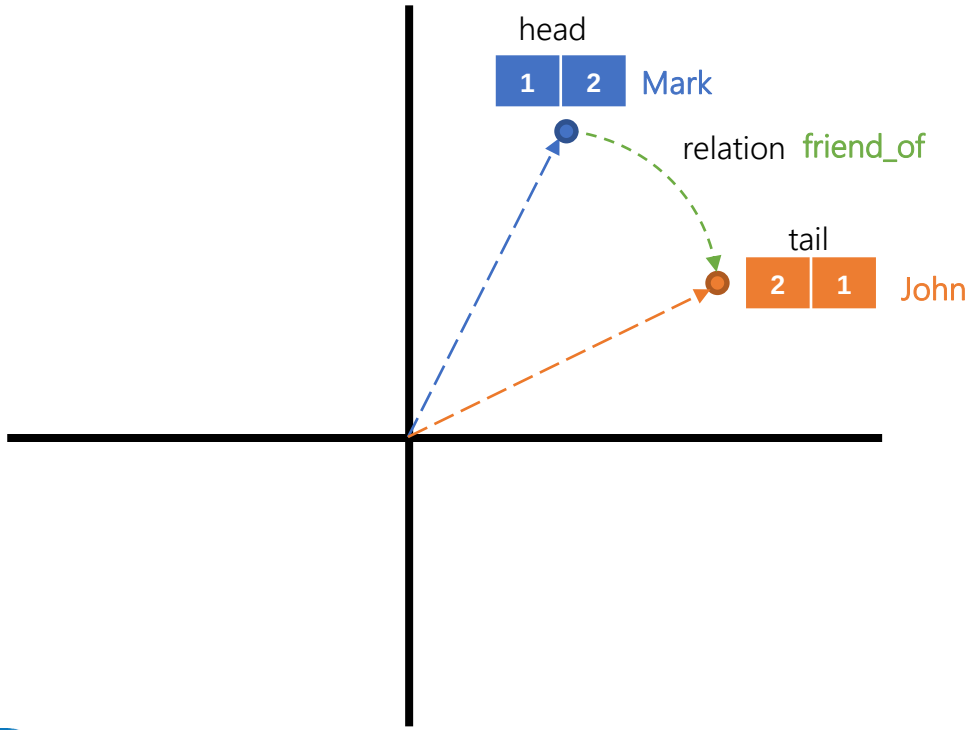
TransE: Loss= |head + relation - tail|

(Mark, friend_of, John)

(John, friend_of, Mark)

KGE Models: Translational

RotatE model defines each relation as a rotation from the source entity to the target entity in the complex vector space.



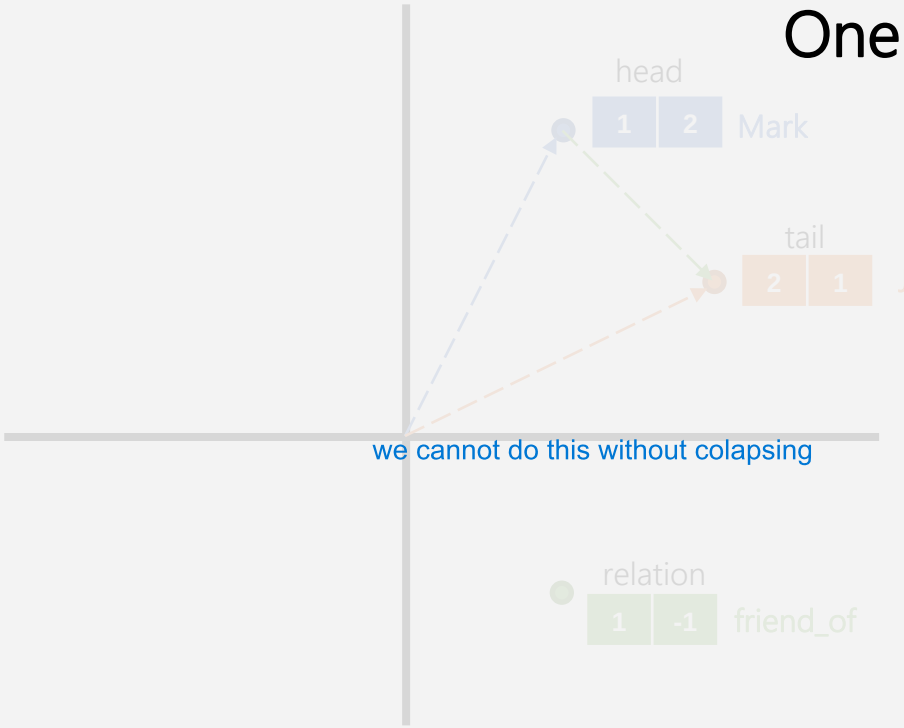
RotatE: Loss= |head ○ relation - tail|

(Mark, friend_of, John)

(John, friend_of, Mark)

LIMITATIONS: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.



One-to-Many | Many-to-one

(entity1, relation, entity2)

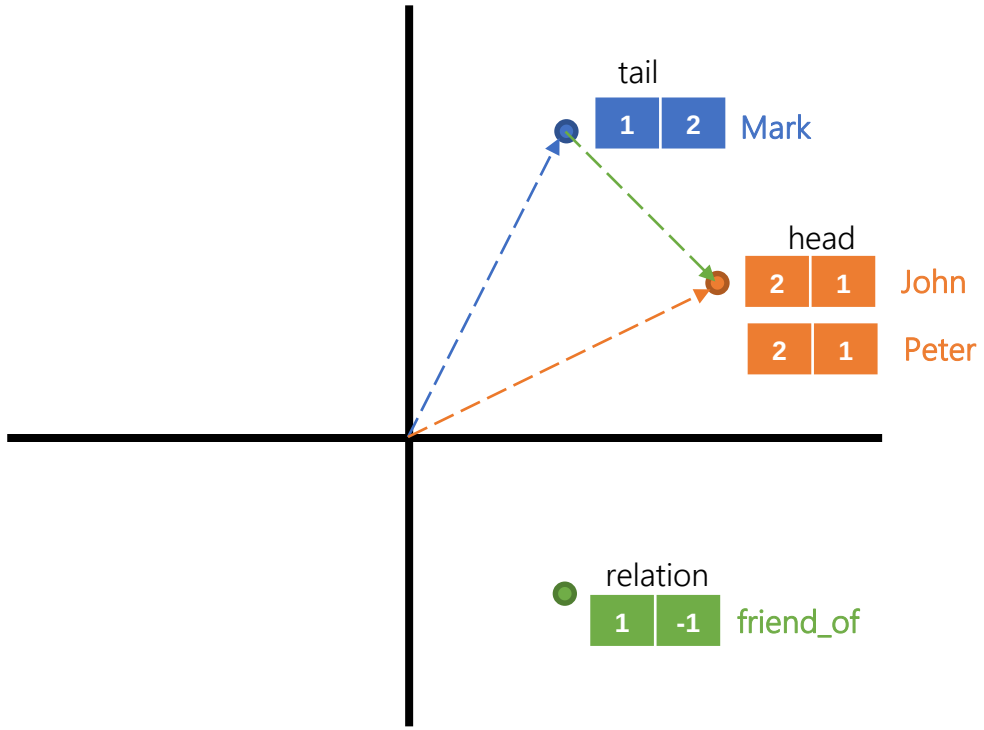
(entity1, relation, entity3)

TransE: Loss= |head + relation - tail|

(Mark, friend_of, John)

KGE Models: Translational

The most simple translational model is **TransE**, which views the translations as vector additions.



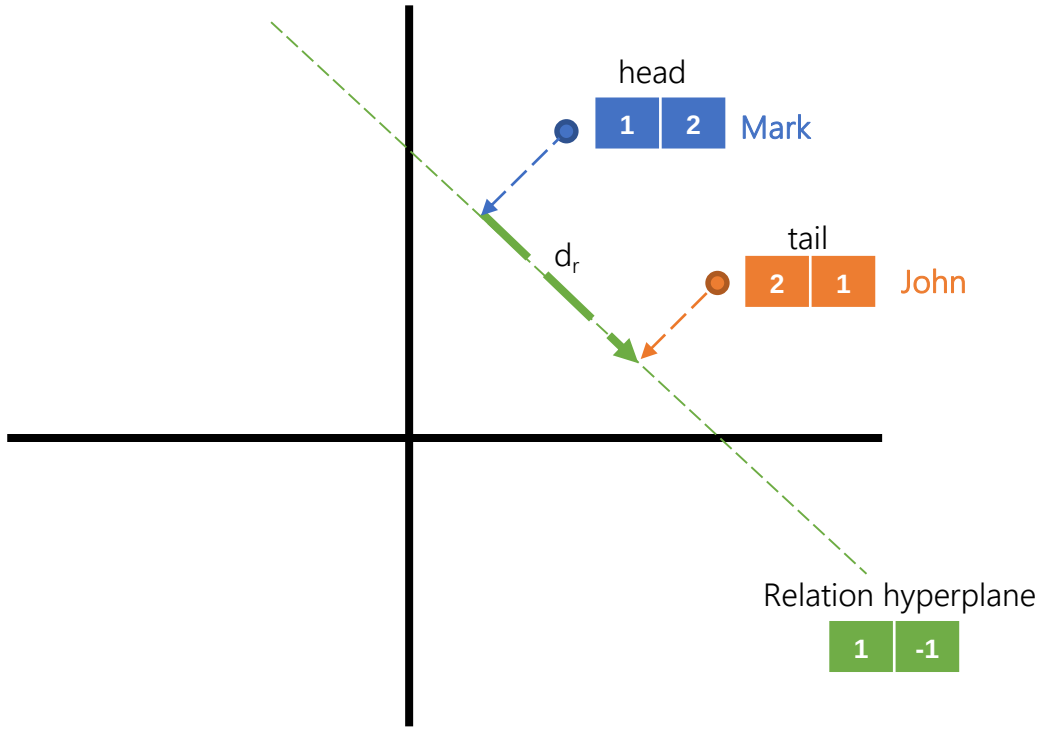
TransE: Loss= |head + relation - tail|

(Mark, friend_of, John)

(Mark, friend_of, Peter)

KGE Models: Translational

In **TransH**, this problem is solved by defining **hyperplanes** for each relation, to which the entities are projected and a relation embedding is also learned.



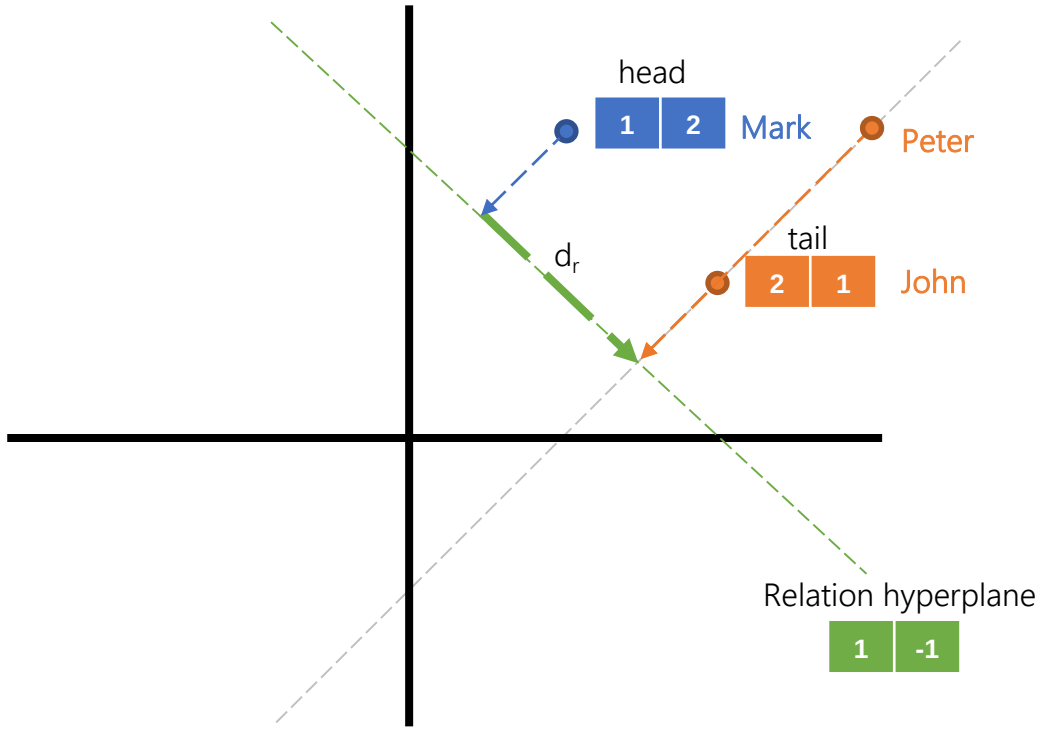
TransH:

$$Loss = |(h - w_r^T h w_r) + d_r - (t - w_r^T t w_r)|$$

(Mark, friend_of, John)

KGE Models: Translational

In **TransH**, this problem is solved by defining **hyperplanes** for each relation, to which the entities are projected and a relation embedding is also learned.



TransH:

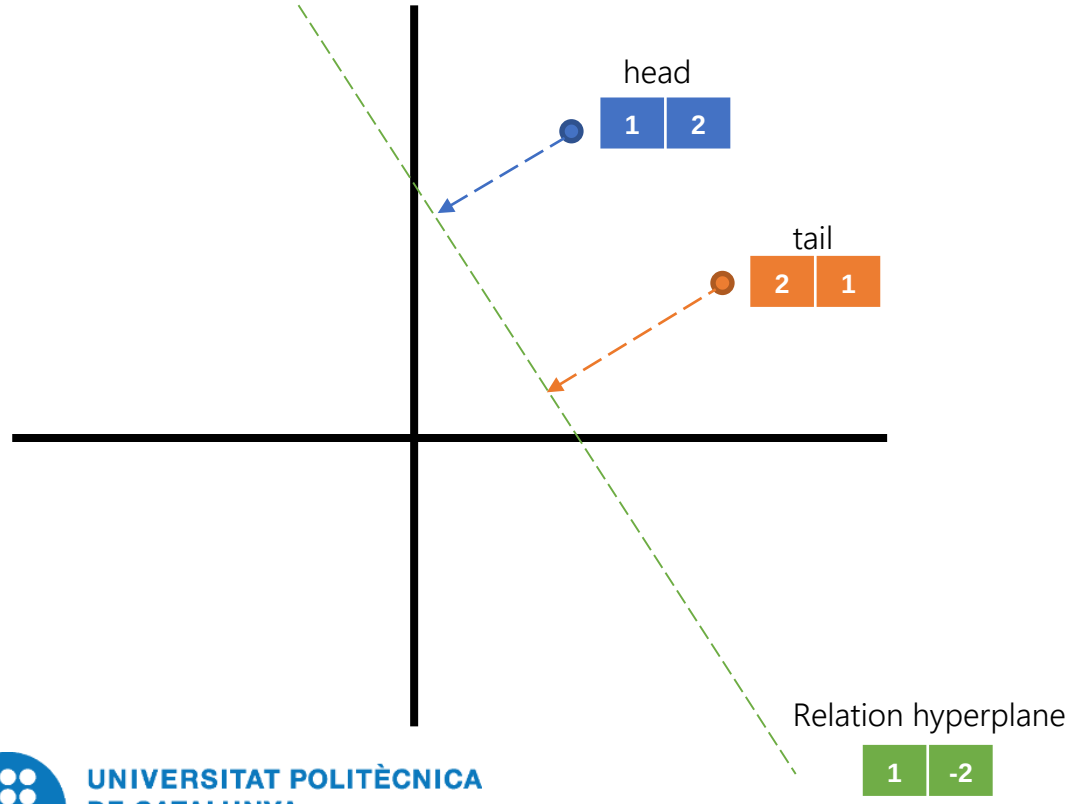
$$Loss = |(h - w_r^T h w_r) + d_r - (t - w_r^T t w_r)|$$

(Mark, friend_of, John)

(Mark, friend_of, Peter)

KGE Models: Translational

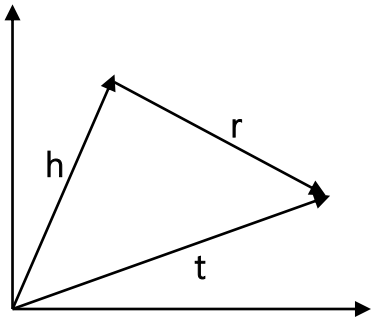
In **TransH**, this problem is solved by defining **hyperplanes** for each relation, to which the entities are projected and a relation embedding is also learned.



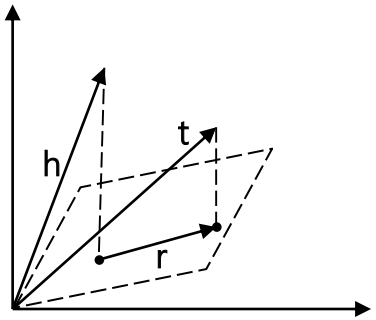
TransH:

$$Loss = |(h - w_r^T h w_r) + d_r - (t - w_r^T t w_r)|$$

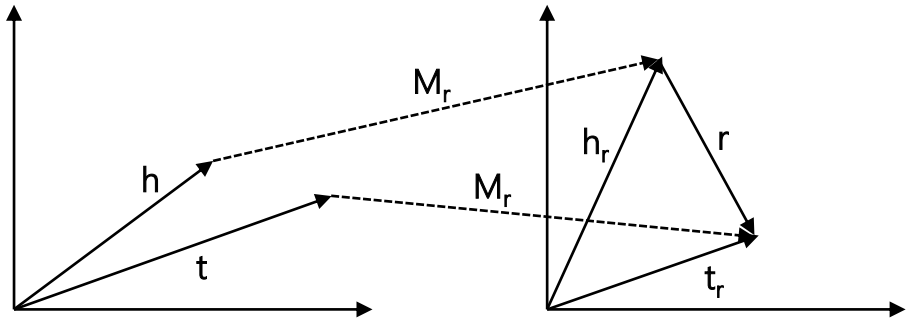
KGE Models: Translational



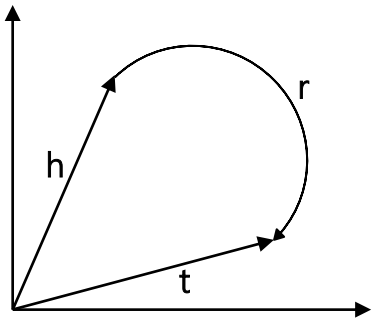
TransE



TransH



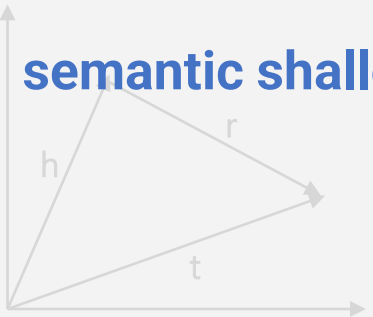
TransR



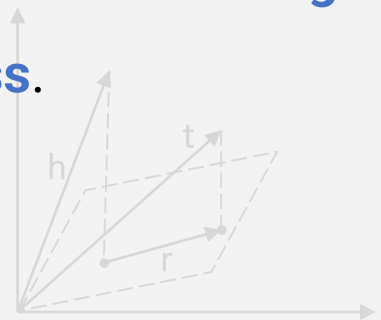
RotatE

LIMITATIONS: Translational

We must decide beforehand what **geometry** matters, with limitations in **expressiveness** and **semantic shallowness**.



TransE



TransH

TransR

RotatE

ADVANTAGES

These models have high **interpretability**.

TransE

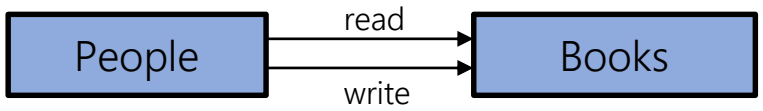
TransH

TransR

RotatE

KGE Models: Translational Exercise (10 min)

John has read a book by an unknown author. Given the following KG schema and existing embeddings, who would you say that is likely to be the author according to the TransE model?



John = [1, 2]

Bob = [2,3]

Eve = [4,6]

Mark = [4,3]

read = [4.2,5.5]

write = [1.3,1.4]

KGE Models: Semantic

Semantic models try to capture **latent semantics** by maximizing the plausibility of a triple. This is achieved by designing loss functions that typically rely on **multiplications**.

Translational

Minimize mismatch distance after translation

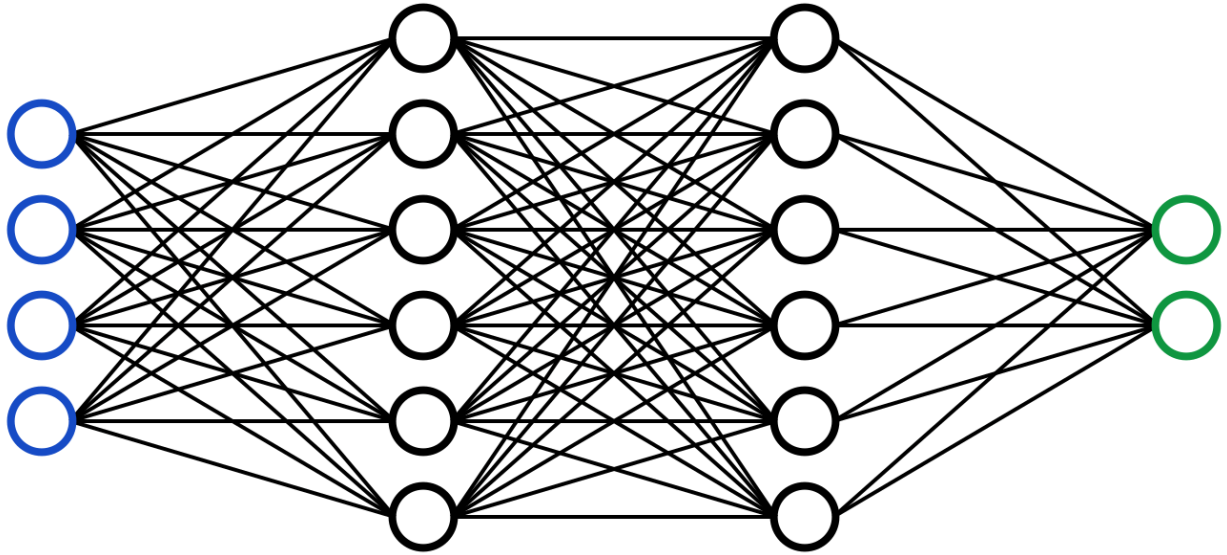
Semantic

maximixar el valor dado una tripla

Maximize the value given to a triple.

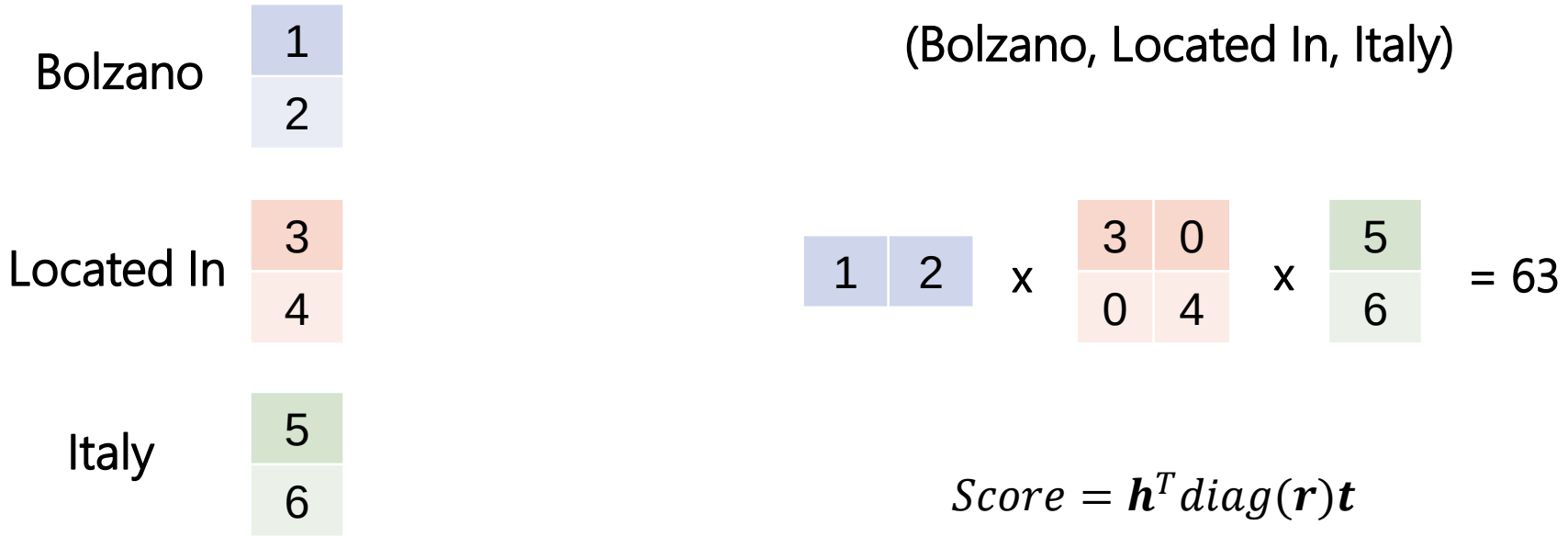
KGE Models: Semantic

Semantic models try to capture **latent semantics** by maximizing the plausibility of a triple. This is achieved by designing loss functions that typically rely on **multiplications**. This is the same principle used in **Neural Networks**, where we design the architecture and rely on the model to learn hidden knowledge.



KGE Models: Semantic

One of the simples models is **DistMult**, where the plausibility of the triple is obtained by just multiplying the **embeddings**.



KGE Models: Semantic

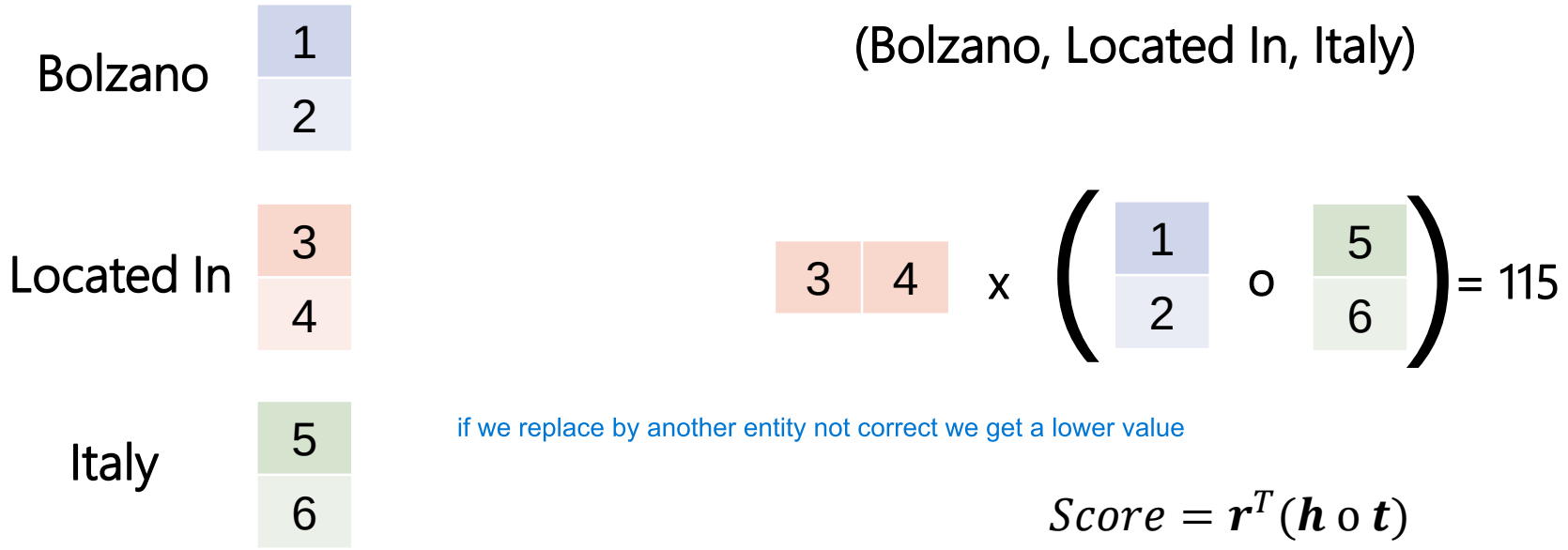
One of the simples models is **DistMult**, where the plausibility of the triple is obtained by just multiplying the **embeddings**. It is extended by **ComplEx** by treating the embeddings as complex values.

Bolzano	1+2i
	2+3i
Located In	3+5i
	4+7i
Italy	5+i
	6+2i

$$Score = Re(\boldsymbol{h}^T diag(\boldsymbol{r})\bar{\boldsymbol{t}})$$

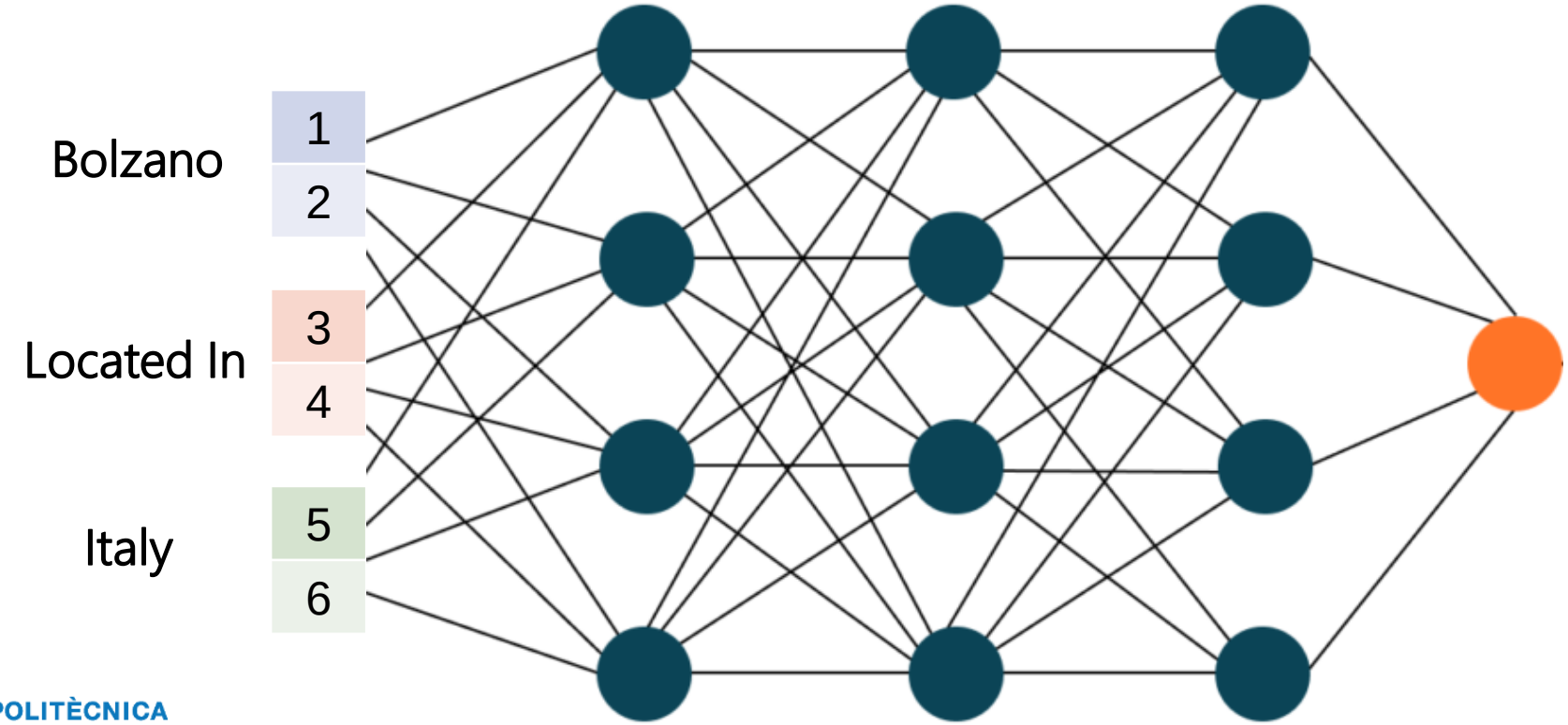
KGE Models: Semantic

One of the simples models is **DistMult**, where the plausibility of the triple is obtained by just multiplying the **embeddings**. It is extended by **ComplEx** by treating the embeddings as complex values. Or by changing how vectors are multiplied in **HolE**.



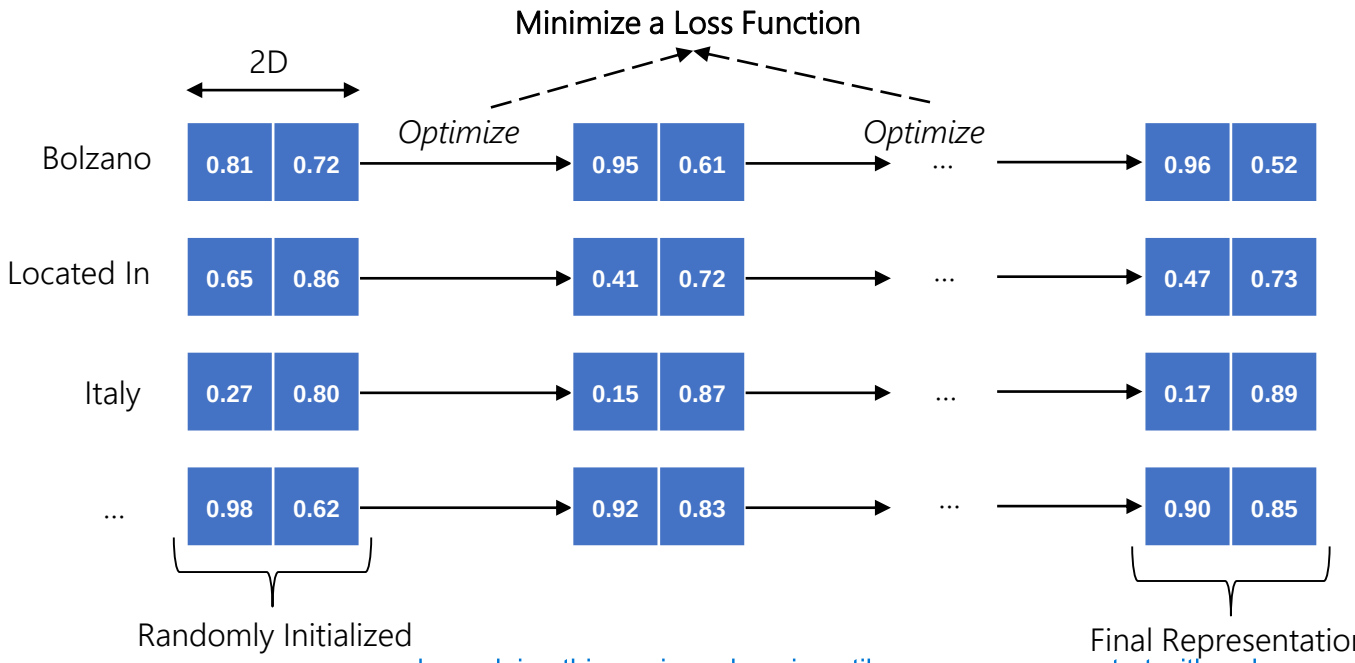
KGE Models: Semantic

However, the interesting part of semantic models is being able to leverage existing neural network architectures/components (e.g., 1D and 2D convolutions, activation functions, etc.), as in **ConvE** or **ConvKB**.



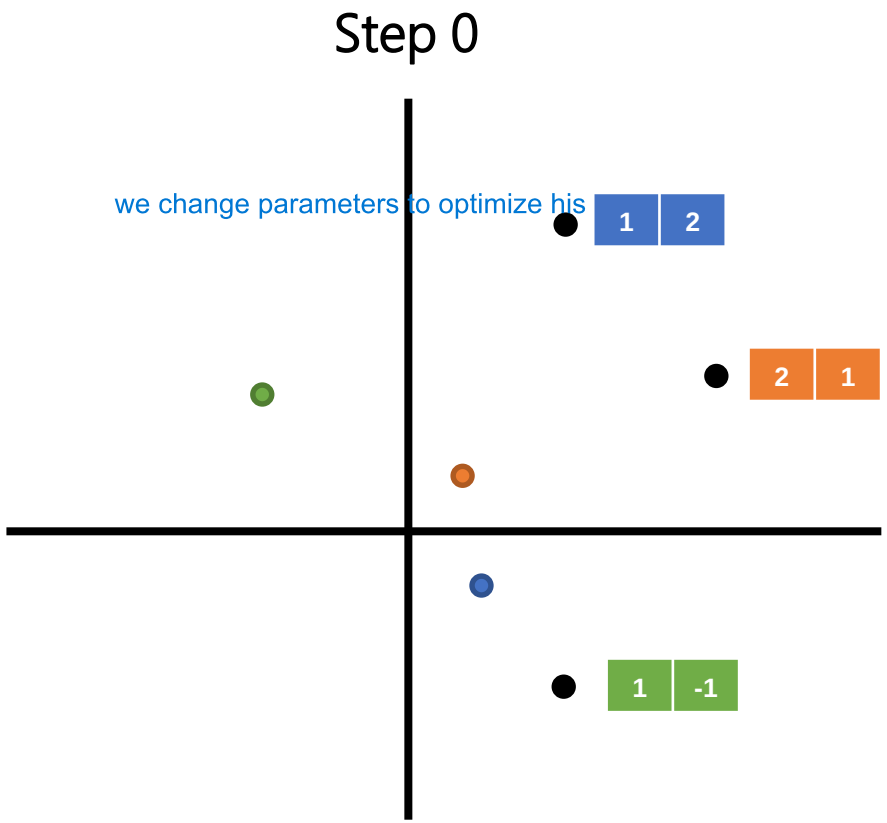
KGE: Training

In the KGE Models, the embeddings are treated as **parameters**, which are optimized based on a **loss function** defined by the KGE Model.



we keep doing this again and again until we converge, we start with radnom embeddings and then this are embeddings (parameters) are trained

KGE: Training



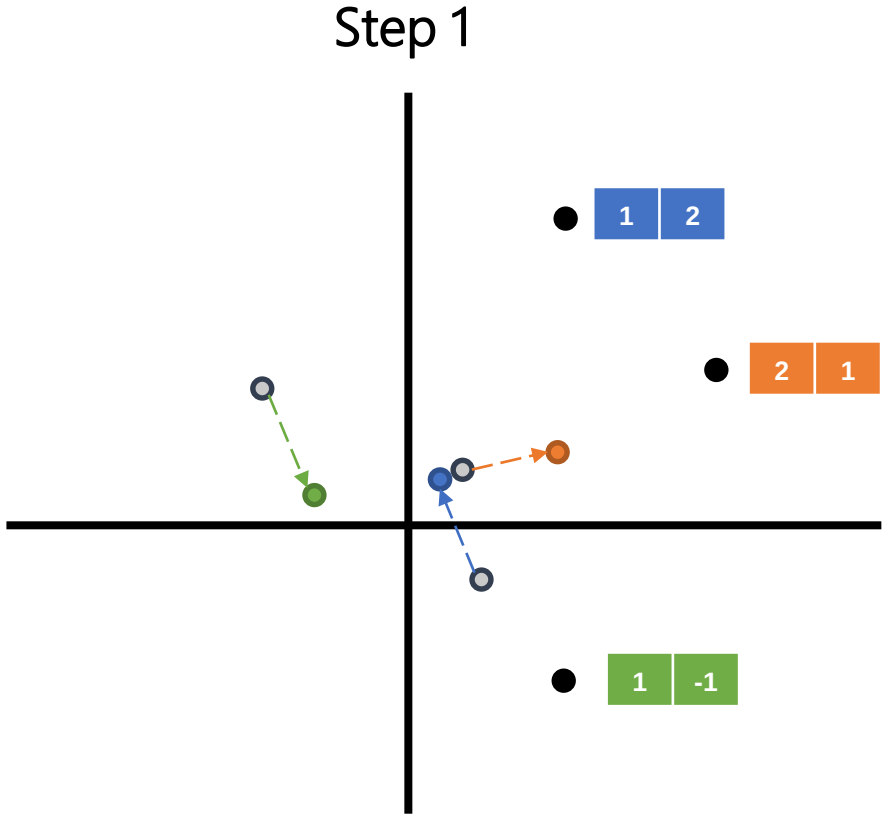
TransE: Loss= |head + relation - tail|

Loss= |(0.4,-0.4)+(-1,0.9) – (0.3,0.3)| = 0.92

Changes parameters that affect directly or indirectly the loss function

KGE: Training

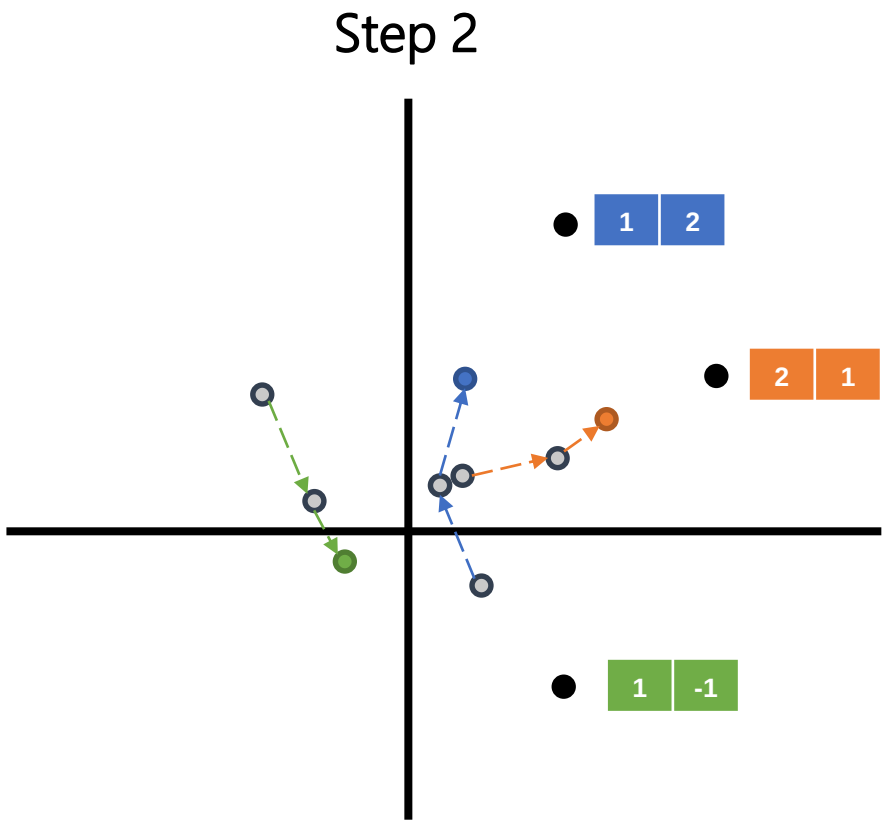
if we collapse embedding in 0 in trasnlational means 100% accuracy that it is also a problem



TransE: Loss= |head + relation - tail|

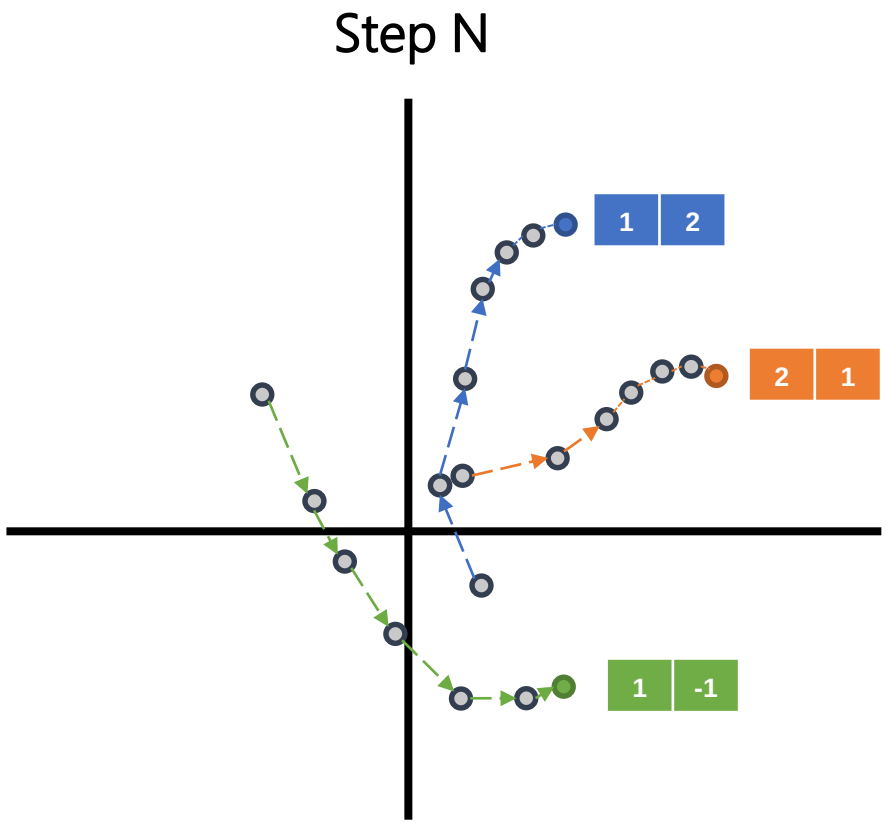
Loss= |(0.2,-0.2)+(-0.5,0.1) – (0.8,0.4)| = 0.70

KGE: Training



TransE: Loss= |head + relation - tail|

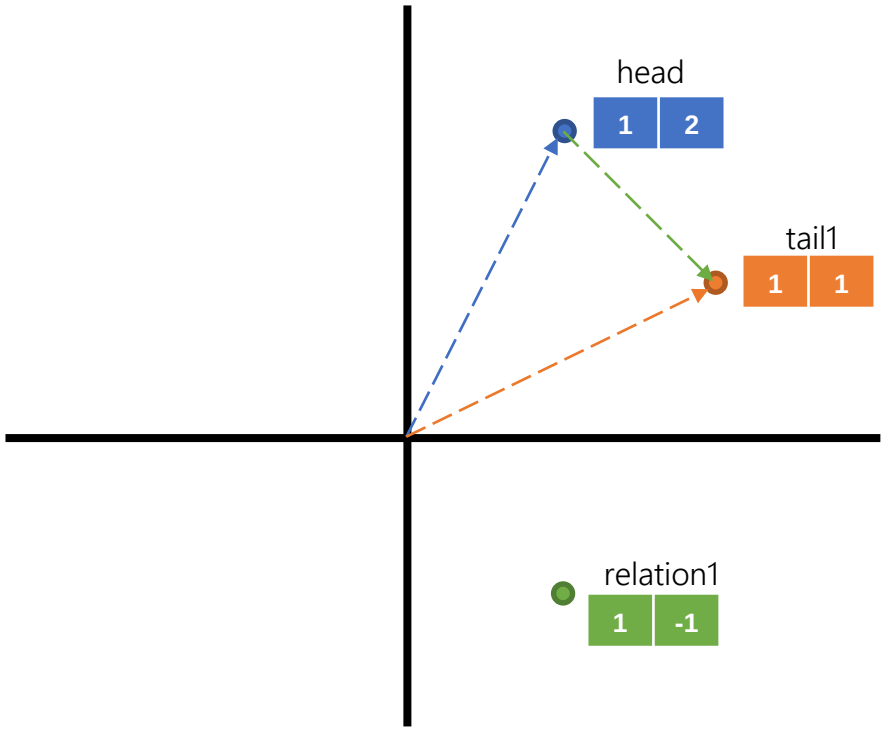
KGE: Training



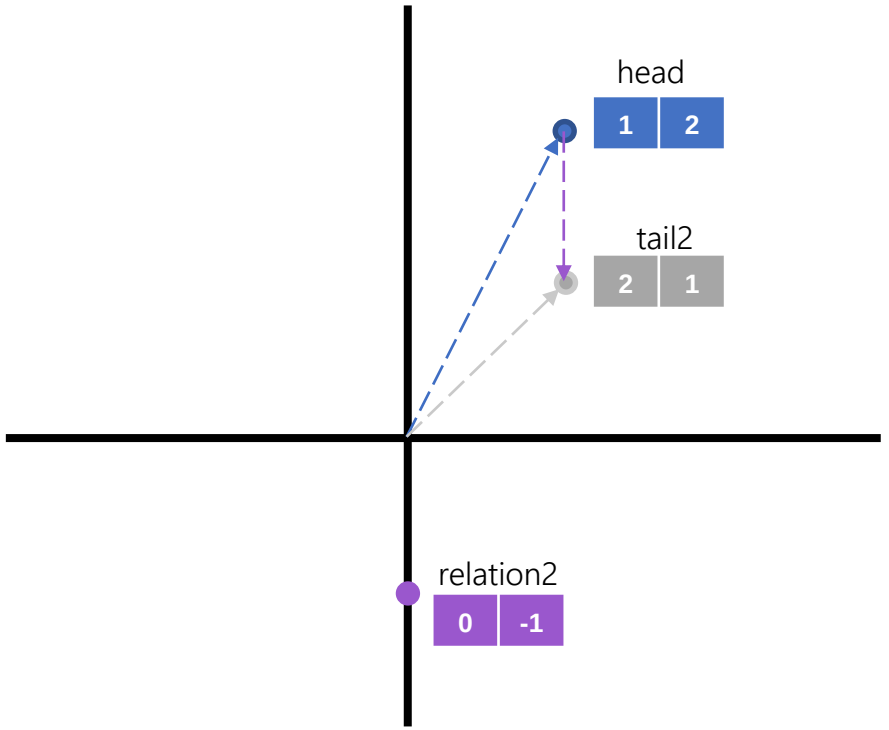
TransE: Loss= |head + relation - tail|

$Loss = |(1,2) + (1,-1) - (2,1)| = 0$

KGE: Training

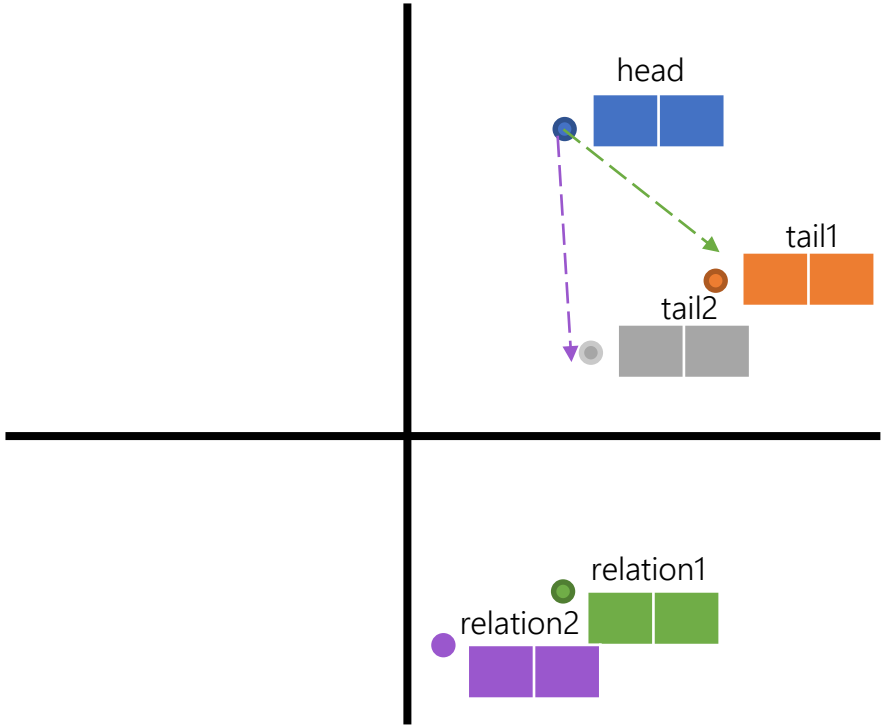


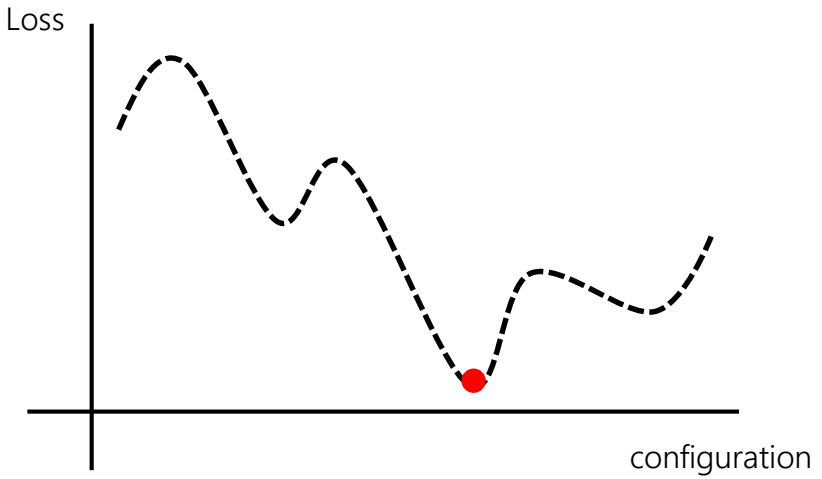
(head, relation1, tail1)



(head, relation2, tail2)

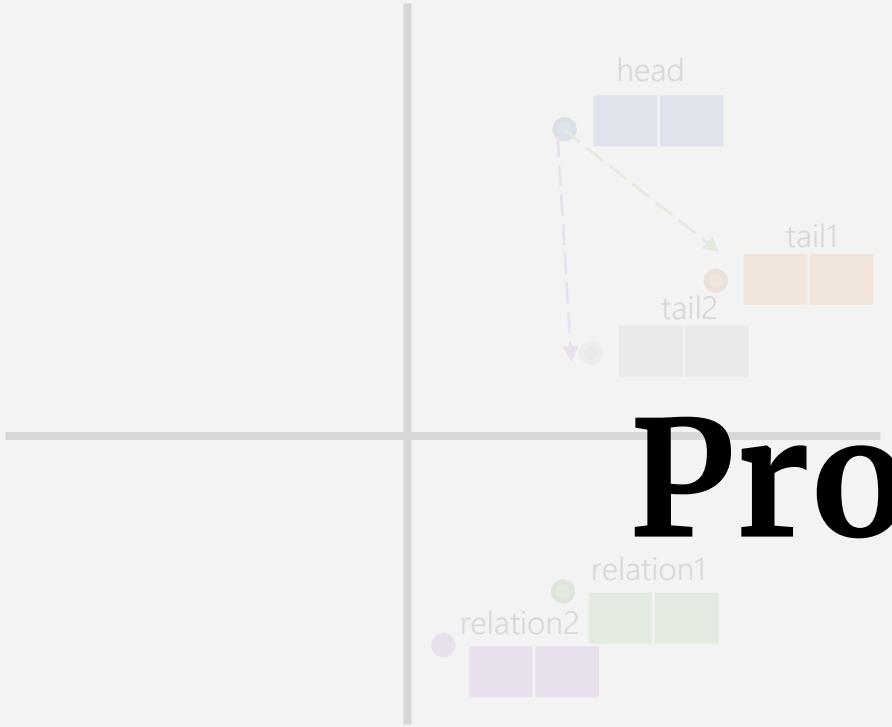
KGE: Training

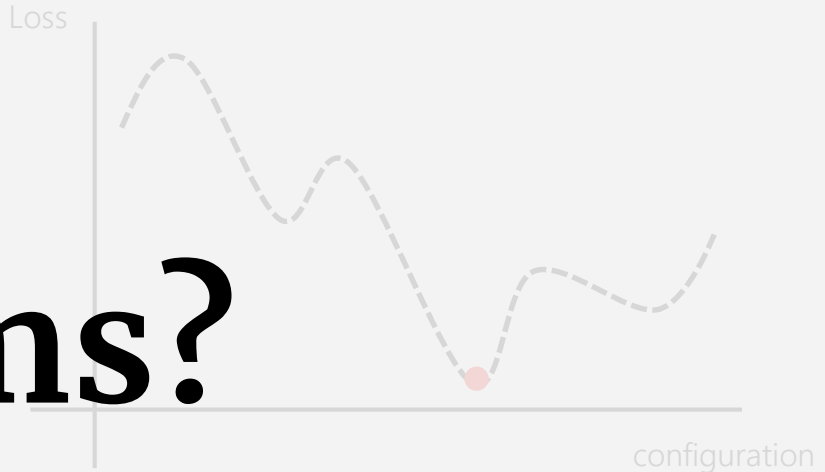




(head, relation1, tail1)

(head, relation2, tail2)





Any Problems?

(head, relation1, tail1)

(head, relation2, tail2)

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **degenerate solutions** would be obtained.

Translational

$|\text{embedding}| \approx 0$

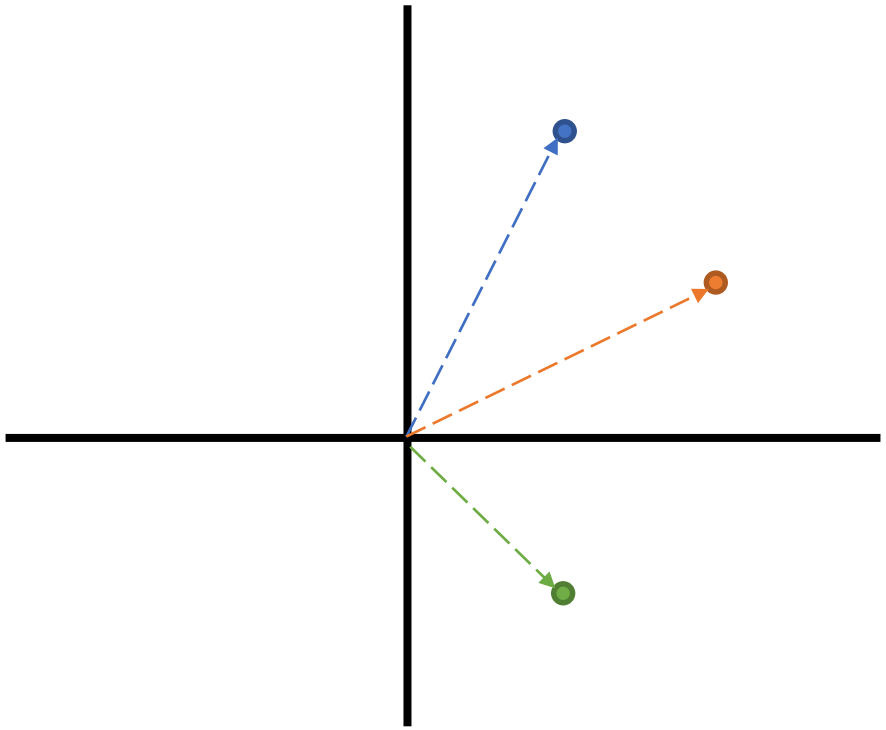
Semantic

$|\text{embedding}| \approx \infty$

$|\text{embedding}| \approx c$

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



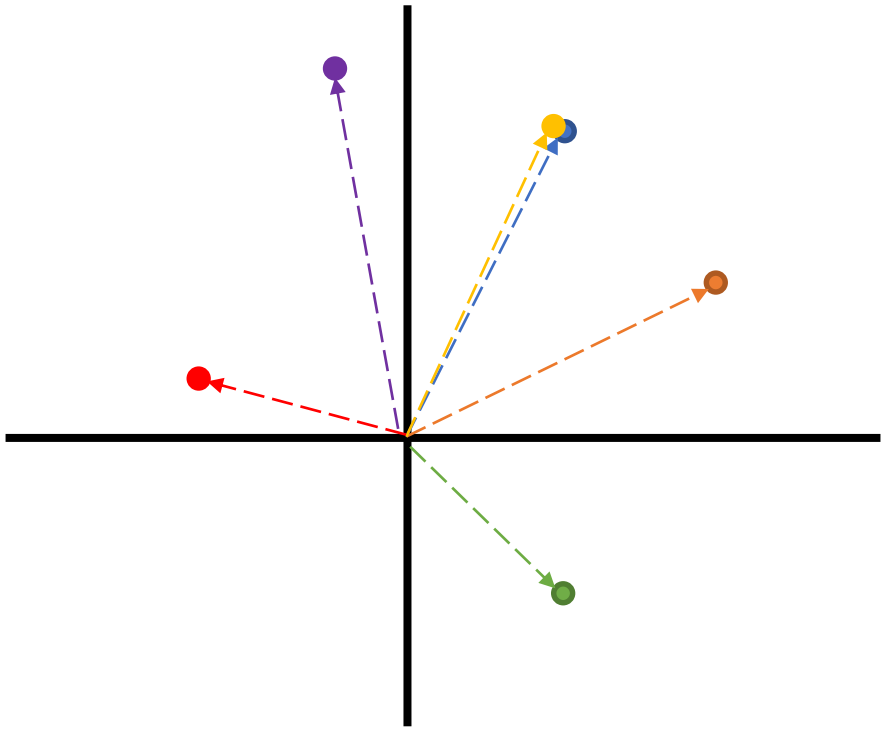
Bolzano

located_in

Italy

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



Bolzano

located_in

Italy

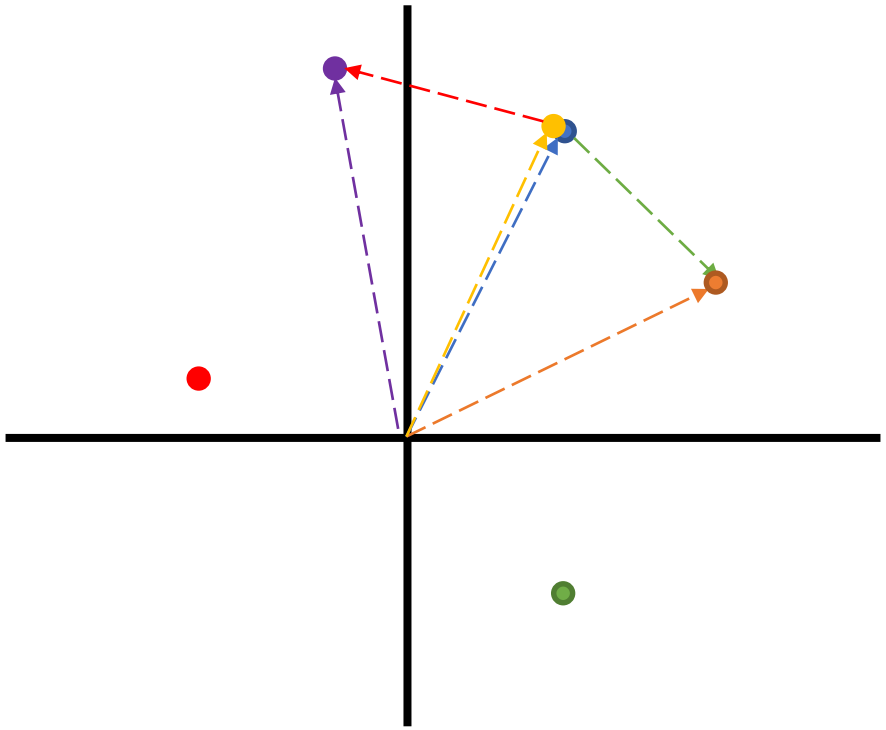
Barcelona

has_monument

S. Família

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



Bolzano

located_in

Italy

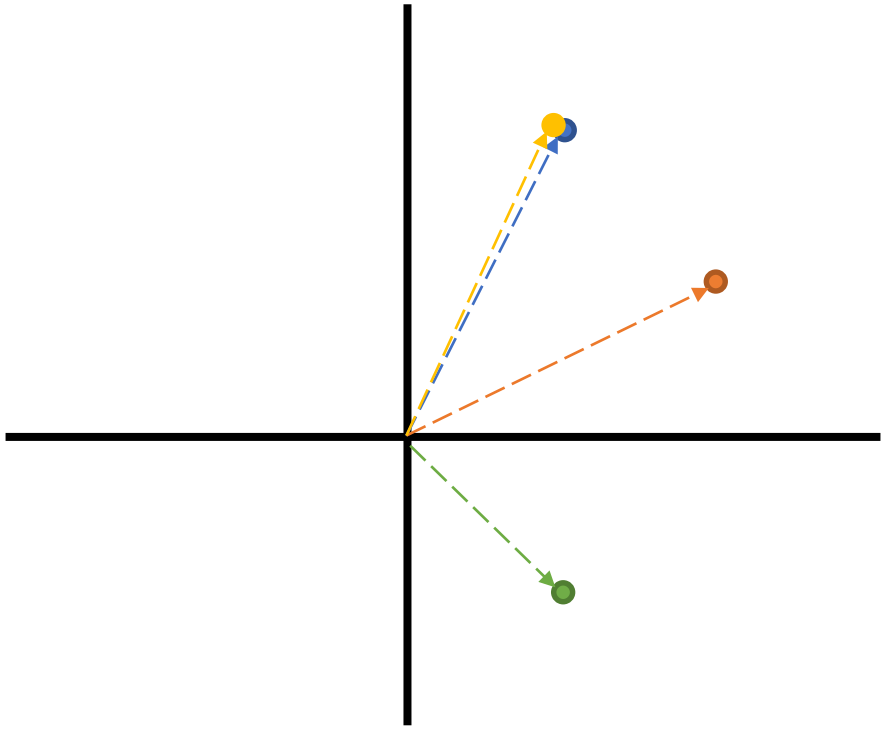
Barcelona

has_monument

S. Família

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



Bolzano

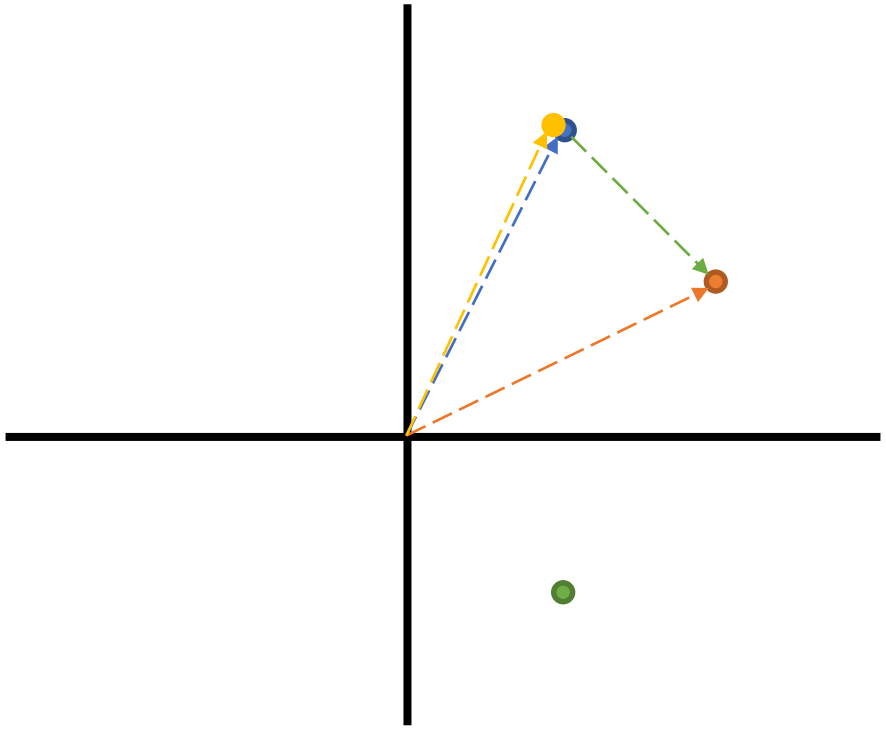
located_in

Italy

Barcelona

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



Bolzano

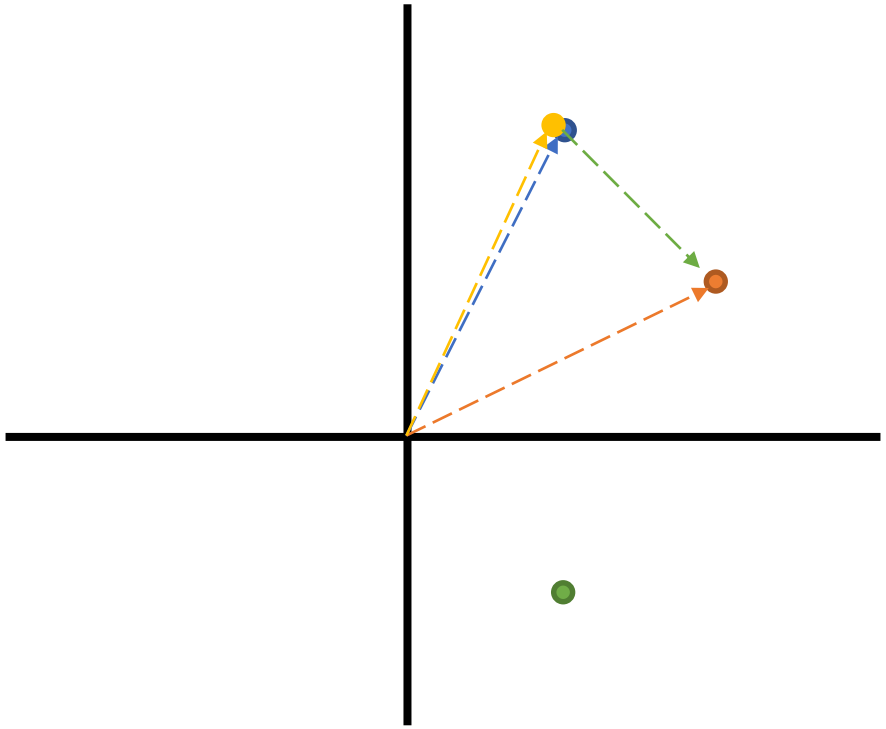
located_in

Italy

Barcelona

KGE: Training

If we trained KGE by simply optimizing the KGE model's assumptions, **incorrect information** could be learned.



Bolzano

located_in

Italy

Barcelona

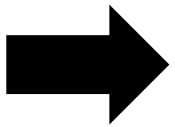
KGE: Training

To mitigate this issue, loss functions based on **Negative Sampling** are used when training KGEs.

we detect good and bad things at the same time

*Given a positive triple (h, r, t), a negative triple is created by **replacing** the head or tail entity by any other entity of the KG, originating (h',r,t) or (h,r,t'), such that the negative triple **does not exist** in the original KG.*

(Bolzano, located_in, Italy)



(Barcelona, located_in, Italy) neg value

(Bolzano, located_in, Barcelona)

(Bolzano, located_in, S.Família)

~~**(Rome, located_in, Italy)**~~

KGE: Training

The **Margin-Base Pairwise Loss** is based on assigning **higher plausibility** to positive triple than to a negative triple.

Translational

Plausibility = minimizing the translational distance

Semantic

Plausibility = maximizing the score

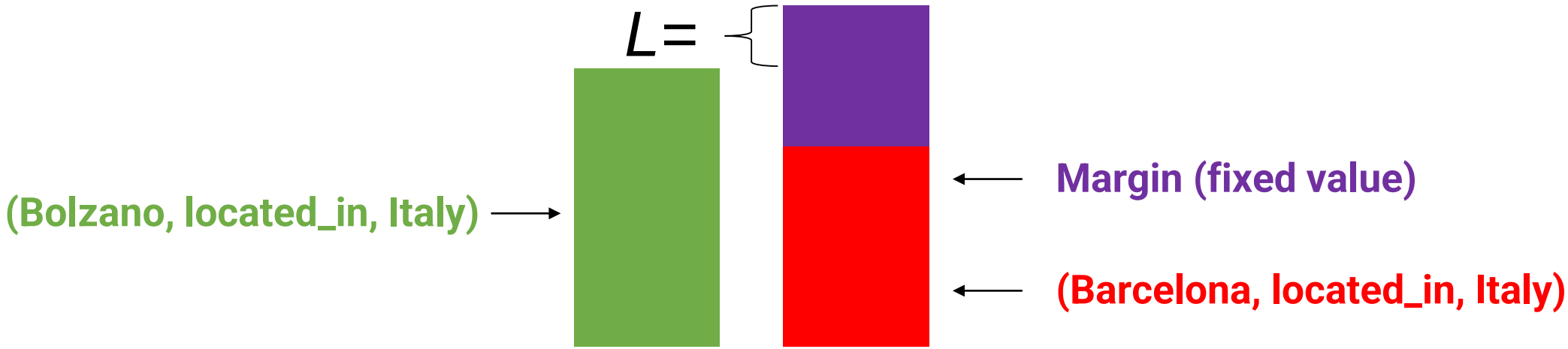
KGE: Training

We want the score/plausibility given to the positive triple to be **greater than** the one of the negative triple by a **margin**.

$L=0$

KGE: Training

We want the score/plausibility given to the positive triple to be **greater than** the one of the negative triple by a **margin**.



KGE: Training

To mitigate this issue, loss functions based on **Negative Sampling** are used when training KGEs.

$$Loss\_1 = -Score(Positive\ Triple)$$

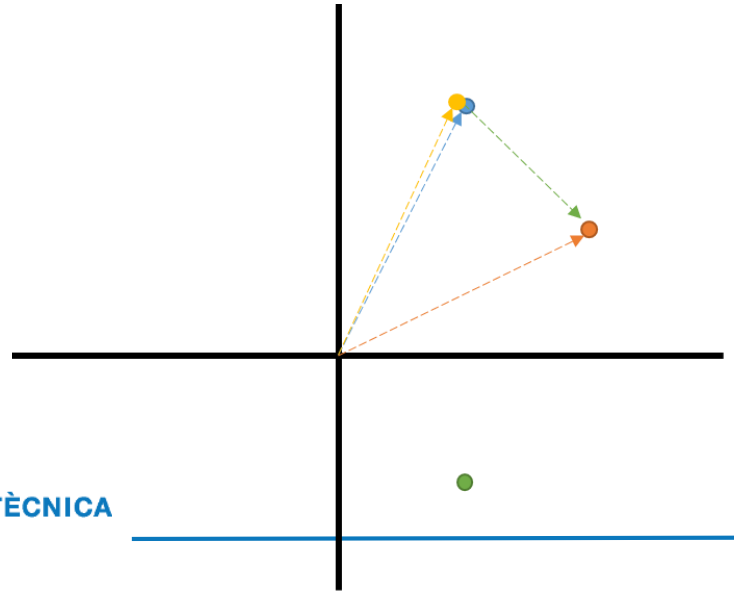
$$Loss\_2 = max\{-Score(Positive\ Triple) + Score(Negative\ Triple) + margin, 0\}$$

The higher the score, the more probable a triple is.

hyperparameter

Different generation mechanisms

$(h,r,t) \rightarrow (h',r,t)$

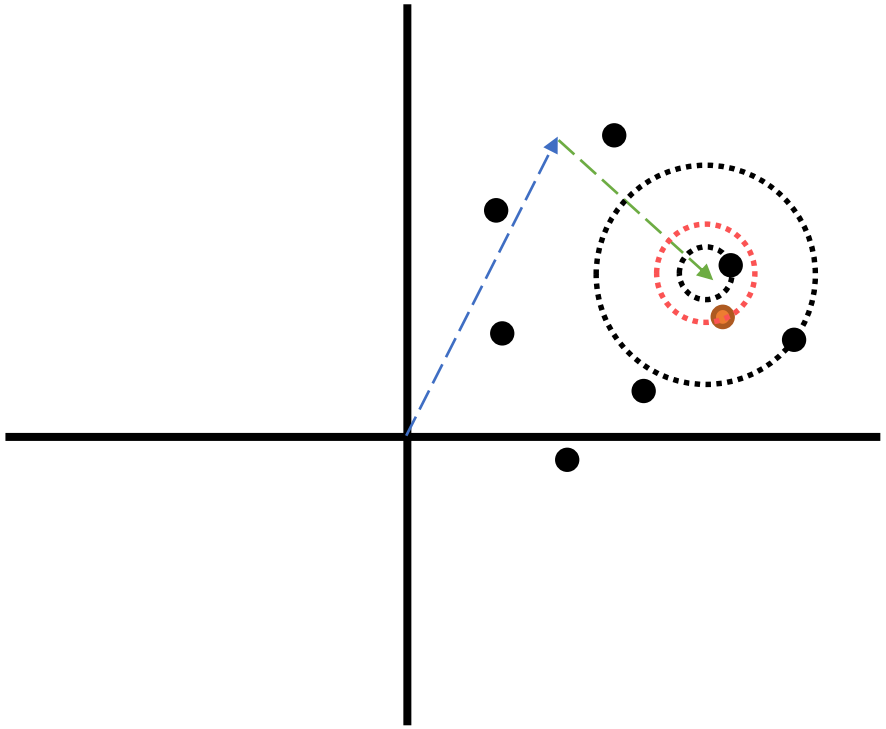


Loss_1 (head, relation, tail) ↓

Loss_2 (head, relation, tail) ↑ (head, relation, tail)

KGE: Evaluation

Hits@3



TEST SET

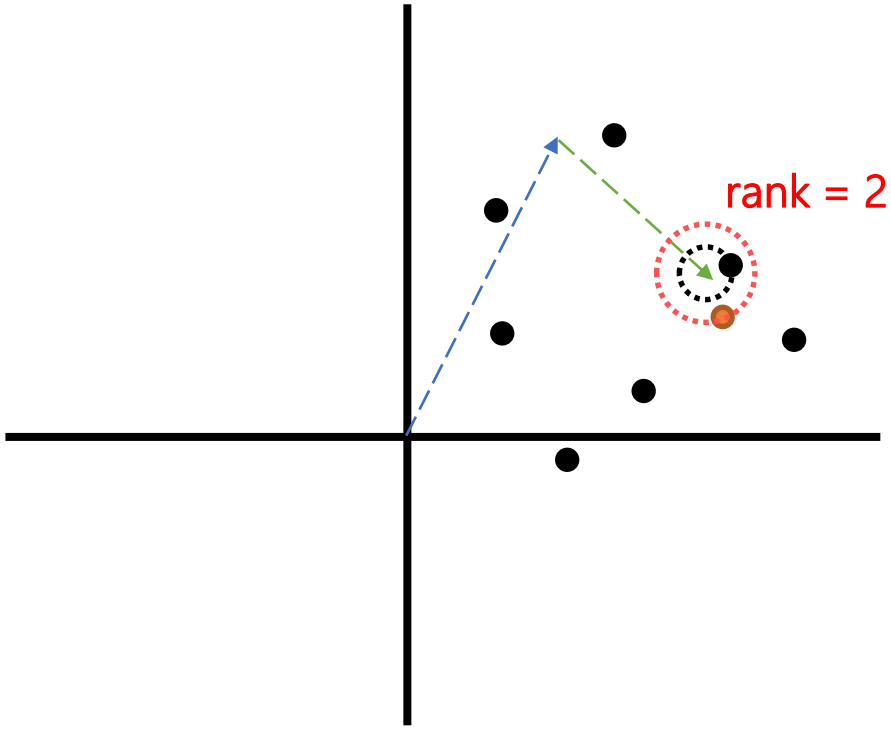
(head,relation,tail) ⟶ (head,relation,?)

$$tail \approx head + relation$$

Hits@3: Proportion of test triples in which the correct missing entity is the first, second or third "closest" entity.

KGE: Evaluation

Mean Reciprocal Rank (MRR)



TEST SET

(head,relation,tail) ⟶ (head,relation,?)

$$tail \approx head + relation$$

$$MRR = \frac{1}{|N|}\sum_{i=1}^{N}\frac{1}{rank_i}$$

KGE: Evaluation

Usually, training KGE involves testing over **multiple KGE models**, as depending on the KG the performance across them will differ.

FB15K237

Model	Loss	Training Approach	Inverse Relations	Hits@10 (%)
ComplEx	CEL	LCWA	True	44.838
ConvE	BCEL	LCWA	True	49.212
ConvKB	SPL	sLCWA	False	32.261
DistMult	CEL	LCWA	True	47.387
ERMLP	BCEL	LCWA	True	45.100
HolE	CEL	LCWA	True	42.225
KG2E	SPL	LCWA	True	45.501
MuRE	BCEL	LCWA	True	47.199
NTN	SPL	sLCWA	False	20.342
ProjE	BCEL	LCWA	True	41.616
QuatE	CEL	LCWA	True	46.166
RESCAL	CEL	LCWA	True	46.460
RotatE	NSSAL	sLCWA	False	49.750
SE	NSSAL	sLCWA	True	39.427
SimplE	CEL	LCWA	True	40.307
TransD	MRL	sLCWA	True	41.856
TransE	MRL	sLCWA	False	46.423
TransH	MRL	sLCWA	False	35.295
TransR	CEL	LCWA	True	39.187
TuckER	BCEL	LCWA	True	52.857
UM	CEL	LCWA	False	8.024

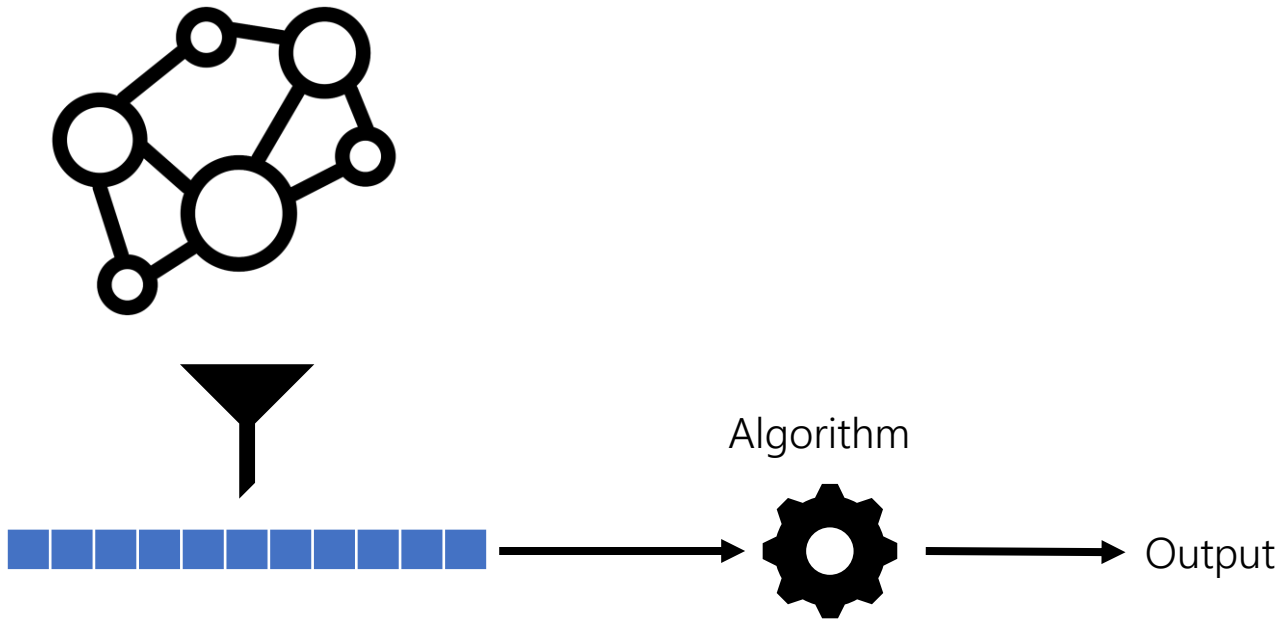
WN18RR

Model	Loss	Training Approach	Inverse Relations	Hits@10 (%)
ComplEx	CEL	LCWA	False	53.745
ConvE	CEL	LCWA	True	56.327
ConvKB	NSSAL	sLCWA	True	42.083
DistMult	CEL	LCWA	True	52.616
ERMLP	SPL	sLCWA	True	47.657
HolE	CEL	LCWA	False	50.017
KG2E	SPL	LCWA	False	52.035
MuRE	SPL	LCWA	True	57.900
NTN	MRL	sLCWA	False	31.857
ProjE	CEL	LCWA	True	51.727
QuatE	CEL	LCWA	False	55.010
RESCAL	CEL	LCWA	False	53.916
RotatE	BCEL	LCWA	True	60.089
SE	SPL	sLCWA	False	45.486
SimplE	CEL	LCWA	True	50.889
TransD	MRL	sLCWA	False	46.546
TransE	SPL	LCWA	False	56.977
TransH	MRL	sLCWA	False	48.170
TransR	MRL	sLCWA	False	42.510
TuckER	CEL	LCWA	True	56.088
UM	SPL	LCWA	False	44.887

Ali, Mehdi, et al. "Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework." IEEE Transactions on Pattern Analysis and Machine Intelligence 44.12 (2021)

KGE: Applications

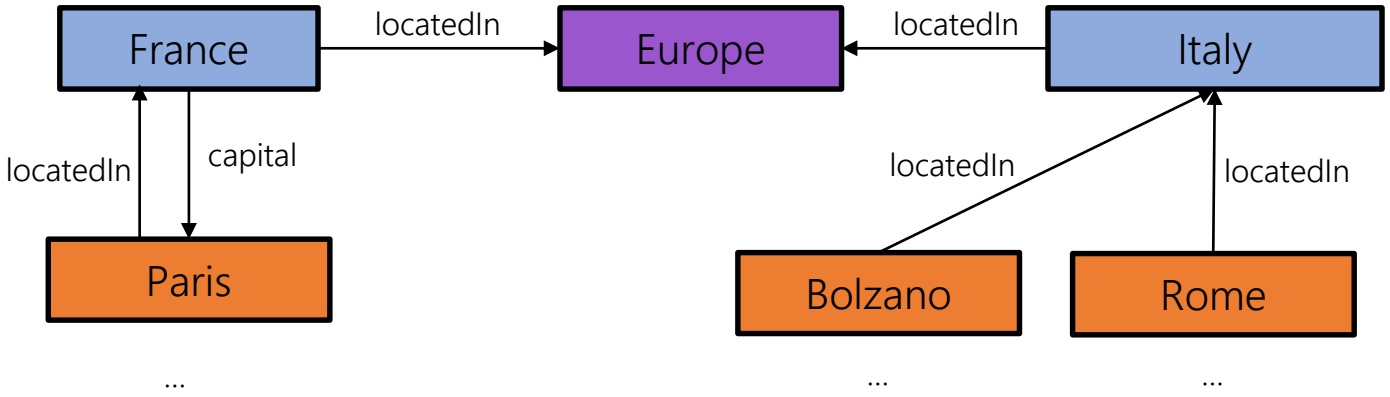
Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…).

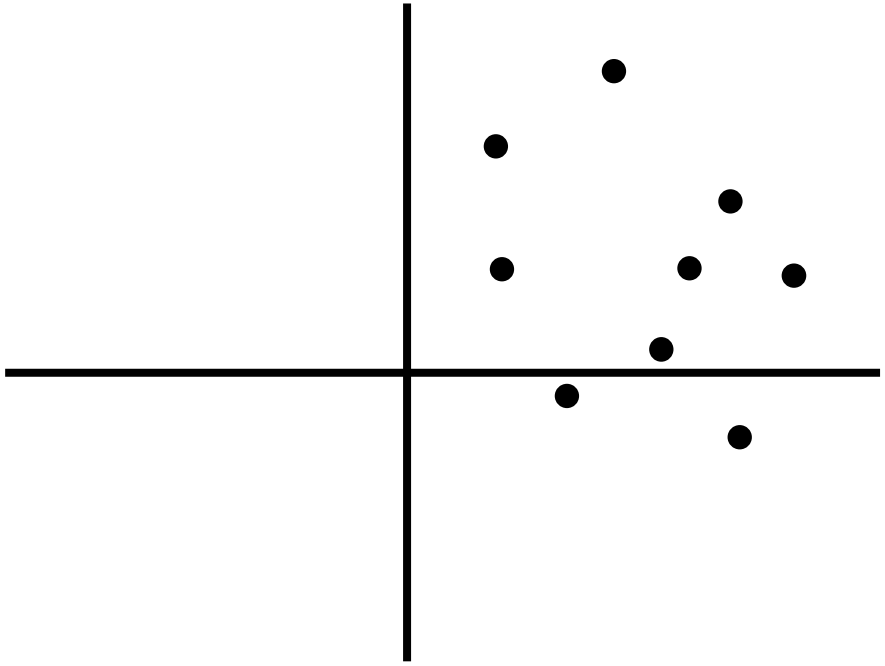


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Knowledge Graph Completion

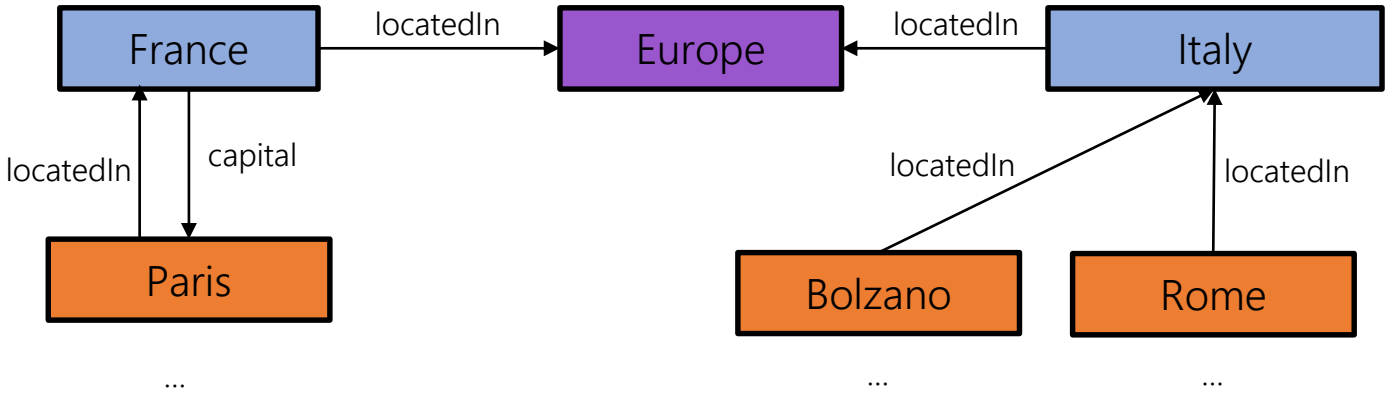


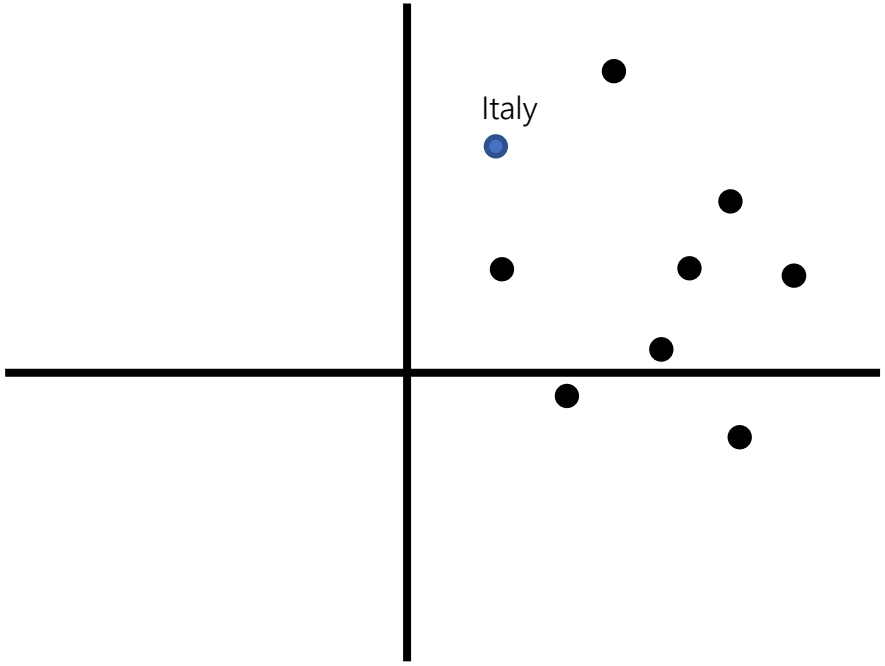


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Knowledge Graph Completion

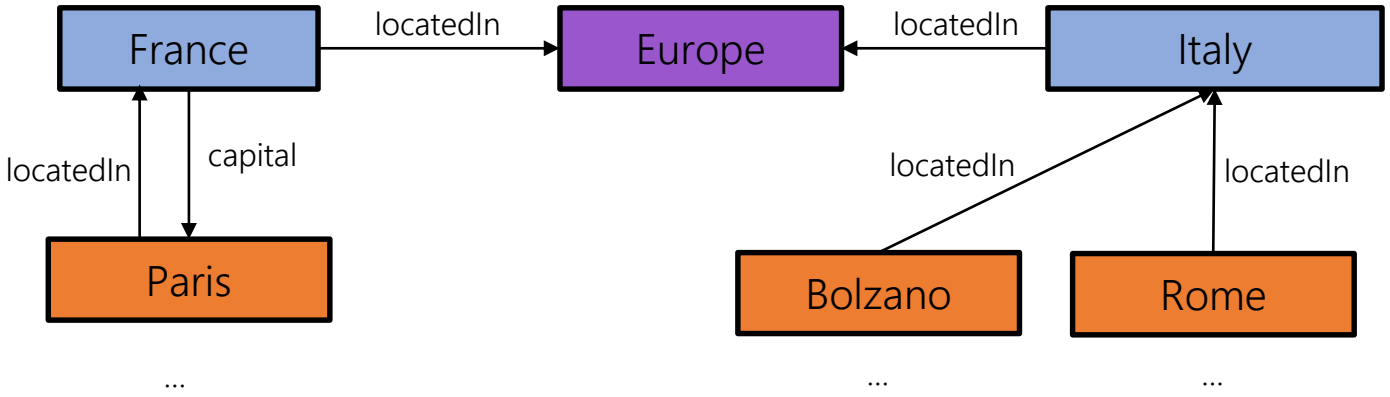


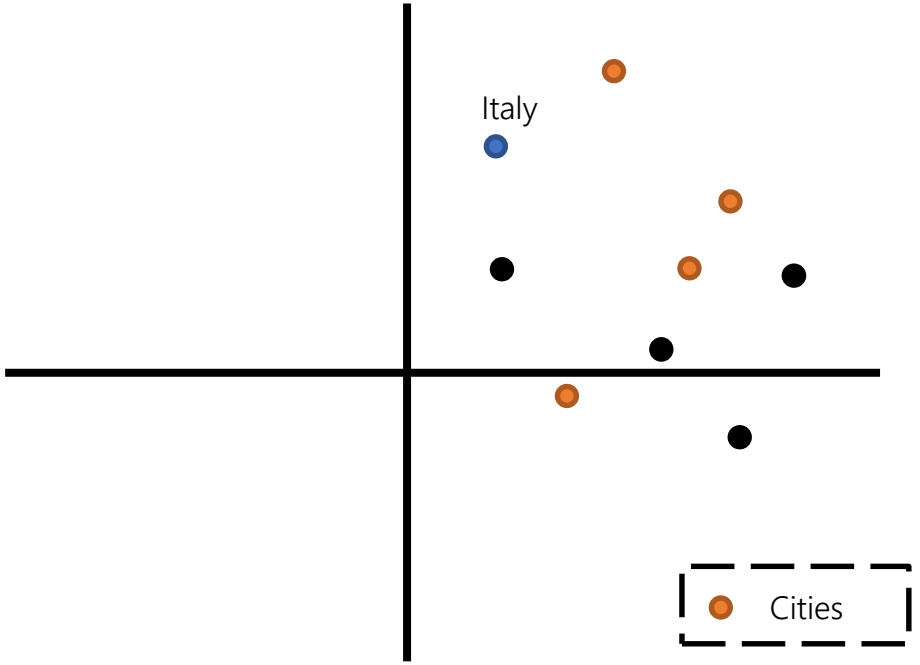


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Knowledge Graph Completion

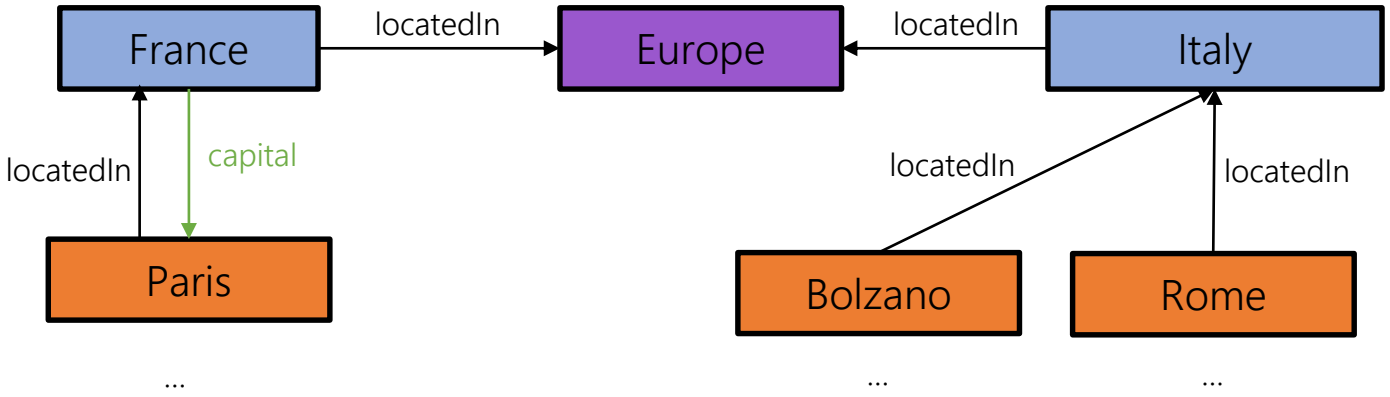


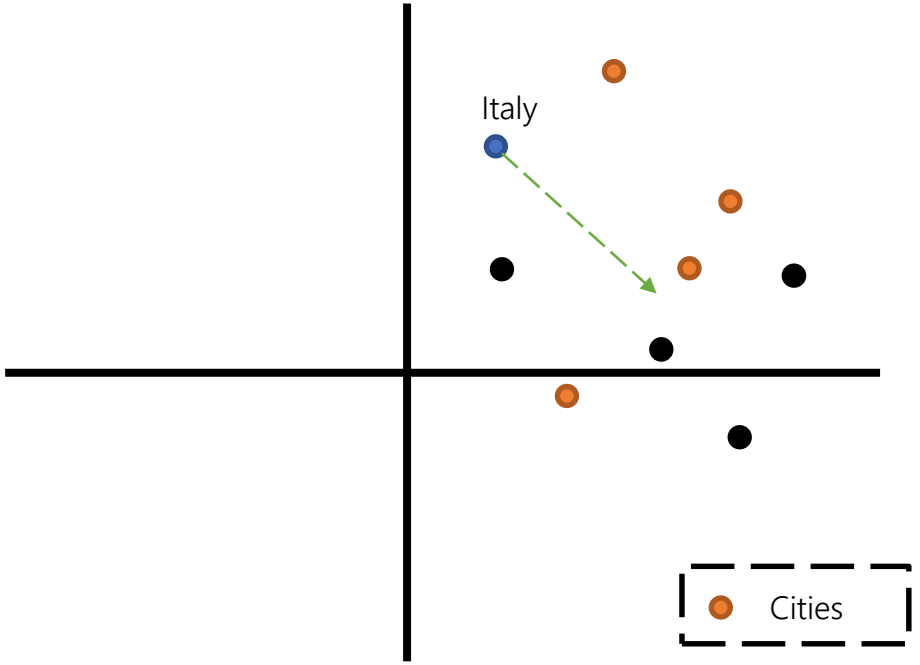


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Knowledge Graph Completion

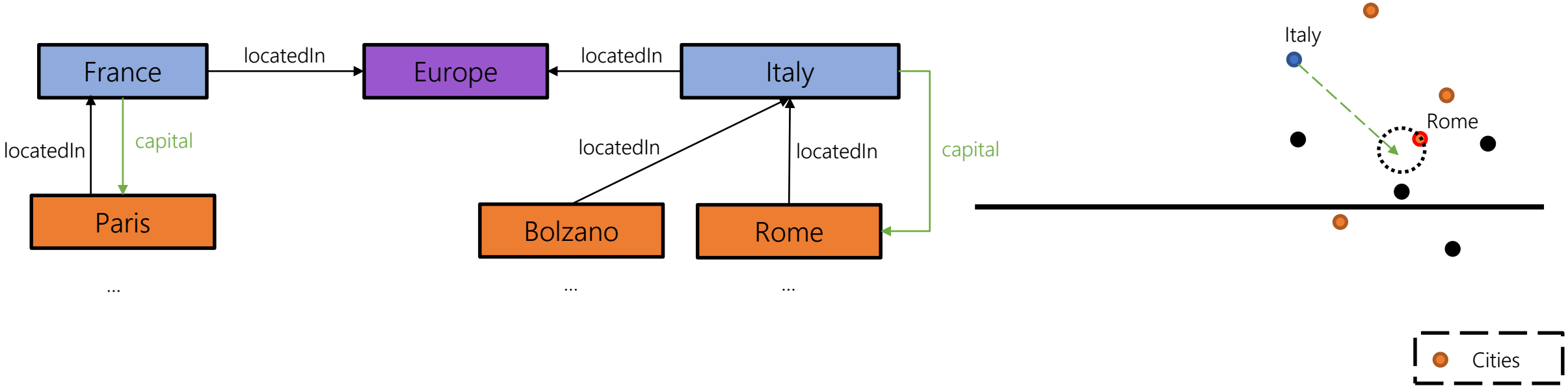




KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

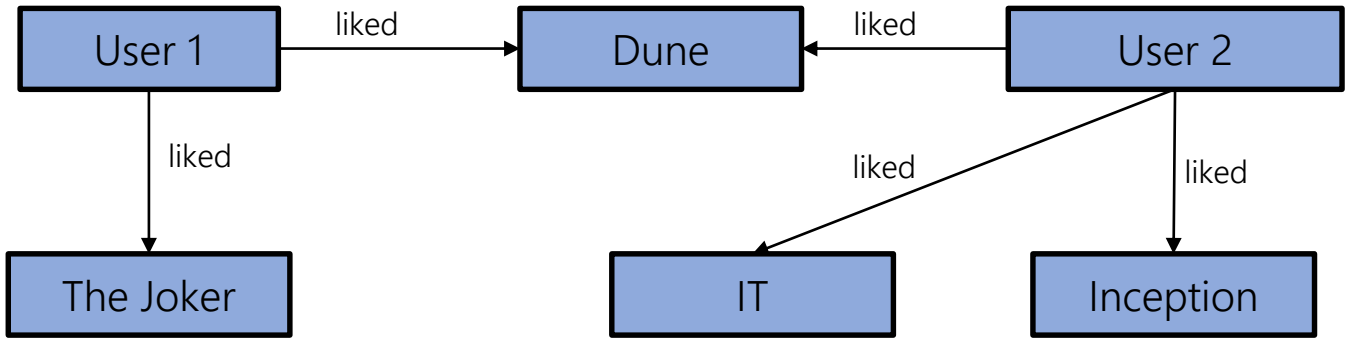
Knowledge Graph Completion

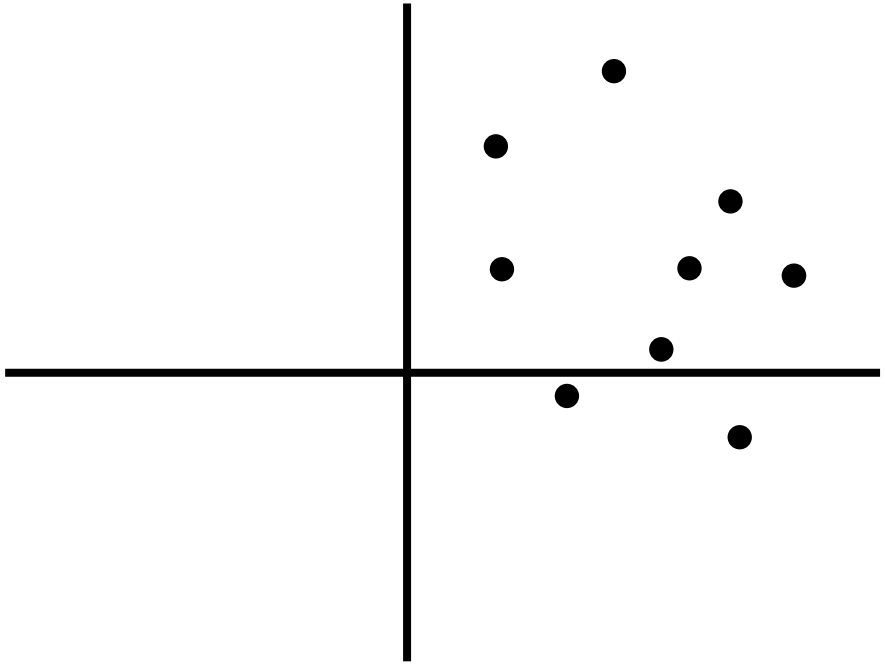


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Recommender Systems

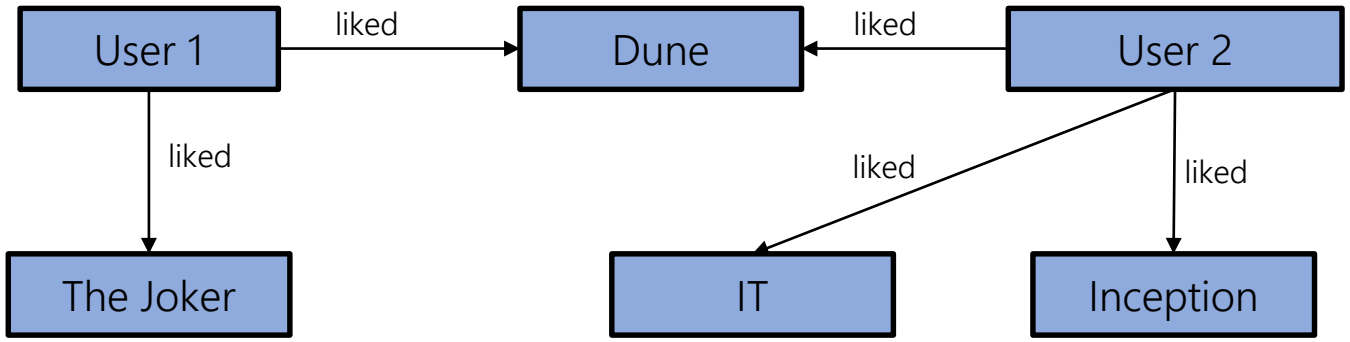


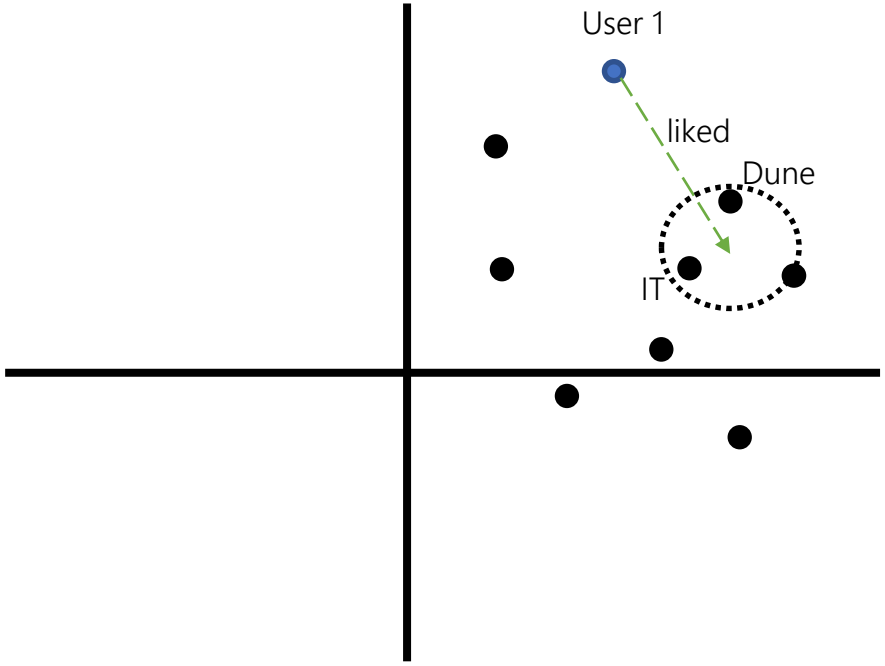


KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Recommender Systems





KGE: Applications

Generating KGE allows to use well-know **machine learning** models over KG (e.g., classification, clustering,…). However, we can also use them directly:

Recommender Systems